Work Agents:

Systems, Capabilities, and Optimization

Zishan Xu[1,*], **Enjun Du**[2,3,*], **Mingqi Gao**[4,*], **Cheng Yang**[3], **Junhao Shen**[5], **Zirong Chen**[4], **Zhiyuan Yao**[6], **Yifu Guo**[7], **Yuquan Lu**[7], **Jinyang Huang**[8], **Zhengxi Lu**[6], **Siyi Liu**[3], **Ziyu Zheng**[2], **Yongqi Zhang**[3,†], **Difan Zou**[2,†], **Weiwen Liu**[1,†]

[1]Shanghai Jiao Tong University [2]The University of Hong Kong
[3]The Hong Kong University of Science and Technology (Guangzhou) [4]Tsinghua University
[5]The Chinese University of Hong Kong [6]Zhejiang University [7]Sun Yat-sen University [8]Central South University
[*]Equal contribution, [†]Corresponding authors

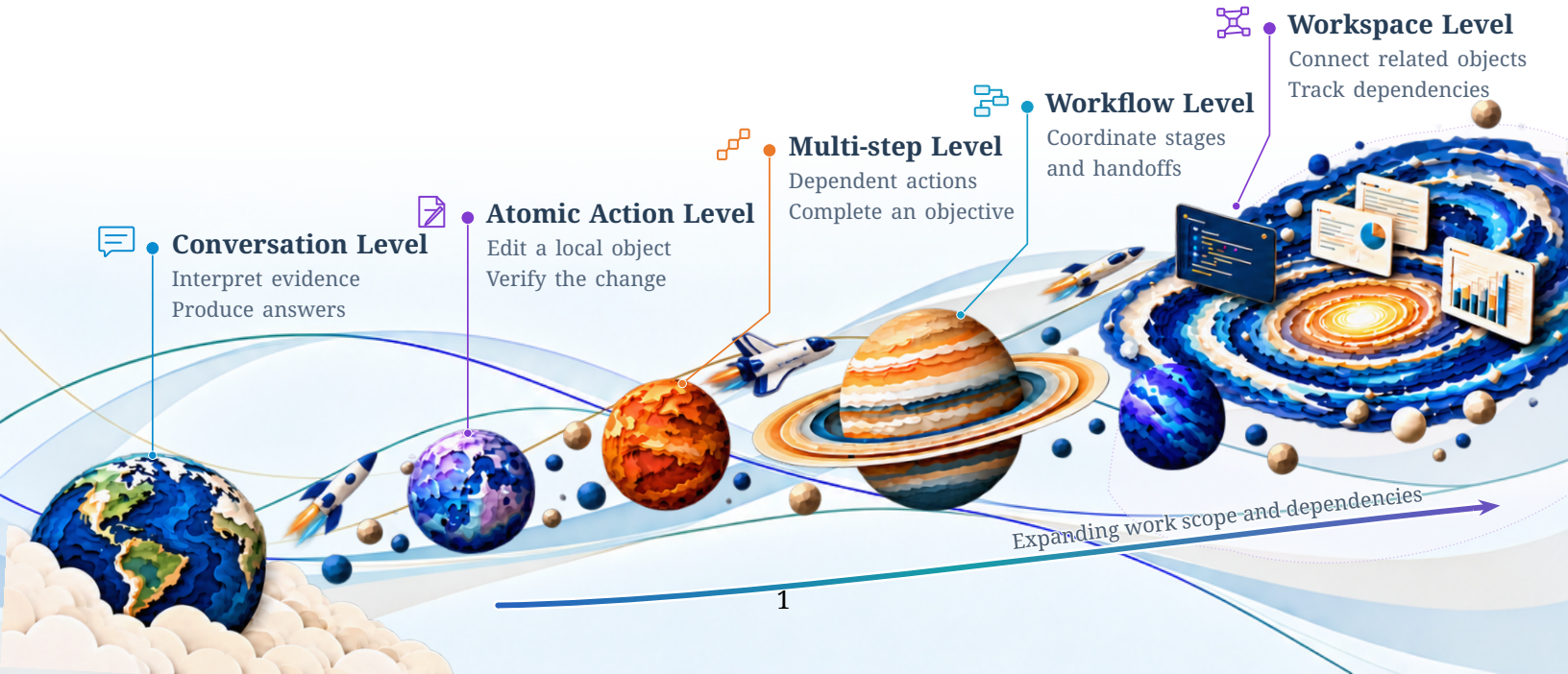
yongqizhang@hkust-gz.edu.cn dzou@cs.hku.hk wwliu@sjtu.edu.cn

Awesome Work Agents | Project Homepage

Abstract

As foundation models evolve from information generation to digital work execution, agents are increasingly capable of completing complex tasks across tools, applications, and workflows. However, existing research lacks a unified perspective on how agents reliably accomplish delegated work. To address this gap, we present a work-centric, system-level perspective that views work agents as systems combining foundation models and runtime harnesses to execute tasks in stateful workspaces and deliver verifiable outcomes. We introduce two complementary dimensions, work persistence and work openness, to characterize working conditions, and classify digital work into five forms according to task scope and organization. Building on this framework, we identify five core capability families required for reliable work execution and examine approaches to model optimization, harness optimization, and their joint optimization. We further analyze existing evaluation methods and metrics from the perspective of work outcomes. Finally, we discuss future research directions, including sustained work delegation, adaptation to open environments, and the interaction between work execution and capability improvement. This paper provides a unified framework for understanding, optimizing, and evaluating work agents, supporting the development of systems capable of delivering reliable, verifiable, and practically valuable work outcomes.

Keywords: Work Agents; Work Persistence and Openness; Agent Capabilities; Agent Optimization



Contents

1 Introduction . . . 4

2 Foundation of Work Agents . . . 5
- 2.1 Definition of Work Agents . . . 6
- 2.2 Working Conditions . . . 7
  - 2.2.1 Work Persistence . . . 7
  - 2.2.2 Work Openness . . . 8
  - 2.2.3 Combining Persistence and Openness . . . 9
- 2.3 Core Capabilities of Work Agents . . . 9
  - 2.3.1 Perception and Understanding . . . 11
  - 2.3.2 Reasoning and Decision Making . . . 12
  - 2.3.3 Planning and Coordination . . . 12
  - 2.3.4 Execution and State Management . . . 13
  - 2.3.5 Verification, Recovery, and Replanning . . . 13
- 2.4 Work Agents and Related Agent Paradigms . . . 14
  - 2.4.1 Long-Horizon Agents . . . 14
  - 2.4.2 Computer-Use Agents . . . 15
  - 2.4.3 Office Agents . . . 15
  - 2.4.4 Relationships among Paradigms . . . 15

3 Work Forms: What Work Do Agents Undertake? . . . 17
- 3.1 Conversation Level . . . 17
  - 3.1.1 Producing Informational Outcomes . . . 17
  - 3.1.2 Interpreting Evidence and Deriving Responses . . . 17
- 3.2 Atomic Action Level . . . 19
  - 3.2.1 Creating and Modifying Local Work Objects . . . 19
  - 3.2.2 Executing and Checking a Local Change . . . 19
- 3.3 Multi-step Level . . . 20
  - 3.3.1 Completing Objectives through Intermediate Results . . . 20
  - 3.3.2 Managing Action Dependencies and Feedback . . . 20
- 3.4 Workflow Level . . . 21
  - 3.4.1 Carrying Work through Functional Stages . . . 21
  - 3.4.2 Coordinating Workflow Progress . . . 21
- 3.5 Workspace Level . . . 22
  - 3.5.1 Locating Relevant Objects in a Workspace . . . 22
  - 3.5.2 Following Dependencies across Workspace Objects . . . 22
- 3.6 Synthesis: Work Scope and Capability Demands . . . 22

4 Optimization: From Work Experience to System Capability . . . 23
- 4.1 Training Environments, Data, and Rewards . . . 23
  - 4.1.1 Work Environment Construction . . . 25
  - 4.1.2 Corpus and Trajectory Curation . . . 27
  - 4.1.3 Feedback Design . . . 27
- 4.2 Model Optimization . . . 28
  - 4.2.1 Pre-training . . . 28
  - 4.2.2 Mid-training . . . 29
  - 4.2.3 Post-training . . . 30
- 4.3 Harness Optimization . . . 33
  - 4.3.1 Observation and Action Interfaces . . . 34
  - 4.3.2 Workflows and Execution Control . . . 34
  - 4.3.3 Long Context, Memory and Skills . . . 35
- 4.4 Model–Harness Coupling and Joint Optimization . . . 35

4.4.1 Harness-Aware Policy Training . . . 37
4.4.2 Joint Model–Harness Adaptation . . . 37
4.4.3 Self-Improvement and Recursive Self-Improvement . . . 38

5 Evaluation and Resources: Assessing Work Agents . . . 39
5.1 Benchmarks and Task Construction . . . 40
5.1.1 Task Construction Strategies . . . 40
5.1.2 Domain Coverage and Work Requirements . . . 42
5.2 Evaluation Methodologies . . . 42
5.2.1 Evaluation Targets and Evaluation Dimensions . . . 42
5.2.2 Evaluation Methods . . . 44
5.3 Evaluation Metrics . . . 49
5.4 Open-Source Harnesses and Execution Environments . . . 50
5.5 Performance, Cost, and Economic Value . . . 52
5.5.1 Trading Performance against Execution Cost . . . 53
5.5.2 The Value of Professional Work . . . 53
5.5.3 Reliable Deployment and Agentic ROI . . . 53

6 Frontiers: Toward Next-Generation Work Agents . . . 53
6.1 Future Applications: From Assistance to Delegated Work . . . 54
6.1.1 Personal Work Agents . . . 54
6.1.2 Human Digital Twins for Professional Work . . . 55
6.1.3 Organizational Work Systems . . . 56
6.2 Technical Frontiers for Sustained Delegation . . . 56
6.2.1 Environment: Learning across Realistic Work Worlds . . . 56
6.2.2 Persistence: Memory, Initiative, and Adaptation . . . 57
6.2.3 Verification: Work Quality, Personal Fidelity, and Return on Delegation . . . 58
6.3 Two Interacting Loops: Delivering Work and Improving Capability . . . 59
6.3.1 The Work Execution Loop . . . 59
6.3.2 The Capability Improvement Loop . . . 60
6.3.3 Coupling Learning to Open Responsibilities . . . 60
6.4 Research Outlook: Useful Work under Openness and Persistence . . . 60

7 Conclusion . . . 61

1 Introduction

Advances in foundation models' reasoning, planning, and tool-use capabilities are expanding the scope of work that agents can undertake [1]. Their responsibilities increasingly extend beyond short, narrowly scoped tasks to larger assignments that require them to maintain an overall objective, coordinate multiple stages, and carry the work through to completion. Furthermore, recursive self-improvement [2] explores how agents can improve their own models and runtime systems, bringing capability improvement itself into the range of work agents undertake.

Humans are already delegating complex productivity tasks to agents across software engineering [3], office work [4, 5], web and computer use [6–8], enterprise operations [9], and scientific research [10]. Frontier agents such as Codex and Claude Code [11, 12] illustrate this shift by supporting multi-day, cross-session work with suitable orchestration and taking on parts of result assessment and feedback-driven revision previously handled by humans. Across these tasks, decisions often depend on evidence in related files, records, or applications and on states established by earlier actions. A work-centric perspective traces these dependencies from the initial request through execution to a verifiable outcome.

From this perspective, we introduce the concept of a **work agent**, defined as a system that uses a foundation model as its decision-making core, relies on a runtime harness to organize environment observations and action execution, and carries out delegated tasks within a stateful digital workspace. The foundation model provides understanding, reasoning, and decision-making capabilities; the harness manages context construction, tool invocation, memory, skills, execution control, and error recovery; and the workspace contains interconnected objects such as files, applications, records, tasks, and other actors. A work agent aims to produce work outcomes that satisfy task requirements, with execution results verified through observable evidence. This perspective connects task objectives, model capabilities, system mechanisms, environment states, and verification, shifting the unit of analysis from isolated model outputs or tool calls to the complete process of accomplishing digital work.

Work agents operate under different conditions depending on the tasks they undertake. To characterize these differences, we introduce two conceptually distinct yet complementary dimensions: ❶ **work persistence** and ❷ **work openness**. Work persistence concerns the conditions under which delegated responsibility can be considered complete, distinguishing work that concludes upon delivery from responsibilities that remain active across multiple stages, feedback cycles, or external events. Work openness describes the extent to which task-relevant objects, information, interfaces, and rules are specified before execution, or must instead be discovered and interpreted during execution. These working conditions impose different demands on an agent's ability to understand its environment, manage task dependencies, maintain execution state, and verify outcomes. Based on the functional requirements that arise throughout the work process, we organize the core capabilities of work agents into five interrelated families: perception and understanding, reasoning and decision making, planning and coordination, execution and state management, and verification, recovery, and replanning. Together, these capabilities support the complete process of interpreting work requirements, organizing and executing actions, and verifying results, forming an iterative execution cycle driven by environmental feedback.

The performance of work agents depends not only on their core capabilities but also on how foundation models and runtime harnesses work together. From an optimization perspective, some capabilities can be learned by models through training data, interaction experience, and feedback, while others can be supported by external mechanisms such as tool interfaces, memory management, and execution control. Improving work agents therefore requires considering model optimization, harness optimization, and their joint optimization. From an evaluation perspective, agent performance is reflected not only in the correctness of answers or individual actions but also in the quality of work artifacts, changes to environment states, workflow completion, and satisfaction of task requirements. Systematic evaluation across these dimensions provides a more comprehensive understanding of the practical performance and reliability of work agents.

As agents move from executing bounded tasks toward sustained work delegation, their responsibilities increasingly extend across multiple stages and events. In this setting, agents must maintain continuity despite environmental changes, interruptions, and external feedback, while also leveraging experience accumulated

during execution to improve their capabilities without disrupting ongoing responsibilities. Work execution and capability improvement thus form two interacting loops, raising new research challenges for the long-term development of work agents. Personal work agents, human digital twins, and organizational work systems provide research settings for sustaining goals between requests, representing a particular person's judgment, and coordinating responsibilities across roles. Future research must address reliability, environmental adaptation, and outcome verification during sustained execution, while also considering the practical value of work outcomes and the human and computational resources required to produce them. These challenges will shape the transition from completing individual tasks to undertaking more persistent, complex, and valuable digital work.

Based on this perspective, the main contributions of this paper are summarized as follows:

- **A unified, work-centric framework for agent systems.** We present a unified perspective centered on digital work, characterizing the structure and execution of work agents through six elements: task, model, harness, workspace, work outcome, and verification. We further describe the scope, organization, and execution conditions of digital work through five work forms and two complementary dimensions, work persistence and work openness.
- **A systematic account of core capabilities and optimization for reliable work.** We identify five core capability families required by work agents and examine their roles under different working conditions. We further examine training environments, data and reward design, model optimization, harness optimization, and their joint optimization, highlighting how model learning and external system support contribute to reliable work execution.
- **An outcome-oriented perspective on agent evaluation.** We systematically analyze existing benchmarks, evaluation metrics, judging methods, and execution environments, examining how they assess work artifact quality, environment-state changes, task completion, and constraint satisfaction. This provides a structured perspective for understanding the practical performance and reliability of work agents.
- **Future research directions toward sustained work delegation.** We explore emerging applications, including personal work agents, human digital twins, and organizational work systems, and discuss key challenges involving persistent responsibility, memory and adaptation, outcome verification, and the value of work in open environments. We further examine the interaction between work execution and capability improvement to outline research directions for next-generation work agents.

This paper focuses on digital knowledge work across office productivity, professional services, enterprise operations, software engineering, and scientific research. Section 2 introduces the foundations of work agents, including their system components, working conditions, and core capabilities. Section 3 examines five forms of digital work undertaken by agents. Section 4 examines training environments, model optimization, harness optimization, and their joint optimization. Section 5 covers evaluation benchmarks, methodologies, metrics, and execution environments. Section 6 discusses future applications, technical frontiers, and the interaction between work execution and capability improvement. Section 7 concludes the paper.

2 Foundation of Work Agents

Section 1 argued that the performance of a system that carries out digital work depends not only on its model but also on its interfaces, the information available to it, the organization of its state, and the verification of its results. Studying such systems therefore requires a common account of what the system consists of, the conditions under which it works, what it must be able to do, and how it differs from related lines of agent research. This section develops this account in four steps. Section 2.1 identifies the elements involved when digital work is delegated to an agent and defines a work agent. Section 2.2 introduces two dimensions of working conditions, work persistence and work openness, which describe when a responsibility may close and how much the agent must find out before and while it acts. Section 2.3 identifies five core capabilities that the work process requires and examines how the two dimensions raise their demands. Section 2.4 compares work agents with long-horizon, computer-use, and office agents.

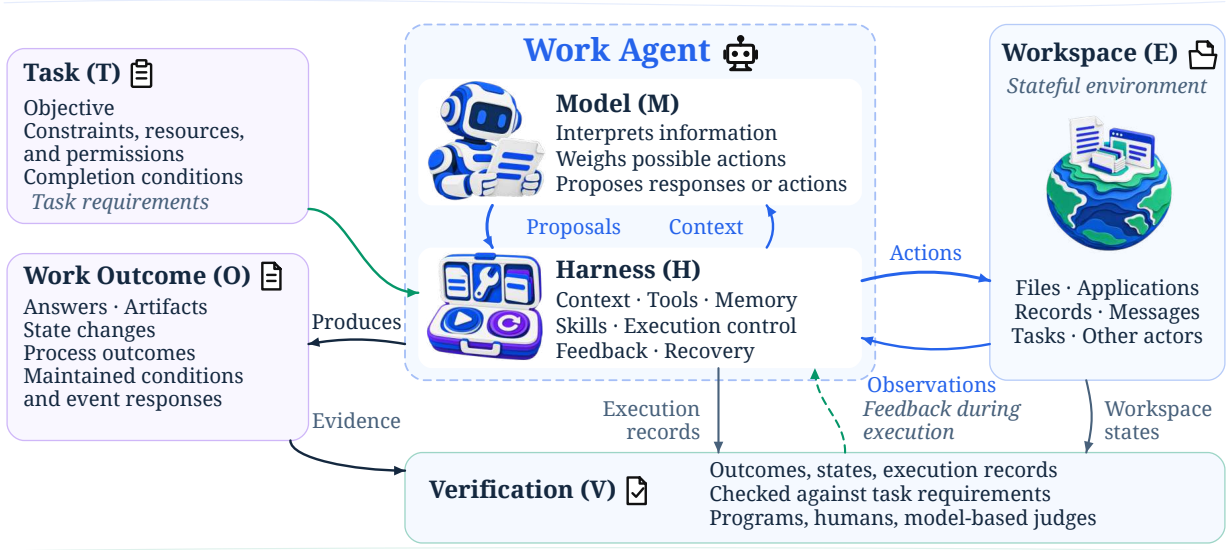


Figure 1 A system view of work agents. The model and harness form the work agent within the dashed boundary. The task specifies the objective, constraints, resources, permissions, and completion conditions. The harness supplies context to the model, carries out proposed actions in the stateful workspace, and receives observations. Outcomes include answers, artifacts, state changes, and process outcomes. For ongoing responsibilities, they also include maintained conditions and event responses over an agreed interval. Verification assesses evidence from these outcomes, workspace states, and execution records against task requirements. Programs, humans, or model-based judges apply task-specific criteria. Verification may occur during execution, with feedback informing subsequent decisions, or afterwards to assess the work.

2.1 Definition of Work Agents

We begin with the system itself. By digital work we mean work carried out on digital objects such as files, applications, and records, for example updating a report or handling a service request. When a person or an organization hands such work to an agent, three things happen: the requester states what should be done, the agent acts on these objects to do it, and the result is checked against what was asked. We call such work delegated: the agent carries it out on the requester's behalf and is accountable for the result. We first describe the six elements involved in this process, as shown in Figure 1, then define a work agent in terms of them, and finally describe how the elements interact during execution.

Task (T) specifies the objective, the relevant constraints, the available resources, the permissions, and the conditions for valid completion. For an ongoing responsibility, it also specifies the conditions to maintain, the agreed interval, and the criteria for closure or renewal.

Model (M) serves as the system's decision-making core. Given context supplied by the harness, it interprets the available information, weighs possible courses of action, and proposes responses or actions [13, 14].

Harness (H) organizes context construction, tool use, memory, skills, execution control, feedback handling, and recovery, mediating between the model and the workspace [14].

Workspace (E) is the stateful environment in which work takes place. It contains files, applications, records, messages, tasks, and other actors. These are linked to one another, and the agent accesses them through interfaces within its permissions. Both the agent's actions and external events change its state as work proceeds.

Work Outcome (O) is the task-relevant result that the agent produces. Outcomes include answers, such as analyses, plans, and executable specifications; artifacts such as documents, spreadsheets, presentations, and code; state changes in applications or business records, such as updated records and sent messages; process outcomes, such as a completed process or the stages, approvals, and handoffs finished so far; and maintained

conditions and event responses over an agreed interval. One assignment may combine several outcomes, such as an updated workbook, a report, and the corresponding changes to business records, and require them to remain consistent across related artifacts, records, and applications.

Verification (V) assesses work against task requirements. It inspects evidence from answers, workspace states, artifacts, and workflow records (Section 5.2.1). Programs, humans, or model-based judges [15] assess this evidence against task-specific criteria, which may be organized as rules, rubrics, or checklists (Section 5.2.2). The checks cover task completion, correctness, completeness, quality, constraint satisfaction, and intended effects. For example, SWE-bench [16] executes tests against a modified repository, whereas PresentBench [17] applies task-specific checklist items through a model judge. Verification may be performed by the agent during execution (Section 2.3.5) or after execution by the requester or a benchmark (Section 5); its judgments support completion decisions, quality measures, and diagnostic feedback for subsequent work.

With these elements, we define a **work agent** as a system that uses a foundation model as its decision-making core and a harness to organize observations and actions in a stateful workspace. It pursues a delegated work objective under specified constraints, resources, and permissions, and its outcomes are assessed against task requirements through observable evidence. The agent itself consists of the model and the harness, shown as the dashed box in Figure 1; the task, workspace, outcome, and verification define the work for which it is accountable. The definition covers a wide range of tasks. In the minimal case, a question-answering system counts as a work agent when answering is delegated to it and its answers are checked against task requirements. Its harness only assembles the question and the supplied material into context and returns the model's answer, and its workspace is that material, which the agent reads but does not change. A system that edits workbooks or carries business processes to completion uses the same elements with a fuller harness and a workspace that its actions change (Section 3).

The six elements interact in a loop during execution. The harness constructs context from the task, workspace observations, and retained execution information and passes it to the model. The model proposes responses or actions; the harness carries out the actions, receives observations, and uses them with verification feedback to update the context for the next decision. At an assessment point t, execution and verification have distinct roles:

$$(T, E_0) \xrightarrow[\text{closed-loop execution}]{M,H} (\tau_{0:t}, E_t, O_t), \tag{1}$$

$$S_t = V(T, E_0, \tau_{0:t}, E_t, O_t). \tag{2}$$

Equation (1) summarizes execution by the model–harness system; Equation (2) defines an assessment of its evidence against the task requirements. Here, E_0 is the initial workspace state, $\tau_{0:t}$ is the observable sequence of actions and feedback up to t, E_t is the workspace state at t, and O_t is the outcome produced so far. The assessment S_t records how the inspected evidence meets the task requirements. Assessment can mark completion or occur during an ongoing responsibility.

All work-agent settings share these elements and this loop; what differs is the conditions under which the loop runs, which Section 2.2 examines.

2.2 Working Conditions

Across tasks, these conditions differ in two respects: whether the work ends once the agent delivers its result, and how much the agent must find out before and while it acts. We capture these two differences as ❶ work persistence and ❷ work openness. Section 2.2.1 defines work persistence and traces it through existing research, Section 2.2.2 does the same for work openness, and Section 2.2.3 shows that the two dimensions are conceptually distinct, illustrates their four combinations, and relates them to the forms of work examined in Section 3.

2.2.1 Work Persistence

The first difference concerns when the agent may consider delegated work finished. In some tasks, the work is finished once the agent delivers its result, such as an answer, a confirmed change, or a tested patch. In others, the agent's delivery alone does not finish the work: other participants must still assess, approve, or contribute,

or later events require earlier results to be revisited. **Work persistence** thus ranges from responsibility that closes at delivery to responsibility that remains open across stages and events. As persistence increases, the agent must carry state across waiting periods and events and must determine at each point which parts of its responsibility are still open.

Existing research covers both sides of this range. Responsibility closes at delivery when closure depends only on what the agent itself delivers. In early work on reading comprehension and question answering over text, tables, documents, and charts [18–23], responsibility closes once an answer is returned. Tool-use research, from Toolformer [24] and API-Bank [25] to Gorilla [26], extends the output from text to tool invocation. When an action changes the workspace, the work also requires a confirmed effect: in AndroidWorld [27], a messaging task counts as complete only when the intended message is recorded in the phone's SMS database. Responsibility can also close at delivery after a sequence of dependent actions. In SWE-bench [16], producing a patch involves localization, editing, and testing; in MINT [28], tool outputs and language feedback across turns guide subsequent actions.

On the other side, closure depends on more than the agent's own delivery: on other participants or on later events. When it depends on other participants, responsibility cannot close until they provide the confirmation or contribution the task requires: τ-bench [29] withholds specified database updates until the user confirms them, τ^2-bench [30] requires the agent to coordinate with a simulated user who carries out part of the work, and TheAgentCompany [31] includes tasks that cannot proceed without information from simulated coworkers. When it depends on later events, earlier results or completed stages may need to be revisited: in OSWorld 2.0 [32], revised budget and vendor constraints arrive through a team chat and require the agent to reconsider affected entries in a purchase-order form. When the task defines an obligation over an agreed interval, the agent remains responsible for responding to such events throughout that interval: E-Commerce Bench [33] spans a simulated year of operational decisions, and VibeLifeBench [34] evaluates proactive and persistent assistance as events change the environment over simulated weeks.

Dividing the examples by what closure depends on also separates persistence from task difficulty and execution length. Answering a question can demand harder reasoning than submitting a form for approval, yet responsibility for the answer closes at delivery, whereas the workflow remains open until approval is received. The same holds for execution length, a central concern of long-horizon research (Section 2.4.1). A long run may pass through many dependent steps, yet its responsibility still closes when the result is submitted; an obligation to keep a service within its operating constraints may require only a short run each week, yet its responsibility remains open for a quarter. The two examples sit on opposite sides of the boundary: the first closes once its run delivers a result, whereas the second stays open across many runs until the agreed interval ends.

2.2.2 Work Openness

The two examples at the end of Section 2.2.1 illustrate when responsibility may close, but do not specify how much the agent must find out before and while it acts. The long run may operate on a repository whose relevant files are named in the request, or on one whose relevant files it must first locate; the weekly run may read the same fixed metrics each time, or have to trace which components a new incident affects. In some tasks, the task world, that is, the objects, interfaces, rules, and actors relevant to the task, is specified with the request. In others, the agent must establish during execution which objects are relevant and how they are represented and related. It may also need to find out where the relevant state is held, how other actors have changed it, and which actions it is authorized to take. **Work openness** thus ranges from a predefined task world to one discovered during execution. As openness increases, completing the work depends more on the agent's ability to build and update its understanding of the task world while acting in it.

As with persistence, existing research spans this range, and the examples below are ordered by how much the agent must establish during execution. Early research largely studied predefined task worlds. Reading-comprehension and question-answering tasks supply the passage, table, document, or chart to be analyzed [18–23], and tool-use studies such as API-Bank [25], Gorilla [26], and BFCL [35] expose an explicit set of tools and interfaces. The agent reasons or acts within a space that has already been defined, and the difficulty lies in using the supplied material and tools correctly rather than in finding them.

Finding them becomes part of the task in web and computer-use environments. In WebArena [6], the information needed for a task must be obtained by navigating and interacting with functional websites rather than read from the prompt. OSWorld [7] extends the setting to real computer environments, where the agent works across applications and heterogeneous digital objects. In Workspace-Bench [36], the agent must also identify which files are relevant and how they depend on one another, so locating material across a workspace becomes the core of the work.

Beyond objects, other environments make the state of the task world, the actions of other actors, and authority part of what must be established. AppWorld [37] spreads the information a task needs across multiple applications, so the agent must determine which applications and records hold it before acting. In τ^2-bench [30], a simulated user acts on parts of the shared environment that the agent cannot operate, so the agent must find out from the user what has changed. Authority must be established when the request does not state who may do what. In EntCollabBench [38], instructions describe business goals rather than workflows, and each agent may use only its own department's tools, so the agent must infer which role is responsible for each step and delegate to it. Applicable permissions may likewise have to be discovered during execution: in a card-unlocking task in τ-Knowledge's τ-Banking domain [39], the agent must retrieve the relevant policy and inspect the card's status to determine that a security hold prevents it from unlocking the card and requires it to offer escalation to the security team.

2.2.3 Combining Persistence and Openness

Having defined both dimensions, we now consider how they combine. The two dimensions answer different questions, as the two examples at the start of Section 2.2.2 already suggest: openness concerns what the agent must find out, whereas persistence concerns when it may stop. The same workspace change can involve both dimensions: discovering which objects it affects concerns openness, while responsibility that remains open as further changes arrive concerns persistence. The two dimensions are thus conceptually distinct: a setting can be high on one and low on the other.

As shown in Figure 2, all four combinations appear in existing research. Question answering (bottom left) combines a predefined task world with responsibility that closes at delivery: the system answers from given material, as in SQuAD [18] and DocVQA [22]. Information seeking (bottom right) asks the agent to discover the relevant material during execution, yet responsibility still closes at delivery, as in GAIA [40], whose questions require finding a source on the web, and Workspace-Bench [36]. Service workflows (top left) keep the task world predefined, yet responsibility remains open while the requested updates await user confirmation, as in τ-bench [29], where the tools, domain policy, and database are specified in advance. Ongoing work under changing conditions (top right) combines a task world discovered during execution with responsibility that remains open as later events arrive: revised constraints arriving mid-task in OSWorld 2.0 [32] require the agent to find the affected entries, and in VibeLifeBench [34], some changes come without notification, so the agent must re-inspect the environment to discover them.

These combinations describe the conditions of work rather than its form. Section 3 distinguishes five work forms by the scope and organization of delegated work, from work at the Conversation Level, which answers questions over supplied material, to work at the Workspace Level, which operates across related objects in a workspace. Persistence and openness characterize the conditions under which each form is carried out: work at the Workflow Level, for example, can run in the predefined task world of τ-bench [29] or require the agent to track a shared state that a simulated user also changes, as in τ^2-bench [30]. Because both dimensions change what the agent must find out and keep track of, they also determine how demanding its core capabilities become, which Section 2.3 examines.

2.3 Core Capabilities of Work Agents

Whatever the working conditions of Section 2.2, carrying out the work process of Section 2.1 raises five questions, each answered by one capability that recurs across office applications [4, 5], software repositories [3, 16], enterprise services [9, 29], and professional workspaces [31, 36]: (1) what the work involves: Perception and Understanding interprets the objects and their relationships; (2) what to do next: Reasoning and Decision Making weighs goals, constraints, and evidence to decide how to proceed; (3) how dependent steps and

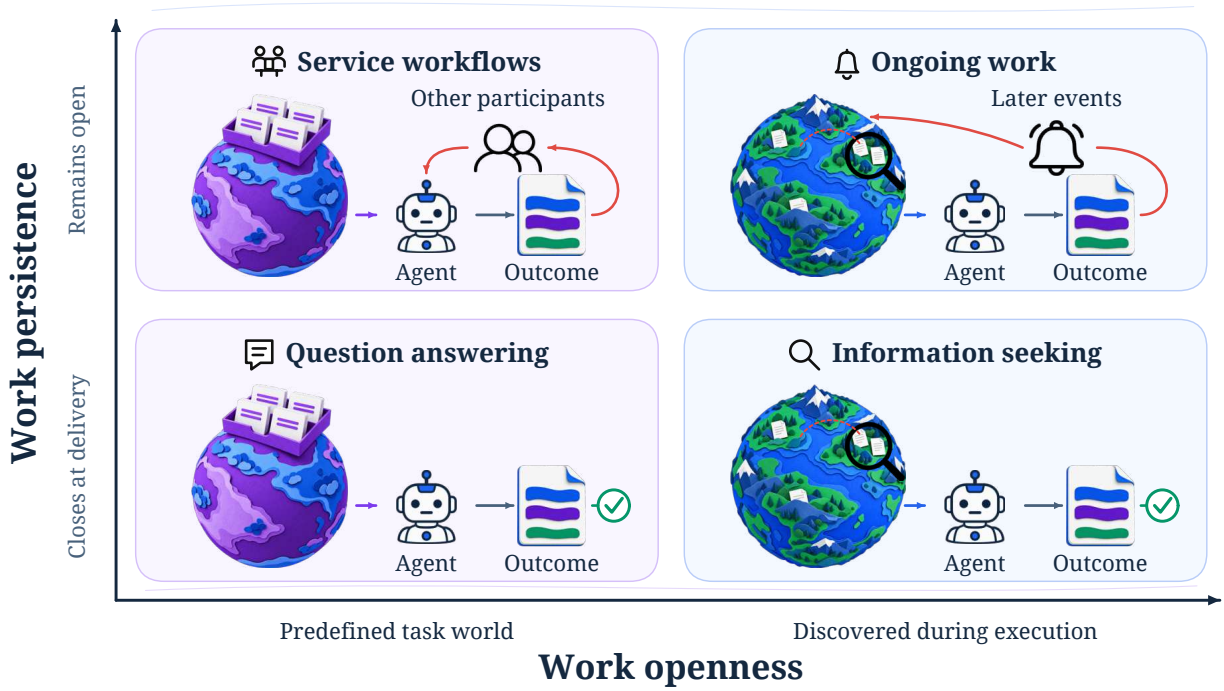


Figure 2 Work persistence and work openness as conceptually distinct dimensions of working conditions. Persistence ranges from responsibility that closes at delivery to responsibility that remains open across stages and events; openness ranges from a predefined task world to one discovered during execution. The four scenes show question answering (bottom left), information seeking (bottom right), service workflows (top left), and ongoing work under changing conditions (top right). Supplied packets distinguish the predefined task worlds; scattered material and magnifiers indicate that relevant objects and relationships must be found during execution. The same compact Agent marker appears in every scene. Checks at the terminal end mark responsibility that closes at delivery. Return paths through other participants (top left) or later events (top right) mark responsibility that remains open. The upper scenes include dependencies within bounded tasks as well as obligations extending beyond an initial delivery.

contributions fit together: Planning and Coordination arranges dependencies and handoffs among steps and participants; (4) what has actually changed: Execution and State Management acts on the workspace through tools and interfaces while recording progress; and (5) whether the results satisfy the requirements: Verification, Recovery, and Replanning confirms outcomes and guides correction. Each capability is a function of the work process that the model, the harness, or their interaction can carry out, and in a simple task several are carried out in one step, as when a question-answering system reads the supplied material and returns a checked answer.

These capabilities form a cycle rather than a pipeline. The numbered panels in Figure 3 illustrate a pass through this cycle: Perception and Understanding passes evidence to Reasoning and Decision Making, which passes a chosen approach to Planning and Coordination; the resulting plan guides Execution and State Management in taking actions and recording observations. Verification, Recovery, and Replanning checks both the updated work objects and these execution records. When the current delivery requirements are met, the outcome can be delivered; otherwise, the work continues at the capability where the problem lies (Section 2.3.5). An ongoing responsibility remains active after delivery until its agreed closure conditions are met. Throughout, the capabilities read context and record updates through a shared work context, which holds task requirements with their completion criteria, relevant evidence about objects and links, plan and progress with pending dependencies, and observed state with results and check status.

The two dimensions of Section 2.2 raise the demands on this cycle in different ways: persistence extends each capability across stages and events, whereas openness increases what must be discovered. Table 1 lists the resulting demands.

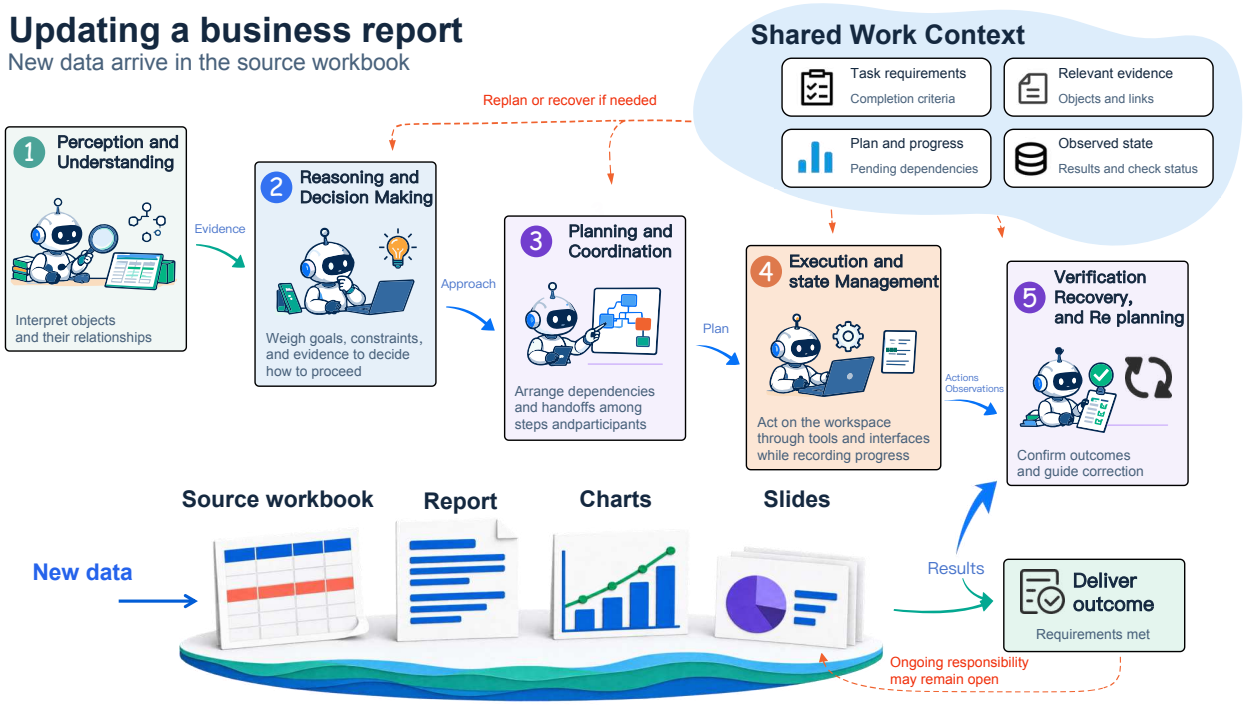


Figure 3 Five core capabilities supporting an illustrative business-report update. New data in the source workbook motivate revisions of the report, charts, and slides. The numbered panels identify the five functions, and solid arrows trace evidence, approaches, plans, actions, observations, and results. Shared work context retains task requirements, evidence and object relationships, plans and pending dependencies, and observed state with check status. Verification supports delivery when current requirements are met. Orange dashed paths indicate possible recovery, replanning, or continued work when an ongoing responsibility remains open after delivery.

The following subsections examine the five capabilities in turn, using one task throughout: updating a business report, together with its source workbook and associated slides, when new data arrive. Each subsection describes what the capability does and what it passes on, examines representative work, and states what it provides. To show how the two dimensions raise the demands listed in Table 1, each subsection then considers an extended version of the task: the agent is responsible for keeping the report up to date until it is approved, a data revision arrives during assessment in an upstream workbook that feeds the source workbook, and the request names neither that workbook nor the owner of the revised data.

2.3.1 Perception and Understanding

Before acting, an agent must know what the task refers to. Perception and Understanding interprets objects and relationships: it identifies the objects involved in a task, interprets the content and structure of each object, and traces its links to other objects. It produces evidence whose meaning and source can be traced, including where each target of an edit is located, for use by Reasoning and Decision Making. In the report task, a value in the workbook becomes usable evidence only once its header, unit, and reporting period are known, and an edit can be applied only once its target cells or slide element are located.

Existing work studies both the structure within an object and the links between objects. The structure within an object tells what a value means and where an edit applies. For meaning, SpreadsheetAgent [41] extracts hierarchical headers and layout cues from local spreadsheet regions, and PPTAgent [5] analyzes the functional types and content structures of reference slides. For location, SpreadsheetLLM [42] uses structural anchors, cell-address mappings, and format aggregation to locate relevant spreadsheet regions, and AutoCodeRover [43] searches program structure to locate issue-relevant classes and methods. The links between objects tell what else a task involves: RepoBench [44] evaluates the retrieval of relevant code across

Table 1 How work persistence and work openness raise the demands on the five core capabilities.

Capability	Higher persistence requires	Higher openness requires
Perception and Understanding	Re-perceiving changed state and detecting stale evidence	Discovering relevant objects and relationships during execution
Reasoning and Decision Making	Revisiting earlier decisions when evidence or constraints change	Deciding whether to act or to gather more information
Planning and Coordination	Spanning waiting periods, handoffs, and events	Identifying dependencies and participants during execution
Execution and State Management	Keeping durable state across sessions; avoiding duplicate effects on retry	Establishing applicable interfaces, permissions, and authority during execution
Verification, Recovery, and Replanning	Re-verifying affected results; replanning when prerequisites change	Establishing which sources count as evidence

files, and Workspace-Bench [36] requires the agent to connect content spread over several files.

Perception and Understanding thus provides an actionable account of the task's objects and the links between them. In the report task, it links a reported value to its source cells and to the chart or slide that uses it, so that later checks can follow these links. When the revision arrives during assessment, perception must determine which values already cited in the report are affected; to do so, it must locate the upstream workbook that the revision changes.

2.3.2 Reasoning and Decision Making

Located evidence does not by itself determine what to do next. Reasoning and Decision Making weighs goals, constraints, and evidence to decide how to proceed among the available alternatives. Goals determine the desired result, and constraints limit acceptable actions; goals and some constraints are supplied with the task, while further constraints may need to be established during execution. Evidence establishes the current situation and accumulates as the work proceeds, and domain knowledge, learned by the model or retrieved, determines how it should be interpreted. The result is a chosen approach, which Planning and Coordination arranges into a plan. In the report task, such reasoning determines whether two figures are comparable, which calculation is appropriate, and whether more information is needed before an edit is made.

Existing work studies both domain-guided decisions and the use of evidence gathered during work. For domain-guided decisions, FinQA [21] requires domain knowledge to reason numerically over financial reports, and τ-bench [29] makes domain policies and user requirements constrain tool-use decisions. For evidence gathered during the work, ReAct [13] interleaves reasoning with actions and observations, SheetAgent [45] queries relevant spreadsheet subviews for its planner and uses code-execution results to inform subsequent operations, and CodeAct [46] expresses actions as executable code and feeds interpreter outputs and error messages into subsequent decisions.

Reasoning and Decision Making thus provides decisions that can be checked later. To make this possible, the harness records each decision in the shared work context, together with the goal it serves, the constraints that applied, and references to the supporting evidence. Later steps can then tell whether it worked and whether its basis still holds. In the report task, the record of a decision to recompute a growth rate shows which figures were compared and why they were judged comparable. The revision reopens decisions like this one, and the agent must decide whether it can act on the revision directly or must first gather more information.

2.3.3 Planning and Coordination

When delegated work involves several dependent actions or contributions from other participants, each decision must be connected to the work that follows. Planning and Coordination arranges the dependencies and handoffs among these actions and contributions. Planning specifies subgoals, required inputs, and the order in which results become available [47–49]; coordination obtains and routes the information, authorization, or artifacts supplied by other participants [50]. The result is a plan, which Execution and State Management

carries out. In the report task, the analysis must be completed before the dependent charts are revised, and any information needed from collaborators must be obtained in time.

Representative work emphasizes different aspects of planning and coordination. Planning methods make dependencies explicit and revise them as results arrive: Data Interpreter [51] represents workflows as hierarchical dependency graphs, records task results and status, and adjusts affected tasks as execution develops, and Agent S2 [52] updates the remaining subgoals from the original task and new observations after each subgoal concludes. Coordination frameworks make one participant's output available as another's input: AutoGen [53] organizes role-based exchanges among agents, tools, and human input, and MetaGPT [54] passes structured artifacts such as requirements and designs through a shared message pool and a subscription mechanism.

Planning and Coordination thus provides a plan that records the result each step produces, the inputs needed to produce it, and the steps or participants that will use it, including dependencies that are still pending. In the report task, the record leads from a revised data source directly to the analysis, charts, and slides that must be redone. Because responsibility lasts until approval, the plan must span the assessment period and take in the revision when it arrives, and the owner of the revised data must be identified during execution.

2.3.4 Execution and State Management

Plans take effect only when they are applied to the workspace. Execution and State Management takes actions on workspace files, applications, and records through tools and interfaces, receives observations of their state, and records progress. Execution performs operations through code, tool interfaces, or graphical interfaces within the task's authorized scope; state management tracks the objects and versions involved, completed operations, returned results, and pending items. The two are connected because operations can change workspace state or reveal information that affects the next decision. The results are the artifacts produced and the records kept, which Verification, Recovery, and Replanning checks; in the report task, these are the revised workbook and the list of calculations and dependent edits already completed.

To retain the effects and observations on which later decisions depend, existing systems maintain action and state records during execution. SheetCopilot [4] executes spreadsheet operations through an observe–propose–revise–act state machine and updates its action and state records after each successful execution. InfiAgent [55] extends this practice beyond a single run: it stores plans, intermediate artifacts, and tool outputs in a task directory and updates progress markers, so that an interrupted task can be resumed from the record.

Execution and State Management thus provides a record that tells which effects can be relied on. The record distinguishes an *attempted action*, an *accepted call*, a *committed effect*, and a *verified effect*: an attempted action has been issued; an accepted call has been acknowledged by the interface; a committed effect is reflected in the target state; and a verified effect has additionally been checked against the relevant requirement. These states need not coincide, so acceptance of a call alone does not establish task success. An operation whose effect is known to be committed becomes the starting point for the next one, whereas an operation with an uncertain effect remains pending until its state is inspected. In the report task, the saved workbook and its current version are the basis for the next edit, whereas a chart update that may not have been applied stays pending until the slide is checked. Because the work resumes when the revision arrives during assessment, the record must be kept across sessions so that resumed work does not repeat edits already made; before reading the upstream workbook, the agent must also establish that it is permitted to access it.

2.3.5 Verification, Recovery, and Replanning

Recorded changes show what has happened but not whether it satisfies the task. Verification, Recovery, and Replanning confirms outcomes and guides correction. Its three functions proceed from assessment to corrective action: verification checks results against task requirements and completion criteria, recovery repairs the objects or operations responsible for a problem, and replanning revises the remaining steps when feedback shows that a prerequisite or condition has changed. We examine representative work for each function in turn.

Verification relates each requirement to evidence that can establish whether it has been met. The appropriate evidence depends on the property being checked, whether a benchmark defines the check or the agent runs

it itself. Behavior is checked by execution: SWE-bench [16] runs tests against the modified repository, and SpreadsheetBench [56] applies a spreadsheet transformation to several workbook inputs. Claims are checked against external information: CRITIC [57] uses external tools to obtain evidence for critiquing and correcting its own outputs. Presentation is checked on the rendered artifact: DeepPresenter [58] inspects rendered slides to identify issues such as overlap and overflow. The report task needs all three: executing its calculations, checking its claims against their sources, and inspecting its rendered slides.

When verification detects a problem, recovery repairs it. Self-Refine [59] revises the current output from generated feedback, whereas Reflexion [60] retains reflections on task feedback to improve the next attempt. In a workspace, a repair must also target the object or operation responsible, such as a wrong formula or an overflowing slide. When the effect of an earlier operation is uncertain, the agent must first inspect the current state to decide whether to retain, repair, or retry it; this matters most for operations, such as sending a message, that produce an additional effect when repeated.

Some problems cannot be repaired locally because a prerequisite or condition has changed. Replanning then revises the remaining course of work: it uses the dependencies recorded in the plan (Section 2.3.3) to identify the steps the change affects and draws on Planning and Coordination to rebuild them. LATS [61] searches over alternative reasoning and action paths, using environmental feedback and reflection to choose among them.

Verification, Recovery, and Replanning thus provides confirmed outcomes and corrections matched to the scope of each problem: a local defect receives a local repair and recheck, whereas a changed prerequisite leads to a revised plan and a check of its downstream results. After the revision, affected results already checked must be re-verified, and the agent must first establish which source counts as evidence for the revised figures.

With it, the cycle in Figure 3 closes: a confirmed result is delivered or becomes the basis for further work, and a remaining problem is directed to the capability it concerns, with missing evidence going back to Perception and Understanding, a questionable decision to Reasoning and Decision Making, a changed prerequisite to Planning and Coordination, and an execution error to Execution and State Management. Section 2.4 relates this framework to existing agent paradigms.

2.4 Work Agents and Related Agent Paradigms

Relative to this framework, three established lines of agent research each focus on one facet of the work: long-horizon agents on sustaining execution over many steps, computer-use agents on the action interface, and office agents on the type of artifact produced. A work agent is instead defined by its delegated objective and by outcomes assessed against task requirements, regardless of how long it runs, which interface it uses, or what artifacts it produces. Sections 2.4.1 to 2.4.3 summarize the research focus of each paradigm and state what the work-agent view adds, and Section 2.4.4 shows how the perspectives combine on one shared task.

2.4.1 Long-Horizon Agents

The first paradigm concerns sustaining execution over many steps. Long-horizon agents study how to maintain task progress, context, and state over extended execution and how to handle dependencies whose consequences emerge in later stages. A recent synthesis organizes this research into three levels [62]: the intra-context level covers coupled reasoning steps, the cross-context level covers state continuity beyond a single context window, and the cross-task level covers experience accumulation across tasks. Evaluations quantify this ability: the METR time-horizon analysis [63] measures task length by the time human experts take to complete the task and estimates the length at which an agent reaches a given success rate, while RE-Bench [64] compares research-engineering performance under different time budgets.

To sustain such work, representative methods keep information available beyond a single context window. InfiAgent [55] constructs its reasoning context from a snapshot of its stored task state and a fixed window of recent actions. On the evaluation side, LongMemEval [65] tests memory through timestamped user–assistant histories, including information extraction, multi-session reasoning, temporal reasoning, and updates to user information, and LoCoMo [66] evaluates very long-term conversational memory.

These methods and measures concern continuity of execution and retained information. Work persistence asks a different question: when responsibility may close (Section 2.2.1). Persistent delegation requires tracking open obligations, responding to other participants and later events, and re-verifying earlier results against the current workspace.

2.4.2 Computer-Use Agents

The second paradigm concerns the interface through which actions are taken. Computer-use agents study how to complete tasks through software interfaces in browsers, desktop applications, and operating systems. A central problem is to ground task intent in operations executable in the current interface: identifying relevant controls, locating action targets, and deciding what to do next from the resulting interface state [67–71]. SeeClick [72] studies the visual-grounding part of this process by locating screen elements from screenshots and instructions, and uses GUI-grounding pre-training to improve downstream task performance.

Grounding individual operations is not enough; the operations must also add up to the task outcome. Benchmarks in this line, such as WebVoyager [73], therefore evaluate whole sequences of operations. WebArena [6] checks, depending on the task, whether an answer, page location, or website state satisfies the intended goal, and OSWorld [7] uses task-specific scripts to check the resulting files and application states.

The work-agent view places these interface operations within the delegated task as a whole. Computer use is one interface through which the harness acts on the workspace [74]. Beyond the interface, this view also asks when responsibility closes and which permissions and dependencies the agent must discover during execution, including dependencies on objects that the interface does not show. Relating interface operations to these questions shows what each operation contributes to the task.

2.4.3 Office Agents

The third paradigm concerns the type of artifact produced. Office agents, such as SheetCopilot, PPTAgent, and Huawei Cloud OfficeAce [4, 5, 75], create and modify common work artifacts such as documents, spreadsheets, and presentations. Task requirements concern both content and the structure and use of the objects that carry it: spreadsheet operations must act on the correct data and cells, while presentation production must balance content, layout, and overall coherence. The quality of the resulting artifact depends both on how individual objects are manipulated and on how the task is organized.

Representative studies address these two aspects. For object manipulation, SheetCopilot [4] controls spreadsheets through a set of atomic actions, and PPTAgent [5] generates slides from reference presentations through code-based editing and evaluates content, design, and coherence. For task organization, OfficeBench [76] provides single- and multi-application tasks involving documents, spreadsheets, calendars, and email; its cross-application tasks require information obtained earlier to be used in later operations, and their results are verified through matching rules or execution-based checks.

Office agents thus take office objects and the workflows around them as their main setting. The work-agent view offers a framework for analyzing such work alongside software, service, and organizational work: it describes office outcomes in the same terms as other work outcomes (Section 2.1), so that an updated workbook, the corresponding changes to business records, and the approval required before publication can be analyzed as parts of one delegated task.

2.4.4 Relationships among Paradigms

Having examined each paradigm separately, we now compare them on the same task. Table 2 uses the report task from Section 2.3, updating a business report when new data arrive; as its column "What is tracked" shows, the three related paradigms each track a different part of this task. A single system may need all three, as when an office agent completes a long reporting task through desktop applications and must maintain state over many steps. The work-agent view places these concerns within one work process, from task requirements through execution in the workspace to verified outcomes. This allows the paper to relate research across paradigms, identify which parts of the work each supports, and determine which capabilities must still be combined for a given delegated task.

Table 2 Work agents and related agent paradigms, compared through the task of updating a business report when new data arrive. These perspectives are complementary and can be combined within one system.

Perspective	Core question	What is tracked	Same task, different focus
Long-Horizon Agents	How is work sustained over many steps?	Progress, prior results, and pending dependencies	Resume from prior results and incorporate new data.
Computer-Use Agents	How are actions grounded in software interfaces?	Targets, interface state, and action effects	Locate cells, edit values, and confirm the saved state.
Office Agents	How are office artifacts created and updated?	Content, structure, layout, and cross-artifact links	Update the workbook, report, and associated slides.
Work Agents	How is delegated work carried through to a verified outcome?	Task requirements, workspace state, progress, outcomes, and evidence	Plan the update, integrate new data, revise linked artifacts, and verify the agreed deliverables.

The next section examines the scope and organization of the work delegated to work agents.

3 Work Forms: What Work Do Agents Undertake?

Section 2 describes the system elements, working conditions, and core capabilities of work agents. Together, these provide a foundation for explaining how an agent interprets a task, organizes its actions, and checks their effects. The demands on these capabilities depend on the work delegated to the system. We examine that work through three questions: what outcome must the agent produce, which objects and dependencies fall within its responsibility, and what conditions determine completion? The five work forms below describe how these responsibilities are organized.

Because a workflow can contain informational responses and local actions, Figure 4 assigns each representative task setting to the broadest form it requires.

At the Conversation Level, the agent produces an informational result, whereas at the Atomic Action Level, it executes a local operation on a work object. Moving to the Multi-step Level, the agent combines such dependent actions to achieve a complete objective. This responsibility broadens at the Workflow Level, where the process also becomes part of the delegated work and the agent advances it through stages governed by procedural requirements and interactions with other participants. It extends further at the Workspace Level, where the request no longer specifies every object involved and the agent must establish which files, records, and applications bear on the work and how they are related. As the scope expands, the agent must manage more of the dependencies surrounding an outcome, from intermediate results to stage transitions and relationships among work objects. Table 3 summarizes the resulting work requirements.

3.1 Conversation Level

Work at the Conversation Level produces an informational result from a request and its associated evidence, such as an answer, analysis, classification, or judgment. The agent interprets the relevant material and applies the knowledge or computation needed to produce that result. Completion is assessed on the returned information and does not require modifying the source artifacts, external work objects, or their states.

3.1.1 Producing Informational Outcomes

In these tasks, the delegated deliverable is information that answers the requester's question or supports a judgment. It may range from a short factual answer to an assessment, explanation, or detailed proposal. SQuAD asks for an answer extracted from a supplied passage [18], FinQA requires a numerical answer derived from financial text and tables [21], and MMTU includes classifications and judgments about whether a statement is supported [77]. Professional requests extend these demands: AAAR-1.0 asks for equation checks and critiques of research papers [78]; FinLFQA requests financial analyses grounded in sources and calculations [79]; PRBench asks for justified legal and financial advice [80]; and MMLongBench includes summarization of supplied PDF documents [81]. Responses can also take the form of plans or executable specifications. In Idea2Plan's direct-prompting setting, models develop a supplied research idea into a plan that includes experimental design, while TravelPlanner's sole-planning setting asks for a feasible itinerary based on supplied travel information [82, 83]. WorfBench asks for a workflow graph of proposed subtasks and dependencies [84]. Program outputs specify how an object can be constructed: BlenderLLM's CADBench asks for Blender scripts from modeling instructions, while the CADBench reconstruction benchmark asks for editable CAD programs from supplied images or meshes [85, 86].

3.1.2 Interpreting Evidence and Deriving Responses

The complexity of a response task depends on both the source material to be understood and the reasoning needed to process it. Prose presents facts in linguistic context, while tables relate values through rows and columns [87, 88]. Document images carry information in page layout [89], while charts encode values and relationships visually [23]. Extracting a value or comparing entries therefore depends on interpreting these structures [21, 22, 90]. In GDP.pdf, a footnote changes which employers belong in a chronological ranking [91]. Repository code and structured data also appear as supplied evidence in LongBench v2's multiple-choice comprehension tasks [92].

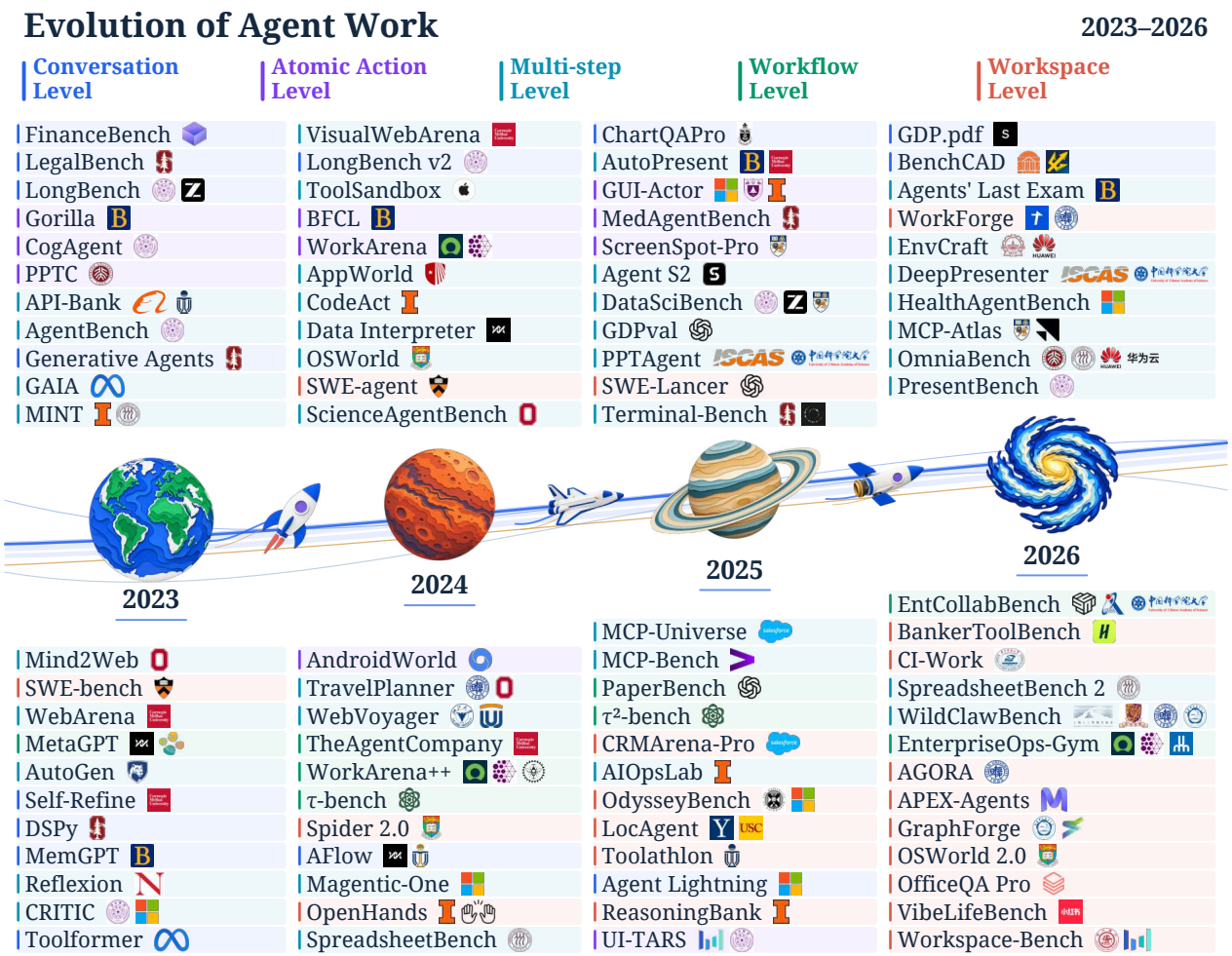


Figure 4 Evolution of agent work through 90 representative works in 2023–2026. Colors follow the five work forms in Section 3 and classify selected task settings, not entire methods or frameworks. Method settings include Generative Agents interviews, Self-Refine FED dialogue, MemGPT MSC dialogue, numerical answers in Toolformer, DSPy, AFlow and Agent Lightning, CRITIC web-evidence QA, AutoGen and Reflexion ALFWorld tasks, Magentic-One GAIA tasks, a specified UI-TARS slide edit, ReasoningBank full purchase-history lookup, and an EnvCraft released literature-synthesis task. Benchmark settings include FinanceBench long context, BFCL single calls, WorkArena forms, PPTC individual edits, MedAgentBench blood-pressure documentation, AndroidWorld SMS, two-stage TravelPlanner, interactive AgentBench, feedback-based MINT, MetaGPT procedures, local GUI action or grounding for CogAgent, GUI-Actor and ScreenSpot-Pro, and BenchCAD numeric QA. Workspace examples use SWE-bench repository repair with SWE-agent and OpenHands, SWE-Lancer individual-contributor tasks, Spider 2.0 DBT projects, CRMArena-Pro records, Toolathlon cross-source work, OfficeQA Pro full-corpus search, GraphForge and WorkForge task synthesis, OdysseyBench history-referenced objects, BankerToolBench data-room discovery, CI-Work tool-based retrieval, and LocAgent repository localization. Terminal-Bench uses 2.0. Years refer to selected public versions; AIOpsLab uses its 2025 paper despite 2024 predecessors. Logos identify recorded first-author or joint-first-author institutions.

The scope of the supplied material determines where relevant evidence must be found, while relationships among its parts determine what must be connected. FinanceBench's long-context setting supplies the relevant filing in the prompt [93]; multi-hop and long-document tasks require linking supporting information across passages, pages, or documents [19, 94–97]. A synthesis must further combine these contributions into an account that preserves their meaning [79, 81]. ConvFinQA extends the dependency to conversation history, with follow-up calculations using values or results established in earlier turns [98].

The requested inference determines how the evidence is used. A numerical answer may require computing with selected values, sometimes through an intermediate program [21, 99, 100], while a legal conclusion may require applying an explicit rule to the facts [101]. Scientific assessments draw on domain knowledge to interpret the evidence [102, 103]. Plans require comparing available options against stated requirements and commonsense feasibility constraints, including whether the proposed choices can be combined into a workable arrangement [78, 83].

An informational response can explain why a report contains an incorrect value. Correcting that value extends the responsibility to changing the relevant workbook entry and checking that the update has taken effect.

3.2 Atomic Action Level

This transition from generating information to modifying state defines the Atomic Action Level, which assigns the agent a local objective with a clearly defined target and effect, such as correcting a spreadsheet cell, repositioning a slide element, or updating a record field. Here, the level is determined by the local scope of the work objective rather than the number of low-level execution steps: a single local action may still require several mouse, keyboard, or API operations to complete. Regardless of the interface mechanics, the agent must apply the change to the intended object while preserving any properties that the request leaves unchanged [104].

3.2.1 Creating and Modifying Local Work Objects

A request may add a bounded element to an existing artifact, such as a formula [105, 106], a conditional formatting rule [107], or a chart in WTM-Bench's single-artifact tasks [108]. String transformations and spreadsheet data manipulation can likewise be specified through examples or natural language [109, 110]. Existing content can also be edited: InstructExcel expresses spreadsheet operations as scripts, PPTC includes individual edits to slide text, styles, and positions, and MiLDEdit specifies design-layer changes that preserve surrounding layout and non-target content [111–113].

The same local scope applies to application records and communication state. WorkArena's form tasks request records with specified field values, while WorkBench includes scheduling a particular event and replying to or deleting a selected email [9, 114]. Each request supplies a target and an intended change that execution must realize.

3.2.2 Executing and Checking a Local Change

To carry out these requests, the agent must identify the target and specify the operation through an available interface. ScreenSpot-Pro isolates target location in professional application screenshots [115]. BFCL's single-call tasks examine function and argument specifications; MedAgentBench adds patient-specific parameters through a blood-pressure documentation request linked to the intended record [35, 116].

After execution, completion depends on the resulting state: an artifact must contain the requested content or property, and an application record or message must reflect the intended change. In AndroidWorld's messaging task, the evaluator checks the destination number and message content in the phone's SMS database [27]. Inspecting the resulting state during execution can likewise tell the agent whether the operation needs correction.

3.3 Multi-step Level

Many work objectives cannot be met by a single local change: they depend on intermediate results that are not available at the start. Preparing a forecast, for example, requires importing data before constructing calculations and checking the resulting workbook. Intermediate operations establish information or state on which later decisions depend. Work at the Multi-step Level coordinates these dependent operations toward a complete objective, adapting subsequent actions to intermediate results and execution feedback [117]. Because only the objective is delegated, completion is assessed on the final result, whichever sequence of actions reaches it.

3.3.1 Completing Objectives through Intermediate Results

A complete objective may culminate in a usable artifact, an informational result assembled from acquired evidence, or a change to an operational environment. Intermediate results supply the components and inputs needed to complete that objective. In data-science tasks, data preparation and code execution lead to an executable analysis, as illustrated by DataSciBench and ScienceAgentBench [10, 118]. Artifact production also requires the parts to work together. Slide generation connects content selection and visual planning [119, 120]; each slide must fit the deck's outline and surrounding narrative [5, 17]. In a financial workbook, a value computed on one worksheet may supply calculations on another [121]. Completion therefore depends on both the individual transformations and the relationships among their results.

Intermediate results can also supply the evidence for an answer or plan. TravelPlanner's two-stage setting requires collecting travel options through tools before assembling an itinerary that satisfies the user's constraints [83]. GAIA includes questions that require locating a study on the web before recovering the requested fact [40].

For tasks that change an environment, intermediate changes establish the state needed to complete the requested operation. Terminal-Bench 2.0 includes configuring systems and migrating databases [122]. In AIOpsLab's mitigation tasks, the objective is to restore a running service through interventions such as configuration changes or version rollbacks [123].

Desktop and web tasks carry these intermediate results across applications. In OSWorld, payment records determine the recipients of a reminder email; WebArena's museum-itinerary example combines reference pages and a map before recording the route in a project README [6, 7].

3.3.2 Managing Action Dependencies and Feedback

These dependencies shape the order and content of subsequent actions. A tool result can supply arguments for a later call, as in MCP-Bench and MCP-Universe [124, 125]. An interpretation of business context can determine which data to join or aggregate before a result is computed, as in DABstep [126]. A change of state can enable a previously unavailable operation: ToolSandbox's messaging example requires cellular service to be enabled before the message can be sent [127]. TimeArena's simulations add duration and resource constraints: agents must schedule around unfinished actions and occupied objects while advancing independent work [128].

Feedback from execution can require revising the sequence. Execution feedback guides SheetAgent's workbook editing loop, while Spreadsheet-RL provides recalculated values during execution in Microsoft Excel [45, 129]. In DeepPresenter, rendered slides guide revisions; VIGA likewise executes candidate programs and inspects the rendered scenes before continuing scene edits in BlenderBench [58, 130]. The agent must retain these observations and changes so later actions use the current state.

In many organizational settings, completing a task is only one step within a broader operational process. A finished financial forecast, for example, may need to pass through compliance verification, managerial assessment, and budget allocation before taking effect, where advancing the work depends on procedural rules and interactions with other participants.

3.4 Workflow Level

The Workflow Level widens what is delegated from a single objective to the operational process around it. The process carries the objective through stages with distinct responsibilities and conditions for advancement, and each stage establishes an output, decision, or confirmation on which another stage or participant relies. The agent therefore coordinates the handoffs between stages and the procedural requirements that govern them, including cases where intermediate findings change the remaining work. Because these stages belong to the delegated work, completion covers the process outcome as well as the final output (Section 2.1): a correct forecast still cannot take effect while a required approval is missing. CXBench scores final outcomes and process compliance separately in regulated customer-service tasks, where a run that reaches the correct final state but fails a process check, such as providing a required disclosure or obtaining consent, counts as a false pass [131].

The same criterion separates workflow stages from the stages an agent system introduces into its own execution. Anthropic's Research system, for example, runs a research stage, in which a lead agent decides how many search subagents to create and adds more when the evidence is insufficient, followed by a citation stage that attaches sources to the final report [132]. The requester, however, asks for a sourced answer to a research question, and even an explicit request for multi-agent execution specifies how the system should work without defining any stage on which the result depends. Any other arrangement of stages that yielded an equally well-sourced report would complete the request just as well, so the research and citation stages organize the execution of a multi-step objective rather than forming part of the delegated work.

3.4.1 Carrying Work through Functional Stages

Stages group actions around an output or decision that subsequent work requires. A handoff can make an artifact available under an agreed specification, confirm an input, or authorize further work. In WorkArena++, employee-onboarding tasks connect user creation with ordering and assigning hardware [133]. A similar handoff occurs in an OfficeBench training task: the agent reads an Excel participant list, creates calendar events, and emails the training details to participants [76].

The same dependence on stage outputs appears in artifact production. MetaGPT follows the standard operating procedures of a software team: a requirements document guides architecture and interface design, then implementation and testing [54]. SlideRL uses research findings and an outline to guide slide generation and revision [134]. EXP-Bench links experimental design and implementation to execution and conclusions, while PaperBench requires implementing and executing a paper's experiments to reproduce its empirical results [135, 136]. DAComp makes the handoffs explicit in pipeline tasks that connect cleaning, entity integration, and aggregation under specified data contracts [137].

3.4.2 Coordinating Workflow Progress

Once a stage has produced its output, assessment, confirmation, and policy conditions determine when work can advance. In CHI-Bench's healthcare administrative tasks, assessment can yield a decision or a request for missing information [138]; τ-bench requires user confirmation before specified database updates [29]. In EnterpriseOps-Gym, warranty status determines whether the agent records a customer interaction and schedules a warranty discussion [139]. AutomationBench's meeting-conflict task applies a spreadsheet priority policy, marks the lower-priority meeting for rescheduling, and reports the decision to Slack [140].

Progress also depends on other participants. In τ^2-bench's telecommunications tasks, the agent coordinates its actions with changes made by a tool-using simulated user [30]. In EntCollabBench, roles with different tools and data access pass intermediate outputs to one another, while TheAgentCompany includes tasks requiring information from simulated coworkers [31, 38]. In Collaborative Gym, the agent's literature search and the human's feedback feed revisions to a shared related-work draft [141]. Retaining pending changes and their prerequisites lets the agent integrate each reply into the current workflow. These dependencies can extend across repeated operations: E-Commerce Bench connects procurement, fulfillment, returns, and cash-flow decisions over a simulated year [33].

3.5 Workspace Level

In the preceding forms, the request or task setup largely specifies which objects the work involves. As agents become able to work across entire code repositories and an organization's documents, emails, and chats [11, 12, 142], a request can state only the goal. The agent must then establish which objects are relevant and how their relationships affect completion. A report may depend on a particular workbook version, while an approved decision may need to be reflected in several records. The Workspace Level makes discovering and following these relationships part of the delegated responsibility. The agent locates relevant objects, reconciles their evidence and versions, and, when the task requires updates, preserves consistency among the affected objects.

3.5.1 Locating Relevant Objects in a Workspace

Relevant material may be distributed across documents and spreadsheets, message histories, code repositories, or linked business records. Workspace-Bench makes identifying relevant objects and their dependencies part of the task [36]. GraphForge constructs assignments requiring comparison, reconciliation, or derivation across files, while WorkGenesis requires drawing on public source files and synthesized companion attachments to produce occupational deliverables [143, 144]. OfficeQA Pro's full-corpus tasks require finding the pages needed for an answer, while DRBench's application-based tasks require retrieving enterprise material for a research report [145, 146]. Finding the right page can itself require reading charts and layouts, as in ViDoRe, introduced with ColPali [147]. As a retrieved object points to further sources, the relevant context develops through these connections.

3.5.2 Following Dependencies across Workspace Objects

The connections uncovered during search determine how retrieved objects fit into the task. They may be explicit references, shared entities, or relationships of derivation and version. In CRMArena, identifying which order a customer refers to requires relating product information to that customer's purchase history [148]. LOCA-bench makes these cross-source relationships explicit in a task adapted from Toolathlon: assembling an exam spreadsheet requires matching Canvas announcements and emails to the correct courses [149, 150]. In LocBench, repository tasks similarly require locating the files and functions relevant to an issue description [151]. Repository repair extends this process to modifying code and testing the affected components [3, 16, 152, 153]. Spider 2.0's DBT tasks extend this dependence to data-transformation projects: the agent reads SQL, YAML, and Markdown files to complete a missing transformation and check the resulting database state [154].

Using related objects together can also require reconciling what they say. Agora tasks require deriving a numerical answer from documents whose terminology, units, and time periods must be reconciled, while HERB queries connect people, products, and release events across heterogeneous enterprise records [155, 156]. APEX-Agents makes source selection concrete in a valuation task that combines the implied share price in a discounted cash-flow model with earnings per share from a financial statement for a specified date [157]. Tracing historical versions and derived-content relationships helps establish which sources support the conclusion [36].

Object relationships also determine what must be rechecked when work changes. In OSWorld 2.0, revised budget and vendor constraints arriving through TeamChat require the agent to reconsider affected entries in a purchase-order form [32]. Linking the updated constraints in the chat to the affected form entries identifies which earlier work needs revision. Discovering these dependencies exercises work openness; maintaining the result as relevant inputs change exercises persistence (Section 2.2).

3.6 Synthesis: Work Scope and Capability Demands

Across the five work forms, the foundational capabilities introduced in Section 2 support work over an expanding set of dependencies. As an assignment grows, an interpretation of the evidence can affect decisions and changes across several stages or objects. The agent must carry those dependencies through the work and use feedback from the resulting state to decide what can proceed and what needs revision. Work complexity thus scales through the web of dependencies that these capabilities must manage in concert.

Table 3 relates the five work forms to the capability requirements introduced in Section 2.3. Work at the Workspace Level requires tracking the sources and dependencies behind a result, with consistency checks extending to affected objects when updates are required.

Table 3 Capability demands across the five Work Forms

Work Form	Perception and Understanding	Reasoning and Decision Making	Planning and Coordination	Execution and State Management	Verification, Recovery, and Replanning
Conversation Level	Interpret supplied evidence and context	Derive supported answers and judgments	Organize local reasoning when needed	Produce the requested response or specification	Check evidence support, constraints, and output validity
Atomic Action Level	Identify the target object and local state	Select an appropriate operation	Establish a valid local action path	Commit the intended local change	Confirm the target effect
Multi-step Level	Track intermediate evidence and state	Adapt decisions to intermediate results	Decompose a coherent objective	Maintain state across related actions	Check intermediate and final outcomes
Workflow Level	Identify stages, rules, and shared state	Reason about prerequisites and policies	Coordinate stages, branches, and handoffs	Commit consequential state transitions	Validate stage transitions and workflow outcomes
Workspace Level	Discover relevant objects, relations, versions, and authority	Reconcile evidence across objects	Plan over object and dependency structure	Track object state; preserve consistency when updating related objects	Check source lineage, freshness, and dependencies

This dependency-centered view also clarifies where a work agent needs improvement: missing evidence can interrupt a sequence of actions, an incorrect decision can propagate into later stages, and a changed constraint can invalidate dependent outputs. Section 4 examines how experience from such work can improve the model, the harness, and their interaction, while Section 5 considers how to assess the resulting work and the progress made by these improvements.

4 Optimization: From Work Experience to System Capability

The capabilities required by work agents must be cultivated via systematic optimization. As discussed in Section 2, this optimization can take place at both the model and harness levels. Model updates improve interpretation, reasoning and action selection, while harness updates enhance the interfaces, tools, memory and execution workflows that support the model. These improvements draw on diverse corpora and experience gathered during work, including observations of the environment, action trajectories and feedback on task outcomes. As illustrated in Figure 5, Section 4.1 examines the environments, data, and feedback that provide learning signals. Section 4.2 examines how pre-training, mid-training, and post-training develop model capabilities, while Section 4.3 explores improvements to interfaces, workflows, and memory. Finally, Section 4.4 examines how models and harnesses can adapt together and support continued self-improvement.

4.1 Training Environments, Data, and Rewards

Agents learn to perform work through environments, data and feedback. Environments provide tasks for interaction, while corpora offer relevant knowledge and trajectories show how actions affect task progress. Feedback helps agents identify successful decisions and outcomes. For example, when preparing a report from multiple workbooks, an agent needs to understand how source values relate to calculations and reported findings, and how to update affected results when the source data changes. Table 4 compares methods for constructing environments, curating corpora and trajectories, and designing feedback.

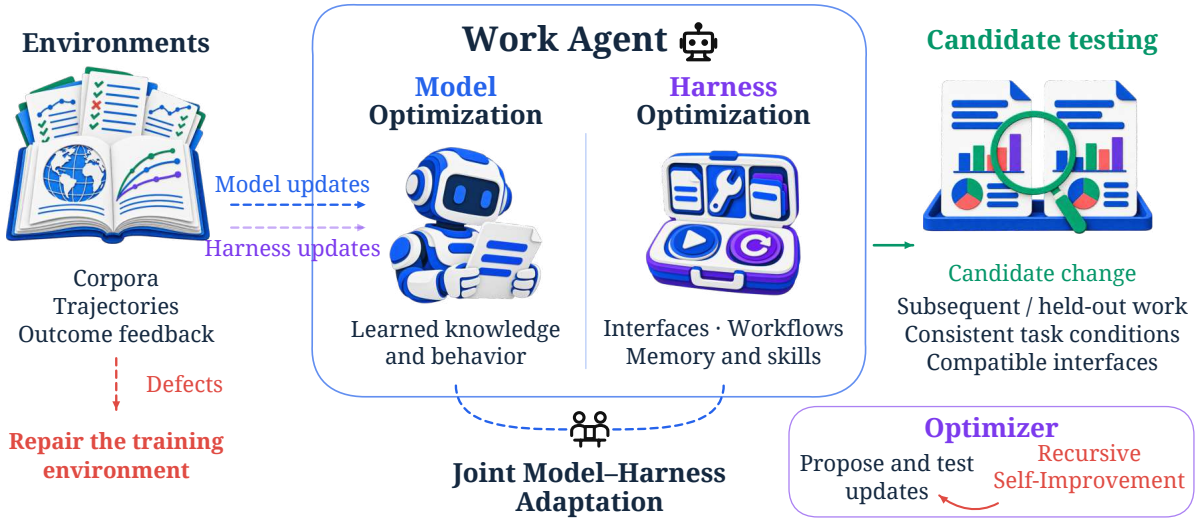


Figure 5 Optimization targets in a work-agent system. Training environments, corpora, trajectories, and outcome feedback inform candidate changes to the model, harness, or their coordination. Model optimization changes learned knowledge and behavior; harness optimization changes interfaces, workflows, memory, and skills. Joint adaptation coordinates model and harness changes. Defects in the training environment may require environment repair. Comparisons on subsequent work under consistent task conditions and compatible interfaces test whether a candidate update helps. The optimizer proposes and tests candidate updates; optimizer-level recursive self-improvement also changes how updates are proposed and tested.

Table 4 Constructing experience for work-agent optimization. Representative methods for constructing work environments, curating corpora and trajectories, and designing feedback. Rows describe the mechanisms used to produce training inputs and learning signals.

Representative work	Main mechanism	Training inputs or feedback
Work Environment Construction		
daVinci-Env [158] Spreadsheet-RL [129] Paper Code Data Website	Reconstruct tasks from repository changes or paired workbooks, with executable operations and result checks.	Initial workspaces, requested changes, and checkable outcomes.
NexForge [159] Paper Website WorkGenesis [144] Paper	Use capability requirements and occupational activities to guide task and resource collection.	Professional requests with supporting work materials.
WorkForge [160] Paper WorkBorne [161] Paper	Expert workflows (WorkForge) and artifact transitions (WorkBorne) guide task construction from real files.	Source materials, dependent operations, and intended outputs.
Agent World Model [162] Paper Code Data AgentMercury [163] Paper Data Website	Connect databases, business entities, and tool interfaces across applications.	Executable tasks with related records and changing state.
EnvCraft [164] Paper Code CompoWorld [165] Paper	Build dependent task chains (EnvCraft) or compose reusable services through task graphs (CompoWorld).	Workflows that require information and results from earlier operations.

Continued on the next page

Table 4 continued.

Representative work	Main mechanism	Training inputs or feedback
CoworkEnv [166] Paper Code Cowork-Gym [167] Paper	Validate requests, records, tools, and checks together (CoworkEnv); include a stateful user (Cowork-Gym).	Training interactions with workflow constraints, user input, and workspace changes.
WorkRover [168] Paper	Diagnose incompatible environment components and replay candidate repairs under fixed requirements.	Repaired environments that support further execution and data collection.
Corpus and Trajectory Curation		
daVinci-Dev [169] Paper Code Data	Link issues, repository context, and edits; combine reconstructed histories with real tool executions.	Context-rich work records and action–observation sequences, including failed attempts.
AgentTrek [170]	Filter web tutorials, replay their instructions in real environments, and verify the resulting trajectories.	Demonstrations linking task instructions to observations and executed actions.
EnterpriseLab [171] Paper Code Website	Derive tool dependencies from schemas and pass valid returned values into later calls.	Executable tool sequences and task descriptions that preserve data dependencies.
OpenThoughts-Agent [172] Paper Code Data	Compare task sources, teachers, mixtures, and filters through downstream training.	Selected trajectory mixtures evaluated for transfer across agent tasks.
CurateEvo [173] Paper	Use development-set failures to revise augmentation, filtering, and refinement.	Updated SFT and RL data and inference-time memory from a fixed raw corpus.
Feedback Design		
Spreadsheet-RL [129] Paper Code Data Website Anchor [174] Paper Code Website	Recalculate workbook outputs (Spreadsheet-RL) or check business constraints and solution quality (Anchor).	Outcome signals from the resulting workspace state.
GraphForge [143] Paper Data SlideRL [134] Paper Code Data	Link rubrics to sources (GraphForge); combine content, structure, rendering, and specification recovery (SlideRL).	Artifact feedback on factual support, presentation, and fulfillment of the brief.
CompoWorld [165] Paper	Give more weight to rubric items with lower pass rates within a rollout group.	Partial-progress reward; the maximum is reached only when all criteria pass.
VERA [175] Paper WorkRover [168] Paper	Check stage requirements (VERA); trace verified progress back to supporting turns (WorkRover).	Stage-level feedback and credit for earlier decisions that support verified progress.

4.1.1 Work Environment Construction

Training environments convert work requirements into tasks that agents can execute and evaluate. Each task includes an initial workspace, available actions, a request, and checks for evaluating the results. These components can be constructed from existing work records. For example, daVinci-Env reconstructs software tasks from repositories, and Spreadsheet-RL uses paired workbooks and an Excel runtime [129, 158]. Real-world tasks often depend on information spread across documents, applications, and people. Therefore, training environments should preserve these dependencies so that agents can learn which actions to take and how to verify their results.

Grounding tasks in work materials. Recent pipelines connect work materials to requirements in several ways. NexForge organizes task generation around capabilities and task forms, while WorkGenesis builds occupational requests with supporting resources [144, 159]. GraphForge represents evidence across retrieved files as a graph, then uses it to generate instructions and source-linked rubrics [143]. WorkForge starts from expert workflow plans, retrieves real files, and identifies facts that guide the instructions, solution plans, and programmatic and semantic checks [160]. WorkBorne organizes tasks around artifact transitions: a specified

actor transforms input materials toward reference outputs [161]. Together, these approaches cover different aspects of task construction, from generating realistic requirements to grounding them in evidence and defining the expected results.

Constructing shared and changing state. Tasks spanning multiple applications also require consistent states and operations. Agent World Model creates database-backed tool environments, EnvCraft builds sequences of dependent tasks, and AgentMercury constructs business environments with shared entities and constraints [162–164]. CoworkEnv jointly generates requests, service records, completion checks, and permitted actions, then validates their consistency before collecting teacher trajectories [166]. Cowork-Gym introduces a stateful user who can provide information, revise requirements, and modify the shared workspace [167]. Such changes can affect later actions. For example, an updated customer record may change which operation is valid, while a revised workbook may invalidate an earlier analysis. WorkRover repeatedly checks and repairs consistency among the context, task, harness, and verifier [168]. These methods highlight the importance of maintaining a valid and consistent environment as tasks progress.

CompoWorld extends environment construction to tasks involving multiple services [165]. It builds reusable services with typed states and common interfaces, uses a world model for tools that are difficult to implement reliably, and samples dependency graphs across services. Each service maintains its own state, while the agent transfers information between services through observations and actions. This allows cross-service dependencies to be controlled alongside the number of services. In a matched comparison using the same backbone, training procedure, and 1K SFT trajectories, AutomationBench success increases from 12.83% with single-service environments to 33.33% with composed environments.

Scaling coverage, depth, and validation. Training environments can scale by adding more workspaces, more tasks within each workspace, or more rollouts per task. These choices expose agents to different materials, task dependencies, and ways to complete or recover from work. WorkBorne compares broader artifact coverage with more transitions per corpus under matched trajectory budgets [161]. Broader coverage performs better at 1K, 3K, and 5K trajectories, while both strategies achieve similar results at 8K. WorkForge reports that increasing training data from 1K to 16K trajectories improves ALE by 13.0 points and WildClaw by 8.4 points [160]. Its comparisons using equal numbers of trajectories also favor longer interactions, although these trajectories differ in task content and decision depth. Together, these findings highlight the importance of tracking both task diversity and interaction structure alongside training volume.

Reported dataset sizes also refer to different outputs of generation pipelines. WorkForge constructs 16,665 tasks, GraphForge retains 2,169 training trajectories from 2,153 selected tasks, and WorkBorne produces 7,882 task environments and 15,485 trajectories from 1,061 artifact corpora [143, 160, 161]. These counts are difficult to compare without specifying what is measured and at which stage. Thus, we suggest reporting generation attempts, accepted tasks, usable trajectories, and verification costs to assess how efficiently environment construction produces useful training experience.

Validation is equally important as environments scale. An executable task may still contain incomplete requirements, inconsistent sources, or a verifier that accepts incorrect results. Validation must establish both that a task can be completed and that its verifier accepts correct solutions while detecting errors. It must also cover any new materials introduced during scaling. In WorkForge's matched 1K-trajectory ablation, combining programmatic and semantic checks achieves an ALE score of 22.11, compared with 15.62 for semantic-only and 14.89 for programmatic-only selection [160]. GraphForge's corruption audit shows that deleting a worksheet reduces the target rubric score by 0.377, while finer row-level, numerical, and citation errors change scores by less than 0.03 on a 0–1 scale [143]. These results suggest combining source-based judgments with exact checks of critical values and using deliberate errors to test verifier reliability. Failed checks and their dependencies can also guide environment repairs, as discussed in Section 4.4.3.

Beyond scale and validation, training environments vary in difficulty, openness, persistence, and business fidelity. Difficulty reflects the decisions and dependencies involved in completing a task, while openness reflects what the agent must discover during execution. Persistence concerns how long the task state and responsibilities remain relevant, and business fidelity describes how closely the materials and constraints reflect real work. Human participants may also provide clarification, corrections, approvals, or final assessments

at different stages. A useful training set should combine controlled tasks that expose specific failures with interactive workspaces that require multiple capabilities. Evaluation on new business tasks can then test whether these capabilities transfer to unfamiliar materials and conditions.

4.1.2 Corpus and Trajectory Curation

Corpora provide the materials agents work with, while trajectories record how agents use those materials to complete tasks. Documents, code, and completed artifacts offer domain knowledge and examples, while interaction records capture observations, actions, and their results. Curation selects and organizes these sources according to the capabilities being trained. For example, repository-level data can link issues to relevant files and edits, while execution trajectories also capture tool outputs and failed attempts leading to those changes [169]. Preserving this context helps models learn what actions to take and why. Task, workspace, and trajectory identifiers support coverage analysis and data splits based on shared sources. Interactive tasks also require records of changing requirements and actions taken by other participants [167]. CoworkEnv collects teacher demonstrations from validated environments, where SFT improves the mean Claw-Eval score from 46.6 to 64.1. The same environments also support student RL comparisons [166].

Generated trajectories should preserve the dependencies between actions. AgentTrek uses tutorials and guided replay to construct web interaction trajectories [170]. EnterpriseLab derives tool dependencies from enterprise schemas and reuses identifiers returned by earlier calls in later operations [171]. TDScaling organizes tools into business scenarios and generates examples for underrepresented domains and reasoning patterns [176]. Exploration offers another way to collect experience. SEE builds a verified graph of mobile interface transitions and generates tasks involving interruptions and recovery [177]. Scenario-based generation provides workflows with clear goals, while exploration captures states that standard demonstrations may miss. Successful trajectories show how to complete tasks, and recovery segments demonstrate how to proceed when execution is disrupted.

Data selection should reflect the capabilities being trained. Artifact checks can filter for successful outcomes, while failed attempts may still provide useful examples for recovery and state tracking. OpenThoughts-Agent evaluates task sources, teachers, data mixtures, and filtering strategies through downstream training, including comparisons under matched token budgets [172]. CurateEvo uses development-set failures to improve data augmentation, filtering, and refinement [173]. As policies improve, repeated easy successes provide less useful information, while errors involving changed inputs, missed requirements, or costly recovery can guide further data collection. Construction and development data should remain separate from held-out evaluation tasks. When tasks share files, splitting data by workspace or source family provides a stronger test of whether models can generalize to unfamiliar materials and requirements.

4.1.3 Feedback Design

Feedback turns work requirements into learning signals. When success can be measured from the resulting workspace, direct checks are possible. Spreadsheet-RL recalculates edited workbooks in Excel and compares relevant regions against target workbooks [129]. Anchor derives business tasks and verifiers from constraint programs to check feasibility, solution quality, and relationships among records [174]. Other tasks require judgments about content and presentation. Preference-based reward learning uses human comparisons [178], GraphForge links rubric items to source evidence, and SlideRL evaluates slide structure, rendering, and content [134, 143]. CogEvol also checks rendered content and browser interactions [179]. The appropriate feedback depends on the output, which may require checks of its values, organization, and behavior. Requirements specify what counts as success, while verifiers assess the supporting evidence. Keeping these roles separate allows missing checks or parsing errors to be corrected without changing the task itself.

Feedback can also focus on requirements that agents frequently miss. The Completion-Focused Rubric Reward of CompoWorld assigns greater weight to criteria with lower pass rates within a rollout group [165]. It rewards partial progress but gives the maximum reward only when all criteria are satisfied. This encourages agents to address the remaining steps needed to complete a workflow.

Final outcomes alone do not reveal which actions contributed to success. Intermediate feedback helps connect

results to earlier decisions [180, 181]. VERA restores workspace states and checks intermediate requirements using executable tests and rubrics [175]. WorkRover's Verifier-to-Turn Credit Assignment uses recorded dependencies to assign bounded positive credit to earlier turns that contributed to verified progress [168]. For example, reading a source may receive credit if its information supports a later correct change. Stage checks identify when progress occurs, while dependency records help determine which earlier decisions contributed to it. Both require sufficient records of workspace states and their relationships.

Reliable feedback must also account for changing requirements. Cowork-Gym links requirements to success conditions, supporting evidence, and checkers while allowing users to introduce new constraints [167]. Maintaining current requirements alongside relevant artifacts helps agents detect omissions when earlier instructions leave the active context or source data changes. Verification also has a cost. Structural and state checks can detect common errors efficiently, while semantic inspection and human assessment handle requirements that need more context. As execution becomes faster and more parallel, verification may become a bottleneck. Thus, we propose using dependencies among sources and artifacts to determine which checks need to be repeated, which evidence remains valid, and when further assessment is necessary. Feedback design should also account for the verification and rework required before an output can be accepted.

4.2 Model Optimization

Model optimization develops the knowledge and decision-making capabilities agents need to perform work. Pre-training builds broad semantic and computational knowledge; mid-training adapts this foundation to domain materials, interaction patterns, and long context; post-training strengthens behavior through demonstrations, execution feedback, and teacher guidance. Figure 6 connects these roles. The model first learns about work objects, then how to use their interaction histories, and how to improve decisions from task outcomes. The stages differ in their data mixtures, prediction targets, and training schedules, while token prediction and agent trajectories contribute across stages.

4.2.1 Pre-training

Pre-training establishes the world knowledge and general capabilities that work agents rely on to understand unfamiliar tasks and environments. Through large-scale learning from text, code, mathematics, and other materials, models acquire factual knowledge, domain concepts, common conventions, and computational skills. These are essential for professional work, where requests often assume knowledge that users do not explicitly provide. For example, an agent revising a budget report must understand financial categories and units, interpret relationships between tables and written claims, and recognize how formulas or code produce numerical results. This background knowledge allows agents to interpret workspace materials, understand task requirements, and reason about possible actions before adapting to specific workflows. Work materials can in turn enrich pre-training with structured and multimodal prediction signals. For autoregressive language models, pre-training uses the standard next-token prediction (NTP) objective. Given a token sequence x, the model learns to predict each token from its preceding context by minimizing

$$\mathcal{L}_{\mathrm{NTP}}(\theta) = -\mathbb{E}_{x\sim\mathcal{D}} \sum_{t=1}^{|x|} \log p_\theta(x_t \mid x_{<t}), \tag{3}$$

where $\mathcal{D}$ denotes the pre-training corpus. Under this objective, the selection and organization of training data determine what knowledge and relationships the model encounters. DeepSeek-V4, for example, combines web data with mathematics, programming, and long technical documents [182]. Work materials further expose cell addresses, formulas, layout, and relations between inputs and computed outputs. DocLLM uses text-infilling pre-training over document text and bounding-box layout [183]. Such structure can support executable prediction targets: TAPEX pre-trains a model to predict SQL execution results from queries and serialized tables, improving downstream table reasoning [184]. This illustrates how structured data can develop computational skills alongside factual knowledge.

Additionally, work agents must recognize relevant knowledge when it appears in different forms. A task requirement may be described briefly in a user request, explained in detail in a reference document, or

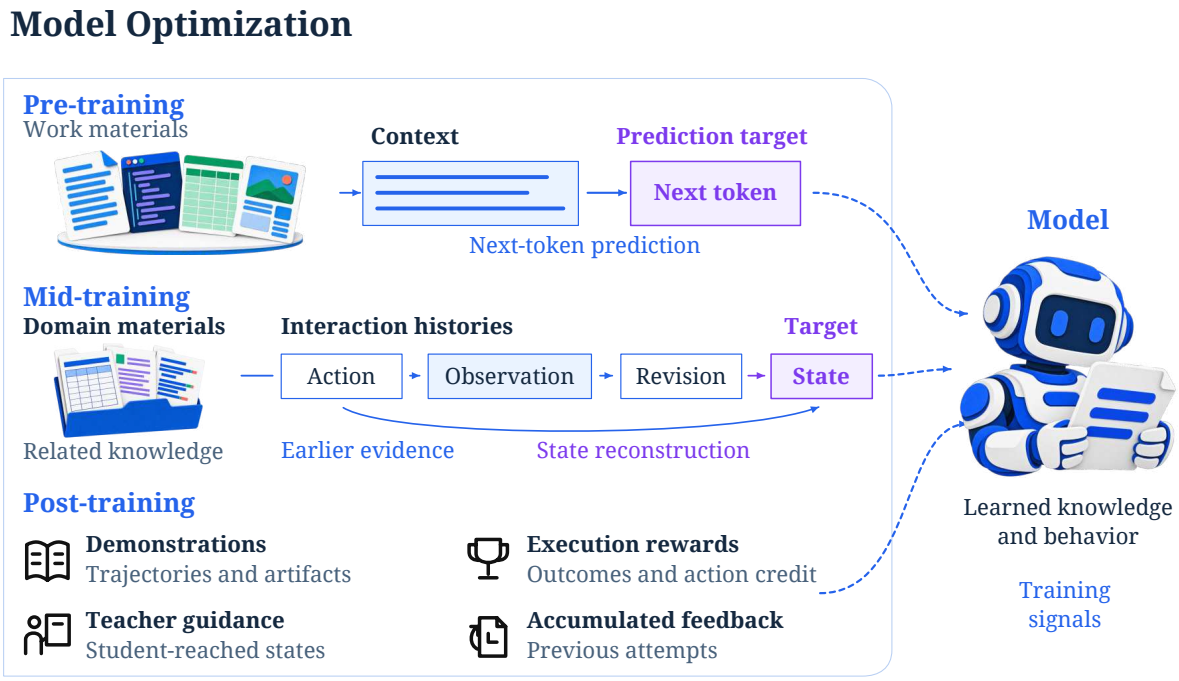


Figure 6 Training emphases that develop the model. Illustrative samples distinguish visible context from prediction targets and feedback. Pre-training builds semantic, computational, structural, and visual knowledge. Mid-training enriches domain knowledge, introduces interaction patterns, and develops long-context use in preparation for subsequent optimization. Post-training uses demonstrations, execution rewards, teacher guidance, and accumulated feedback to supervise decisions, trajectories, artifacts, or earlier attempts. Links to the model denote learning signals, not live execution. Inputs and objectives can overlap across stages; the samples do not prescribe a shared dataset or universal training recipe. Supervision granularity differs across the four post-training groups.

reflected in the content of an output artifact. Kimi K2 addresses this challenge by rewriting knowledge-rich passages in different styles and from different perspectives while preserving their original meaning [185]. Its early-checkpoint experiment reports 28.94% SimpleQA accuracy for ten distinct rephrasings used once, versus 23.76% for ten repetitions of the original text. For work agents, this principle could extend to aligned views of the same content, such as source cells, rendered charts, and report passages. Preserving their correspondences could provide training signals for maintaining semantic consistency across formats and associating claims with supporting data.

Working environments also contain visual information that agents must interpret alongside text and code. Kimi K2.5 jointly trains on text and visual inputs, including screenshots and image–code pairs; its pre-training corpus also includes dense OCR, multi-page documents, GUI screenshots, and fine-grained grounding annotations [186]. Dense work artifacts require reading small text, distinguishing visually similar elements, and interpreting spatial relationships. Pairing rendered workbooks or slides with native object structure could extend this supervision to locating cells, interpreting chart elements, and connecting visible content to its underlying representation. Such data could strengthen transferable perception and understanding capabilities (Figure 3), establishing the knowledge foundation for subsequent reasoning, planning, execution, and verification.

4.2.2 Mid-training

Targeted Knowledge Enrichment. Work agents often encounter tasks that require knowledge beyond the information provided in a user request. Understanding an unfamiliar repository, interpreting business records, or following domain-specific conventions may depend on knowledge that is underrepresented in general

pre-training data. Mid-training addresses this gap by increasing the density of specialized and long-tail knowledge while preserving the relationships among relevant materials. GLM-5 organizes related files, issues, pull requests, and commits into training sequences, then increases the proportion of long documents and synthetic agent trajectories in later stages [187]. Step 3.5 Flash begins with software engineering and tool-use data at 32K context, retaining 21% pre-training replay, before expanding to 128K with long documents, synthetic reasoning, and agent data [188]. Kimi K2.5 similarly combines high-quality text and multimodal materials while extending context from 32K to 256K [186]. These approaches increase exposure to specialized work knowledge while using replay to preserve broader capabilities. For work agents, organizing knowledge around related files, requirements, and revisions is particularly useful, as understanding a work object often requires information distributed across multiple sources.

Agent Interaction Familiarization. Domain knowledge alone cannot prepare an agent for the interactions required to complete work. An agent must also understand what a tool call changes, how to interpret its response, and what to do when an operation fails or produces an unexpected result. Mid-training can introduce these interaction patterns before post-training begins to optimize task behavior. DeepSeek-V4 incorporates agentic coding data at this stage [182]. Nemotron combines repositories, pull requests, edit histories, and build logs in a 500B-token mixture at 256K context, including 10% agent traces and 20% pre-training replay [189]. With subsequent SFT held fixed, this stage improves SWE-bench Verified Pass@1 from 60.7% to 64.5%. daVinci-Dev further combines trajectories reconstructed from pull requests with trajectories collected through actual tool execution [169]. Its mid-training uses both successful and failed executions, while later SFT focuses on test-passing trajectories. This distinction is important for work agents, as unsuccessful interactions can expose errors and intermediate states that success-only demonstrations may omit. Learning from these records helps models become familiar with the observation, action, and state-change patterns involved in execution. This provides a stronger starting point for post-training, where feedback can focus on improving decisions and completing tasks reliably.

Long-Context Learning and Post-Training Readiness. Work agents frequently perform tasks in which later decisions depend on information or changes from much earlier interactions. For example, modifying a source workbook may affect several calculations and invalidate parts of a previously generated report. Simply extending the context window does not ensure that a model can identify these dependencies or reconstruct the current workspace state. TrajLong addresses this problem by transforming interaction histories into retrieval, information-combination, and file-state reconstruction tasks using full-token next-token prediction [190]. Under a fixed 40B-token budget and identical later SFT, these tasks outperform raw-trajectory training on all seven reported ablation benchmarks. Function-aware fill-in-the-middle training similarly uses code dependencies to select functions or function groups for reconstruction, improving the mean across SWE-bench Verified and Lite from 13.95 to 15.60 in a controlled 7B comparison [191]. The choice of prediction targets also matters. In Nemotron's 100B-token ablation, masking tool responses while retaining them as context raises SWE-bench Verified Pass@1 from 27.5% to 37.8% after identical SFT [189]. This distinction is relevant to agent training because tool responses provide evidence for subsequent decisions, even when the model does not need to generate those responses itself. Efficient access to long histories presents another challenge. Qwen3.8-Flash-Next introduces sparse attention during continued training at 256K context to reduce the cost of accessing relevant information [192], connecting this stage to the memory mechanisms in Section 4.3.3. Ultimately, the value of mid-training should be measured by its contribution to later optimization. Step 3.5 Flash reports stronger and more stable RL gains after incorporating task knowledge and tool-use patterns [188], while Nemotron compares recipes through identical SFT runs because intermediate model metrics do not reliably predict downstream agent performance [189].

4.2.3 Post-training

Post-training develops the policies that allow work agents to apply their knowledge to real tasks. Beyond understanding a request, an agent must select appropriate actions, respond to changes in the workspace, and determine whether its results satisfy the requirements. Different training methods provide supervision for these decisions. Demonstrations show how tasks can be completed, execution rewards evaluate the consequences of actions, teacher guidance improves decisions at states encountered by the agent, and retrospective learning extracts useful information from previous attempts. Table 5 summarizes these methods according to their

learning signals and update mechanisms.

Learning from demonstrations. Supervised fine-tuning (SFT) teaches agents to follow instructions, use tools, and produce completed work through examples [193–195]. Synthetic instructions provide additional supervision before task-specific interaction [196]. For work agents, demonstrations can capture both the steps taken during execution and the structure of an accepted deliverable. Kimi K2 generates tool-use trajectories by constructing domains, tools, and tasks, simulating interactions, and filtering the results, while also incorporating real execution environments into post-training [185]. Interactive demonstrations teach agents how to respond to observations and tool feedback [197, 198]. Artifact-based supervision instead connects task requirements to completed outputs. CogEvol trains on verified representations of slides and interactive pages [179], while GraphForge uses source-linked rubrics to select generated solutions for rejection fine-tuning [143]. CoworkEnv provides curated teacher executions that improve multi-turn performance on Claw-Eval, with the same executable environments supporting subsequent RL [166]. Together, these approaches teach agents how to carry out tasks and what acceptable results should look like.

Learning through execution rewards. Reinforcement learning improves an agent's policy through the outcomes of its own interactions [199–201]. Web navigation, device control, and repository repair provide different environments in which actions can be executed and evaluated [202–204]. For artifact-producing tasks, Spreadsheet-RL evaluates the resulting workbook, while SlideRL combines structural, visual, and content feedback to assess generated slides [129, 134]. However, work agents often complete tasks through many dependent steps, making it difficult to determine which earlier decisions contributed to the final result. Intermediate rewards and credit assignment help address this problem. ADMIRE derives adaptive milestones from successful GUI trajectories, while HiPER learns hierarchical policies with subgoals and associated value estimates [205, 206]. GiGPO combines episode-level advantages with comparisons of actions taken from repeated environment states [207]. WorkRover uses recorded dependencies to assign credit to earlier turns that support verified progress [168]. These methods provide different ways to connect final outcomes with intermediate decisions, allowing agents to learn from both completed work and progress made along the way.

Reliable execution feedback also requires accurate records of the interactions being trained on. Mercor and SkyRL's knowledge-work recipe uses 1,928 expert-created tasks and emphasizes environment repair, exact token histories, and small-set overfitting tests before scaling RL [208]. Its loss ablations compare weighting at the token, sequence, and prompt-group levels, with prompt-group weighting reducing the influence of unusually long trajectories. The resulting 397B model improves held-out APEX-Agents Pass@1 from 16.11% to 27.29%. These practices are particularly important for work agents, where tool failures, inconsistent environments, and long execution histories can make rewards unreliable. Preserving the exact observations and actions behind each reward helps distinguish policy errors from infrastructure failures and keeps learning aligned with the interaction that actually occurred.

Learning from teacher guidance. Demonstrations alone may not cover the states an agent encounters during its own execution, especially after incorrect actions or unexpected tool responses. On-policy distillation addresses this gap by providing teacher guidance at states reached by the student [209]. TCOD gradually increases student-controlled interaction, while Guided-OPD reduces teacher-controlled turns and combines teacher imitation with distillation on student-generated states [210, 211]. This allows supervision to follow the developing policy more closely [212, 213]. Step-wise on-policy distillation adjusts guidance based on student–teacher divergence [214]. Other methods improve supervision at specific decisions. LiteGUI selects a human-verified valid action that matches the student's attempted action, while Gated Hindsight Distillation uses a later screenshot to guide an earlier decision [215, 216]. Foundation-model training also uses on-policy distillation to combine specialized capabilities or preserve them across training stages [182, 187]. For work agents, these methods are useful because guidance can address errors and partial progress encountered during actual execution, rather than relying only on previously collected demonstrations.

Learning from accumulated feedback. Work agents can also learn from information accumulated across attempts, including corrections, execution feedback, and completed results [217–219]. SDPO provides textual feedback to a self-teacher and distills its improved predictions into the acting policy, while Skill-SD additionally conditions the teacher on short summaries of earlier attempts [220–222]. RePro uses completed attempts to teach models which intermediate decisions indicate useful progress [223]. These approaches reuse information

revealed during execution to improve future behavior [224]. For example, when generating a report from multiple sources, demonstrations can teach the initial workflow, execution rewards can assess the completed report, and teacher guidance can help correct errors encountered during revision. Retrospective feedback can then turn these corrections into useful experience for subsequent tasks. Section 4.4.3 extends this process beyond model updates to systems that also improve their environments, external procedures, and methods of learning.

Table 5 Post-training methods for work agents, grouped by their supervision and update mechanism. The final column describes the information used for learning and its purpose.

Representative work	Learning mechanism	Learning signal and purpose
Demonstrations and accepted work products		
AgentTuning [193] Agent-FLAN [194]	Supervised mixtures of agent interactions.	Action formats, tool-use patterns, and instruction-following behavior.
CogEvol [179] Paper Code CoworkEnv [166] Paper Code	Supervision from checked artifacts or curated teacher executions.	Deliverable structure and multi-turn behavior grounded in an executable task.
GraphForge [143] Paper Data	SFT followed by rubric-selected rejection fine-tuning.	Source-linked acceptance criteria select useful self-generated solutions.
Execution rewards and credit assignment		
Spreadsheet-RL [129] Paper Code Data Website SlideRL [134] Paper Code Data	Policy rollouts receive artifact-level rewards.	Numerical, structural, visual, and semantic properties of the produced artifact.
ADMIRE [205] Paper HiPER [206] Paper Code Website	Milestone-based feedback and hierarchical subgoal learning.	Progress through multi-step tasks and decisions between subgoals.
GiGPO [207] Paper	Episode advantages plus relative action credit at repeated anchor states.	Global outcome quality and local action effectiveness in comparable states.
WorkRover [168] Paper	Verifier-to-turn credit assignment.	Checked progress and supporting tool/artifact dependencies.
Mercor / SkyRL [208] Report Code	Asynchronous RL with exact token accounting and group-balanced loss.	Rewards remain aligned with the sampled multi-turn interaction.
Teacher guidance at student-reached states		
TCOD [210] Paper Code Guided-OPD [211]	Increase student-controlled interaction while transferring teacher guidance.	Supervision follows the states encountered by the developing policy.
SOD [214] LiteGUI [215] Paper GHD [216] Paper	Adapt teacher guidance using divergence, verified actions, or future observations.	Targets for correcting errors observed during execution.
Feedback-conditioned and retrospective learning		
SDPO [220] Paper Code Data Website SDAR [222]	Feedback or compact experience summaries condition a self-teacher.	Improved continuations transfer lessons from earlier attempts to the policy.
RePro [223] Paper	Completed attempts supervise retrospective progress learning.	Evidence about which intermediate decisions support later success.

4.3 Harness Optimization

Work agents rely on external support to inspect information, execute operations, and manage work across multiple steps. Even with a capable model, missing evidence, unreliable tools, or poorly organized procedures can prevent successful task completion. Harness optimization improves this external support through observation and action interfaces, execution workflows, and reusable experience. Interfaces determine what the model can observe and how it interacts with the workspace. Workflows organize execution, verification, recovery, and delivery, while context management, memory, and skills preserve task progress and useful procedures across interactions. Execution traces reveal where these components need improvement, such as missing information, repeated tool errors, or unnecessary rework. Table 6 compares representative methods based on what they change and how feedback guides their optimization.

Table 6 External mechanisms for organizing and improving work execution. The middle column identifies what each method changes; the final column explains its effect on later decisions.

Representative work	Editable object and feedback	Effect on work execution
Observation and action interfaces		
SWE-agent [3] Paper Code Website TACO [225] Paper Code	Agent-oriented repository operations and execution-driven output filters.	Show the state needed for decisions while controlling inspection and context cost.
Trace-Free+ [226] JTPRO [227] Paper	Learned tool descriptions or coordinated instruction/description edits.	Align tool choice and arguments with the intended operation.
PRISM [228] Paper Code Data	Prompts and executable middleware, revised from traces and state changes.	Repair recurring calling errors and enforce operation prerequisites.
Workflows and execution control		
GEPA [229] Paper Code Website AFlow [230]	Feedback-guided prompt revision and workflow-graph search.	Reorganize reasoning, tool use, and inspection across a task.
Meta-Harness [231] Paper Code Website AutoDesign [232] Paper Code Data Website	Harness code or design procedures, revised from repeated execution failures.	Turn recurring defects into reusable production and recovery changes.
JIT-Agent [233] Paper Code Data Website	A learned designer produces task-specific harnesses.	Organize execution using the frozen agent's reward, latency, and cost.
Context and reusable experience		
MemGPT [234]	Movement between active context and external storage.	Recover information whose relevance extends beyond the current context window.
ReasoningBank [235] Paper Code ACE [236]	Retrieved strategies and a playbook updated over time.	Carry lessons from successful and failed executions into later tasks.
SkillOpt [237] Paper Code Website SkillAlpha [238] CodeSkill [239] Paper	Validated or learned edits to procedural documents and skill libraries.	Improve the instructions that guide recurring work operations.
SkillSmith [240] Paper	Library composition and the boundary between procedural skills and tools.	Coordinate reusable procedures under dependencies and a context budget.

4.3.1 Observation and Action Interfaces

Observation interfaces determine what information an agent can inspect before making a decision. For work agents, this often involves locating relevant evidence within large repositories, documents, or tool outputs. SWE-agent introduces agent-oriented repository operations that make software files easier to search, inspect, and edit [3]. TACO further improves observations by discovering terminal-output filters, refining them during execution, and reusing effective rules across tasks [225]. These filters reduce unnecessary output while preserving relevant state and error information. The main consideration is whether an interface provides the evidence needed for the next decision without consuming excessive context or requiring repeated inspection.

Action interfaces determine how an agent translates its decisions into operations on files, applications, and services. Unclear tool descriptions or inconsistent argument conventions can cause repeated errors, even when the intended operation is appropriate. Trace-Free+ trains a tool-description generator, gradually moving from supervision based on execution traces to generation using tool descriptions alone [226]. JTPRO jointly optimizes global instructions and tool descriptions using execution feedback [227]. The former learns tool-use conventions that can transfer across tool sets, while the latter coordinates instructions within a particular environment. Both aim to improve tool selection and argument generation, helping agents carry out the operations required by a task.

Executable support can also correct errors during tool execution. PRISM analyzes tool traces, state changes, and errors to improve prompts, middleware, or both [228]. Middleware can correct arguments, check prerequisites, and provide error information that supports recovery. These mechanisms are particularly important when work agents modify shared records or related objects. For example, checking a file version before writing helps ensure that an edit applies to the same object the agent previously inspected. Tool responses should also indicate what changed so the agent can decide whether to continue or recover. Evaluating interfaces with the same model and tasks helps isolate improvements from external support. In the Mercor/SkyRL study, correcting tool behavior and observation handling improves the base agent before weight training, while the training gains also transfer to a held-out code-based harness [208]. Such comparisons help distinguish improvements in the execution environment from improvements learned by the model.

4.3.2 Workflows and Execution Control

Workflow optimization improves how an agent organizes and executes work across multiple steps [241, 242]. The editable components range from prompts [243–245] and connections between model calls [246, 247] to executable harness code [248, 249]. Prompt optimization can use feedback-guided revision [250, 251], reinforcement learning [252], or evolutionary search [253, 254]. More recent approaches extend these ideas to complete execution procedures. GEPA uses execution traces and outcomes to refine prompts [229], AFlow searches over workflow graphs [230], and Meta-Harness modifies harness code [231]. These methods provide different levels of control over how an agent performs work [255]. For example, an improved workflow may inspect additional evidence before editing, verify intermediate results before dependent operations, or introduce a recovery step after failure. Its effectiveness should be measured by whether these changes improve execution on subsequent tasks.

Artifact production illustrates how feedback from individual tasks can improve reusable workflows. AutoDesign uses rendering, validation, and critic feedback to revise a poster, then analyzes recurring failures to propose changes to its design harness [232]. This separates improving the current deliverable from improving the procedure used for future tasks. JIT-Agent takes a related approach by training a separate model to construct task-specific harnesses using execution reward, latency, and cost from a frozen executor [233]. The designer learns how to organize execution and produces an explicit procedure for the acting agent. These approaches show how experience from completed work can guide the design of more effective workflows without requiring changes to the underlying executor.

Execution control must also preserve progress when work is interrupted, requirements change, or responsibilities are transferred. For ongoing tasks, a workflow needs clear conditions for continuing, revising previous results, and determining when delivery is complete. These conditions relate to the requirements and responsibilities discussed in Section 2. For example, a report awaiting source confirmation should retain that dependency, while

a completed deliverable should be checked against the latest requirements before the task closes. Repeated inspections, retries, and human corrections provide evidence of where workflow changes could reduce delays and rework. Methods based on retrospective self-preference, adaptive curricula, and task-dependent harness routing further use feedback to select and deploy suitable procedures [256–258]. Evaluating both the quality of accepted results and the effort required to produce them helps ensure that workflow optimization improves real task execution.

4.3.3 Long Context, Memory and Skills

Context management determines which parts of a growing work history remain available for each decision. Work agents may accumulate lengthy tool outputs, repeated observations, and information that becomes outdated as the workspace changes. A larger context window alone does not resolve these issues. MemGPT manages information between active context and external storage, while TACO adjusts the amount and form of tool output retained during execution [225, 234]. For ongoing work, the active context must also preserve current requirements, relevant object versions, and unresolved dependencies. These records help agents determine whether earlier observations and decisions remain valid. Effective context management therefore supports efficient access to information and reliable continuation after interruptions or revisions.

Memory optimization focuses on preserving useful experience across tasks. ReasoningBank extracts strategies from successful and failed attempts for later retrieval [235], while ACE maintains an evolving playbook through generation, reflection, and curation [236]. Skills extend this idea by representing reusable procedures that guide future execution. SkillOpt improves procedural documents through validated edits [237], SkillAlpha learns editing policies and evaluates them through downstream execution [238], and CodeSkill extracts, revises, and maintains a library of coding skills [239]. These approaches allow agents to reuse lessons from previous work rather than rediscovering procedures for every task. Execution feedback also helps determine which instructions remain useful and which should be revised as new situations arise.

As reusable experience accumulates, agents must decide which procedures to retain, how to combine them, and when to apply them. SkillSmith addresses interactions and dependencies among skills under a limited context budget and considers which operations should be represented as skills or executable tools [240]. Stable operations can be implemented as tools, while skills provide guidance on when and how to use them. This distinction is important for work agents that repeatedly perform related tasks with different materials and constraints. Task-specific state and reusable experience also require different update schedules. A change to a source workbook should immediately update the current task record, while a revised reporting procedure should be validated across multiple tasks before broader reuse. Recording the sources and dependencies of both enables targeted updates. For example, a reporting skill may describe how to reconcile information from several workbooks, while the task record identifies which workbook version has been checked and which approval remains pending. Maintaining this distinction allows agents to reuse established procedures while keeping their execution aligned with the current workspace.

4.4 Model--Harness Coupling and Joint Optimization

The effectiveness of a work agent depends on how well its model and harness work together. The model must learn to use the information and tools provided by the harness, while the harness must offer interfaces and procedures that the model can use reliably. This raises an important question of where improvements should be implemented. A capability may be learned through model training, provided through retrieved context, encoded as a reusable skill, or implemented directly in control code. We examine how models are trained within a harness, how the two components can be optimized together, and how repeated execution can improve both the system and its optimization process. Table 7 summarizes the components updated by these methods, while Figure 7 illustrates their feedback loops.

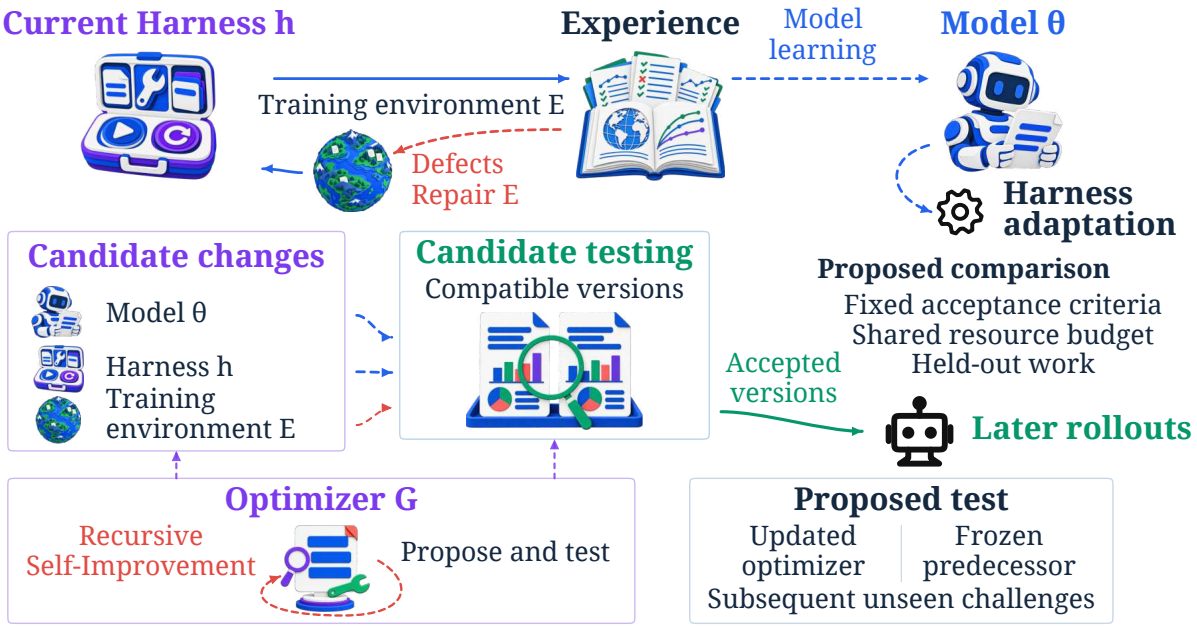


Figure 7 Model–harness coupling and improvement across training executions. The current harness shapes experience used to train the model; model updates can motivate harness adaptation. Evidence informs candidate changes to executor weights θ, harness h, or training environment E. Optimizer G proposes and tests changes, and accepted versions supply subsequent rollouts. The paths illustrate possible component updates, without prescribing their order or requiring every target to change. A separate relation denotes recursive self-improvement of G's search and diagnostic/testing procedures. Fixed acceptance criteria, held-out work, and a shared resource budget are our proposed comparison conditions. We further propose comparing the updated optimizer with its frozen predecessor on subsequent unseen challenges.

Table 7 Joint adaptation and self-improvement, organized by the component changed in each update. θ denotes executor weights, h its harness, ϕ a separate designer's weights, E a training environment package, and G the optimizer's procedure. The last group gives within-study comparisons of the role and cost of verification.

Representative work	Updated object	How updates use feedback
Policy and harness adaptation		
UI-Copilot [259] Paper Code Agent Lightning [260]	θ within a supplied execution harness.	Training consumes decisions made through the available observations, tools, and control flow.
BetterTogether [261] Fast-Slow Training [262] Paper Website	Prompts in h and executor weights θ.	Alternate external prompt search with model updates, refreshing the conditions for each.
Co-Harness [263] VERA [175] Paper	θ and external procedures or skills in h.	Improved procedures shape training rollouts; policy outcomes guide later external revisions.
UI-Mem [264] Paper Website SkillRL [265] Paper Code Data SLIM [266] Paper Code	θ and reusable memory or skills.	Experience retained outside the model guides learning and changes as the policy improves.
SIA [267] Paper Code	Select an update to θ or h.	Failure traces and reward changes guide the choice between policy learning and scaffold edits.

Continued on the next page

Table 7 continued.

Representative work	Updated object	How updates use feedback
Learned designers and adaptive practice		
JIT-Agent [233] Paper Code Data Website DMRL [268] Paper	Designer weights ϕ; generated harnesses or skill documents in h.	Frozen-executor outcomes train a separate design or editing policy; DMRL uses business feedback.
Socratic-SWE [269] Paper	Shared generation/solving policy and a trace-derived skill registry.	Role-specific objectives connect task generation to the solver's experience.
CurateEvo [173] Paper	Data augmentation, filtering, and refinement procedures.	Development failures guide which training material is retained and produced next.
Environment evolution and optimizer-level recursion		
WorkRover [168] Paper	Environment components in E; separate policy post-training.	Versus applying all repairs: cross-task regressions are 0.7% versus 7.0%; normalized model-token cost is 1.00 versus 0.59.
VERSE [270] Paper Code	Executor harness h and optimizer procedure G; model weights fixed.	Controlled study, test accuracy (%): baseline 39.20; verification only 39.81; self-evolution only 31.79; both 41.05. One evolution run per condition.

4.4.1 Harness-Aware Policy Training

A training harness determines what a policy can observe and do. Tool schemas define available operations and their arguments, context management controls access to previous interactions, and execution rules determine how actions affect the workspace. Harness-aware training allows models to learn under these conditions. Spreadsheet-RL trains agents with a spreadsheet runtime and toolset, while UI-Copilot provides structured observations, progress summaries, and a copilot for retrieval and calculation [129, 259]. Through these interactions, the policy learns when to rely on internal reasoning and when to use external tools. The deployment harness must therefore provide the interfaces and support that the policy has learned to use.

Training systems can also optimize a policy while preserving the agent's existing execution process. Agent Lightning connects policy optimization with agent execution, allowing the harness to manage interactions while the training system learns from the resulting decision records [260]. The available tools, memory access, and execution rules all influence the experience collected during training. For example, a model trained with compressed observations needs to learn how to retrieve information that is no longer available in its active context. The training harness therefore shapes the situations and decisions the policy learns to handle, even when only model weights are updated.

Models can also learn to construct tools for future tasks. SMITH alternates between tool construction and tool use, adds generated tools to a reusable pool, and rewards their validity and downstream usefulness [271]. As the available tools evolve, the policy learns how to use the capabilities it creates. For work agents, this provides a way to turn repeated operations into reusable executable tools while learning when to construct and invoke them [272]. Feedback from subsequent execution helps determine which generated tools are useful, allowing the model to improve based on their contributions to later tasks.

4.4.2 Joint Model--Harness Adaptation

Joint adaptation coordinates model learning with changes to external support, which can be revised at a different pace [273]. BetterTogether alternates between prompt and weight optimization, while Co-Harness uses improved harnesses to generate model-training trajectories and then adapts the harness to the updated model [261, 263]. Fast-Slow Training similarly alternates prompt search with blocks of reinforcement-learning updates [262]. In these approaches, each component is optimized using experience collected with the current version of the other. This interaction matters because a procedure that compensates for an earlier model's limitations may become unnecessary after training, while a stronger model may be able to use workflows that were previously unreliable.

Memory and skills provide concrete components for joint adaptation. UI-Mem extracts experience during GUI reinforcement learning and updates memory guidance as the policy improves, while SkillRL maintains a skill library alongside policy training [264, 265]. VERA alternates rubric-guided model updates with validated additions, revisions, and removals in a skill library organized around work stages [175]. SIA uses execution trajectories and reward changes to decide between scaffold editing and policy reinforcement [267]. These approaches allow improvements to target the model, its external support, or both, depending on the observed failures and the capabilities already available.

An important question is where learned experience should be stored. Component Routing compares experience incorporated through weight updates with the same experience provided as context [274]. In its evaluated GUI settings, recurring patterns for locating interface elements favor weight updates, while state-dependent procedures and facts benefit more from context. The effectiveness of transferred procedures also depends on the receiving model. For work agents, this suggests learning recurring behavioral patterns in model weights while keeping changing facts, task commitments, and operating rules in external records. To evaluate these choices, we propose comparing (M_0, H_0), (M_1, H_0), (M_0, H_1), and (M_1, H_1) under consistent task conditions and compatible interfaces. For ongoing work, each configuration can resume from the same saved workspace after an identical source change or user intervention. Comparing these runs with fresh starts would reveal whether updates preserve unfinished dependencies, support necessary rechecks, and improve task completion while accounting for verification costs and human intervention.

4.4.3 Self-Improvement and Recursive Self-Improvement

Work agents may operate over extended periods while the environments in which they work continue to change. New tools, data formats, business rules, and organizational processes can gradually make previously reliable procedures less effective. Although context and memory can help agents recognize these changes, substantial shifts may require persistent updates to their models, workflows, skills, and verification mechanisms. Self-improvement addresses this need by using execution experience to adapt the agent and its supporting environment. Recursive self-improvement (RSI) goes further by improving the mechanisms through which the system identifies limitations, develops new capabilities, and evaluates its own progress. These two directions offer complementary benefits for work agents. Self-improvement supports continued adaptation to changing work, while RSI offers a potential path toward more fundamental growth in agent capabilities.

Adapting the source of practice. Self-improvement allows agents to accumulate experience and revise the capabilities and procedures they use in later work. Socratic-SWE extracts strategies and recurring failures into a skill registry, then alternates task generation and solving through a shared policy with role-specific objectives [269]. Generated tasks guide subsequent training, while new execution traces inform later task generation. CurateEvo uses development failures to improve data augmentation and filtering [173], while SkillRL and SLIM update a procedural library alongside policy training [265, 266]. These methods improve future behavior by changing what the agent practices and which procedures it can reuse. Beyond benchmark tasks, DMRL learns a policy that edits the skill instructions of a frozen advertising-system agent using online outcomes and estimates of longer-term effects [268]. Such feedback connects self-improvement to operational needs, where an agent must adjust its behavior as requirements and working conditions evolve.

Maintaining reliable training environments. Continued learning also requires environments that provide valid tasks and feedback. GraphForge revises generated tasks and rubrics using execution feedback before collecting training trajectories [143], while WorkForge constructs and filters environments through a forward generation pipeline [160]. WorkRover supports repeated environment repair by diagnosing compatibility failures and testing candidate changes through Factorial Compatibility Replay [168]. It selects the smallest repair set that satisfies fixed requirements and preserves necessary dependencies, then validates the result through fresh execution. Its natural-defect study reports 70.3% safe repairs, compared with 60.7% for full regeneration, with rates remaining within 70.3–72.4% across three repair generations. Table 7 further compares regressions and token costs. These methods highlight the need to distinguish failures caused by the policy from defects in the training environment. An incorrect decision may require further learning, while missing materials or inconsistent verifiers require environment repair. Maintaining reliable learning environments helps agents improve from experience as their training needs change.

From adaptive improvement to RSI. Self-improvement can also be guided directly by the outcomes of real work. DMRL learns a policy that edits the skill instructions of a frozen advertising-system agent using online outcomes and estimates of longer-term effects [268]. This illustrates how business feedback can guide improvements to reusable procedures. RSI introduces a further possibility by allowing the optimization process itself to evolve. VERSE explores this direction by keeping the executor and optimizer model weights fixed while modifying the executor harness and the optimizer's prompts, skills, tools, hooks, and notes [270]. Its controlled comparison evaluates execution verification and optimizer self-evolution separately and together, with the combined setting achieving the best results. Without executable checks, self-evolution selects the initial harness and produces a lower test result. Although these results are based on one evolution run per condition, they illustrate how changes to the optimization process can influence subsequent improvements.

These studies provide different perspectives on how improvement should be evaluated. DMRL connects procedural updates to business outcomes [268], WorkRover examines the reliability and cost of repeated environment repairs [168], and VERSE separates the effects of optimizer evolution and executable verification [270]. For work agents, we propose extending these comparisons to longer periods and changing work conditions. Self-improvement can be evaluated by how effectively an agent adapts while preserving unfinished responsibilities and maintaining the quality of accepted work. RSI requires an additional comparison between the updated optimizer and its frozen predecessor, testing whether improvements to the optimization process enable more effective responses to subsequent, unseen challenges. Such evaluations should measure task success, adaptation costs, verification effort, and the recurrence of previously resolved failures. Table 8 links observed failures to candidate optimization targets and the work-agent capabilities discussed in Section 2.3.

Table 8 Choosing an optimization target from observed failures. The comparisons are proposed tests; the cited studies provide methods for the corresponding changes.

Failure evidence	Candidate change	How to test the update
An error recurs with the required evidence visible	Train interpretation or action patterns [169, 188]	Same harness; held-out objects and requests testing the recurring decision
Evidence is absent, truncated, or difficult to retrieve	Revise observations and retrieval [3, 225]	Same model; evidence recovery, task success, and context cost
A dependency is lost after interruption or source revision	Revise workflows and external state storage [231, 234]	Resume from saved state; check unfinished dependencies and affected artifacts again
An artifact marked complete fails an independent requirement check	Revise requirement capture and verification [167, 175]	Independent requirement checks; missed defects, false rejection, and assessment effort
Replay diagnoses incompatible context, task, harness, or verifier components	Repair the training environment [168]	Check that the repaired task runs, preserves requirements, and supports policy transfer to other environments

Together with the continuation protocol, these comparisons suggest evaluating work agents across successive changes in tasks, tools, requirements, and shared workspaces. Targeted failures can test specific improvements, while held-out workflows and later business tasks can assess their transfer. Beyond immediate task success, evaluations should track preserved dependencies, necessary rechecks, human intervention, and the time and cost of accepted work. For RSI, repeated evaluations should further determine whether improvements to the optimization process help agents acquire capabilities needed for increasingly unfamiliar work. This distinction provides a basis for studying both sustained adaptation and longer-term capability growth.

5 Evaluation and Resources: Assessing Work Agents

Rigorous evaluation is essential for understanding the capabilities and limitations of current models on work tasks and guiding the improvement of work agents. By verifying whether delegated work meets explicit acceptance criteria, evaluation reveals where agents succeed, where they fail, and which capabilities require further development. These findings provide empirical feedback for the improvement methods discussed in Section 4. To serve this purpose, evaluation must clearly define the delegated work, specify its acceptance criteria, and collect appropriate evidence of completion. Answers, files, application state, and complete

workflows have different requirements, so assessment must examine the relevant deliverables, workspace state, and execution process rather than rely on the final response alone. Because measured performance depends jointly on the model, tool interface, execution environment, and verification procedure, benchmark scores must be interpreted alongside the tasks and protocols that produced them.

This section covers task construction and domain coverage (Section 5.1), the targets, dimensions, and methods used to evaluate work results (Section 5.2), and the metrics used to summarize those judgments (Section 5.3). It also introduces the execution frameworks and evaluation infrastructure that support task execution and verification (Section 5.4). The final part examines the relationships among task performance, execution cost, and economic value, including how human oversight affects deployment returns (Section 5.5).

5.1 Benchmarks and Task Construction

A work-agent benchmark is scoped by two choices: how its tasks are constructed and which work domains they cover [275, 276]. Task construction determines how requests, materials, and execution conditions are produced [277, 278], and thus how realistic and how controllable the resulting tasks are. Domain coverage determines the professional knowledge, work objects, and acceptance requirements involved, and thus how representative the benchmark is of the work agents are asked to perform.

5.1.1 Task Construction Strategies

Existing benchmarks mainly use four approaches: expert design, conversion of real requests, construction in interactive environments, and construction from professional workspaces. Each emphasizes a different task source and verification condition; Table 9 lists representative instances.

Expert-designed and curated tasks. Researchers or domain experts design tasks around knowledge, reasoning, and operational requirements, and supply reference answers, labels, or explicit acceptance criteria. GAIA, FinQA, and LegalBench cover information-seeking reasoning, financial computation, and legal judgment respectively [21, 40, 101]; CADBench (BlenderLLM) refines modeling instructions into check items such as attributes and spatial relations [85]. This approach makes difficulty and requirements easy to control, but it does not necessarily include every interaction condition of a real delegation.

Tasks built from real requests and existing task collections. These methods extract tasks from real problems, user needs, or existing professional data. SWE-bench uses GitHub issues and their associated code repositories, and SpreadsheetBench curates workbook requests from user forums [16, 56]; CADBench (MIT) instead builds program-reconstruction tasks from existing CAD data [86]. The advantage is that real work objects are preserved, although task coverage depends on the data source and the filtering procedure.

Tasks built around applications and execution environments. These methods place tasks in interactive software environments, constructing repeatably executable tasks by initializing application state, providing operation interfaces, and setting acceptance conditions. WebArena, OSWorld, AndroidWorld, and AppWorld provide web, desktop, mobile, and multi-application API environments respectively, and verify task results against the actual state after execution [6, 7, 27, 37]. Such methods can assess an agent together with the operations it actually performed, but task coverage remains limited by the environment design and by the states that can be verified.

Tasks built from professional activities and workspaces. These methods organize multi-step, multi-object tasks around occupational requests and actual delivery requirements. GDPval supplies professional work tasks authored by occupational experts, TheAgentCompany uses simulated enterprise services, and WorkspaceBench concerns multiple files and their dependencies [31, 36, 279]; StartupBench and BankerToolBench further emphasize complete professional delivery and fine-grained acceptance [280, 281]. Compared with a single operation, these tasks come closer to a complex delegation, but their construction and verification costs are usually higher.

Taken together, the four approaches trade controllability for realism. Expert-designed tasks specify requirements most precisely, whereas workspace tasks reproduce the most of an actual delegation. In between, real requests contribute authentic content and environments contribute executable context.

Table 9 Task construction approaches, sources, and input–output forms in representative benchmarks and an associated agent system.

Benchmark / system	Task source	Input form	Output form
Expert-designed and curated tasks			
GAIA [40] Paper Data	Cross-domain information and reasoning problems	Text prompt; optional files and external information	Concise text answer
DocVQA [22] Paper Data Website	Document images with human-annotated questions	Document image and question	Text answer
FinQA [21] Paper Code Data	Financial reports and expert-designed questions	Question, report text, and tables	Numerical answer and reasoning program
FrontierFinance [282] Paper Code Data Website	Investor research queries written by finance professionals across six use cases	Query, as-of date, and access to filings, market data, and the web	Long-form investment research answer
LegalBench [101] Paper Code Data	Legal cases, rules, and documents	Task instruction and legal text	Task-specific label or answer
CADBench (BlenderLLM) [85] Paper Code Data	Simulated modeling tasks and online forum requests	Natural-language modeling instructions	Blender script and 3D model
BlenderBench (VIGA) [130] Paper Code Data Website	Curated 3D scene reconstruction and editing tasks	Target images, scenes, and editing requirements	Blender scene and rendered result
BenchCAD [283] Paper Code Data Website	Procedurally generated industrial part families anchored to engineering standards	Part images, CadQuery code, and editing instructions	CadQuery program, edited program, or numeric answer
Real requests and existing task collections			
SWE-bench [16] Paper Code Data Website	GitHub issues and the associated code repositories	Issue description and codebase	Code patch
SWE-agent (system) [3] Paper Code Website	Existing software-engineering tasks and execution environments	Issue description and executable repository	Code edits and patch
SpreadsheetBench [56] Paper Code Data Website	Spreadsheet requests from user forums	Natural-language instruction and workbook	Modified workbook
CADBench (MIT) [86] Paper Code Data Website	Existing CAD model and program datasets	Meshes, single-view or multi-view images	Executable, editable CAD program
Applications and execution environments			
WebArena [6] Paper Code Data Website	Functional web applications	Request and browser environment	Answer or target website state
VisualWebArena [284] Paper Code Data Website	Visual webpages and user requests	Request, browser, and visual content	Answer or target website state
OSWorld [7] Paper Code Data Website	Desktop applications and workflows	Request and initialized desktop environment	Changed files or application state
OSWorld 2.0 [32] Paper Code Data Website	Long-horizon tasks in real desktop applications	Request, work files, and desktop environment	Cross-application state and complete deliverables
AndroidWorld [27] Paper Code Data Website	Parameterized tasks in Android applications	Request and initialized mobile application	Changed application state
LOCA-bench (up to 256K) [150] Paper Code Website	Simulated MCP services and controlled state growth	Request, tool interfaces, and continuously growing context	Tool-operation results and target state
Professional activities and workspaces			
PPTC [112] Paper Code Data	Presentation creation and editing tasks	Instructions and presentation state	Created or modified slide deck
PresentBench [17] Paper Code Data Website	Presentation requests and background materials	Prompt and task materials	Generated slide deck
GDPval [279] Paper Data Website	Professional work across occupations	Work request and reference materials	Professional deliverables

Continued on next page

Table 9 (continued)

Benchmark / system	Task source	Input form	Output form
Workspace-Bench [36] Paper Code Data Website	Professional tasks with file dependencies	Request and multi-file workspace	Workspace deliverables
StartupBench [280] Paper Website	Professional workflows with evidence of market adoption	Work request and workspace materials	Complete professional deliverables
Agents' Last Exam [285] Paper Data	Professional tasks designed by industry experts	Work request, materials, and software environment	Verifiable professional work products

5.1.2 Domain Coverage and Work Requirements

Task sources determine how tasks are built, not which work they cover. Domain coverage matters because the evidence needed to accept a completed task depends on the work object. In finance, law, and healthcare, the object is mainly an answer or a professional judgment, and acceptance rests on factual grounding, professional rules, and result correctness [21, 101, 116]. In office and document work, it is the delivered file, whose content, native structure, and computational or visual effects must be checked [56, 112]. In customer service and enterprise operations, it is the state of business systems, subject to permissions and operational constraints [139].

Software engineering, data science, and scientific research usually produce executable or reproducible results, verifiable through tests, database queries, predictive outcomes, or scientific conclusions [10, 16, 154, 286, 287]. CAD and 3D graphics additionally require program validity, geometric fidelity, and rendering quality to hold together [85, 86, 130]. Web, desktop, mobile, and API settings, by contrast, are interaction settings reusable across domains; they assess the environment state after an operation rather than professional knowledge, and are therefore listed as a separate group [6, 7, 27, 37].

Cross-domain benchmarks combine several work objects and acceptance requirements: GDPval examines occupational work products, Remote Labor Index authentic remote tasks, and Workspace-Bench and Startup-Bench complex workspace delivery [36, 279, 280, 288]. Table 10 and Figure 8 summarize the work objects and requirements of each group. Together, they illustrate the range from localized question answering to complete professional tasks.

Figure 8 groups benchmarks by their primary work domain or setting and summarizes representative knowledge requirements.

5.2 Evaluation Methodologies

Evaluating a work agent involves three decisions: which result to inspect, which properties to judge it on, and how to form the judgment. We refer to these as the evaluation target, the evaluation dimensions, and the evaluation method. Targets are the actual answers, artifacts, or environment states; dimensions include correctness, completeness, structure, visual quality, and constraint compliance. Section 5.2.1 covers targets and dimensions, Section 5.2.2 covers methods, and Section 5.3 discusses how judgments are aggregated into scores.

5.2.1 Evaluation Targets and Evaluation Dimensions

By the result inspected at final acceptance, benchmarks fall into four levels: answer, artifact, environment state, and workflow. A task may yield several kinds of evidence; its level is set by the object on which final acceptance is decided.

Answer-level evaluation. The result is text, a number, a label, or a structured response, checked for correctness, completeness, and consistency with reference information. ConvFinQA executes the model's reasoning program for a chain of interdependent questions about a financial report and compares the result with the gold answer [98]; MMLongBench-Doc asks questions over PDFs averaging 47.5 pages, some of which the document cannot answer [299]. CRMArena has agents query a Salesforce organization but scores only the

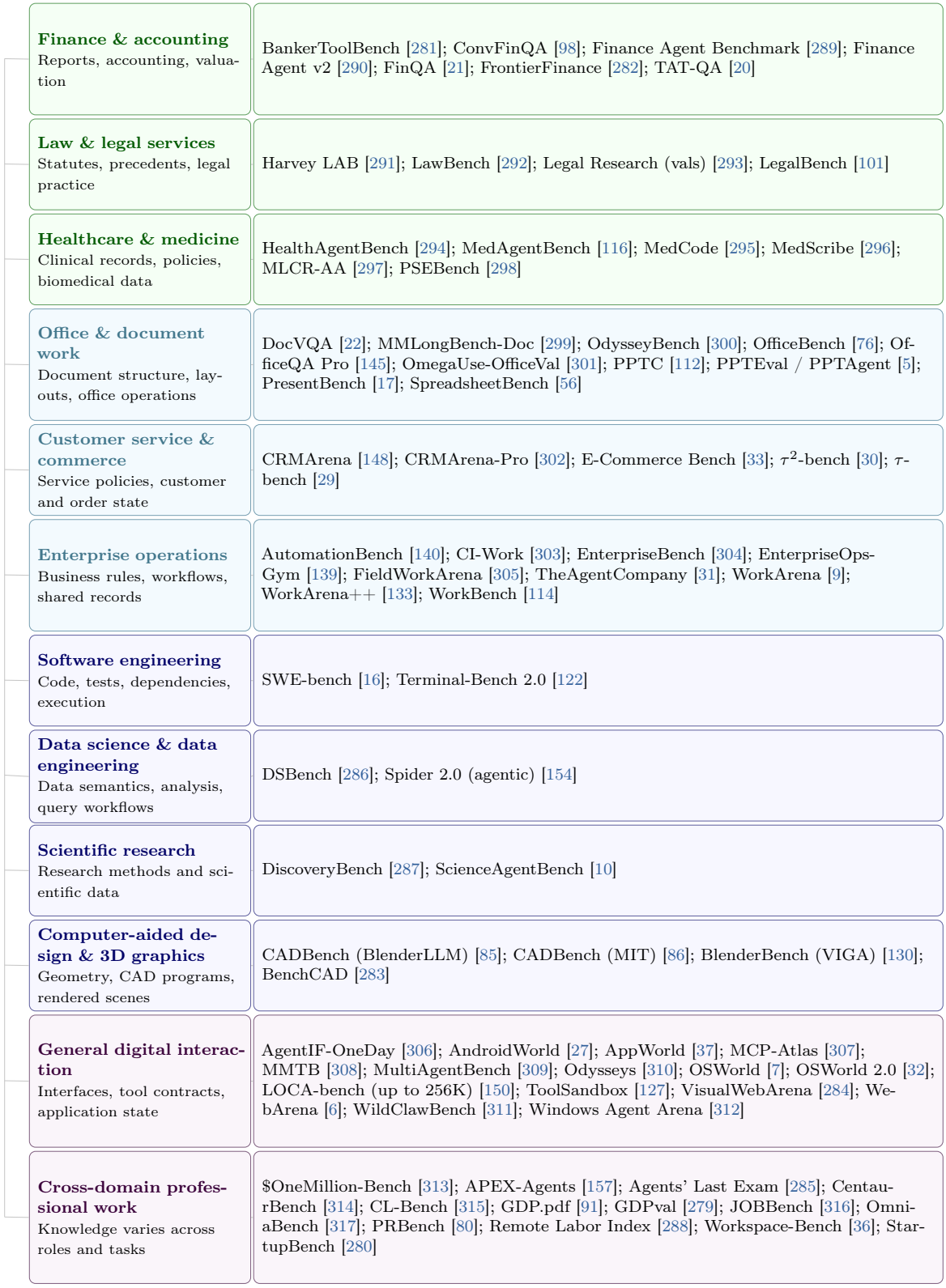


Figure 8 Work domains, settings, and representative knowledge requirements across 79 benchmarks. Studies are grouped by their primary domain or work setting, following the groups of Table 10. Each study appears once for readability; the groups are non-exclusive and connecting lines indicate membership, not a temporal sequence.

submitted answer [148]. Answer-level evaluation suits well-defined outputs, but a correct answer does not establish that the operations behind it, such as CRMArena's queries, were correct.

Artifact-level evaluation. The result is a deliverable the agent created or modified, such as code, a document, or a model file. ScienceAgentBench runs each generated program and checks that it executes and that its output meets task-specific success criteria [10]; OmegaUse-OfficeVal requires Office deliverables to be valid, openable, and editable before scoring them against weighted requirements, with penalties for unintended damage [301]. Because each artifact type has its own requirements of use, text similarity cannot replace structural and functional checks.

Environment-state-level evaluation. The result is the state of a website, application, or business system after execution. Terminal-Bench 2.0 tests the final container state rather than the agent's commands [122], VisualWebArena checks the final webpage state with model-based checks where visual content matters [284], and WorkBench compares five sandbox databases with a ground-truth outcome while counting mismatching side effects [114]. Checking state rather than actions admits different action paths to the same goal, but the check must also catch unintended modifications and violated constraints.

Workflow-level evaluation. The result is a multi-step, cross-application, or multi-file delivery, and evaluation covers the final result, intermediate requirements, and dependencies among objects. ToolSandbox matches the trajectory against milestones that must occur and minefields that must not, scoring zero if any minefield is hit [127]. MultiAgentBench uses an LLM-based detector to record which milestones a team reaches and who contributes [309], and Odysseys has an LLM judge assess the full trajectory against each rubric item [310]. This level comes closest to a complete delegation, but it must make clear which process constraints belong to the acceptance conditions.

5.2.2 Evaluation Methods

Given the collected evidence, automated benchmarks judge it mainly through rules, LLM judges, or agent judges; occupational experts assess open-ended work products or provide reference judgments.

Rule-based evaluation. Programs verify the result by answer matching, execution, or assertions over files and state. AutomationBench checks the end state of simulated business applications with deterministic assertions, including changes that must not occur, and gives no partial credit [140]; OfficeBench saves the file system after each task and applies exact, fuzzy, or execution-based checks [76]. Such checks are clear and repeatable but establish only the requirements their tests cover.

LLM-as-a-Judge. A model judges the result against task requirements, reference material, and a rubric, which suits semantic, professional, and visual qualities that resist formalization [318, 319]. MCP-Atlas has an LLM check each required factual claim in the final answer and passes a task at 0.75 claim coverage [307]; Finance Agent v2 uses a jury of three frontier models [290]; and CI-Work judges, entry by entry, whether an agent's final action conveys required information or discloses sensitive information [303]. These scores depend on the criteria, the reference evidence, and the judge model, and are not objective fact.

Agent-as-a-Judge. An evaluating agent gathers evidence itself by reading files, calling tools, or inspecting traces [15]. Workspace-Bench lets its judge inspect output files, inputs, rubrics, and the execution trajectory with shell tools and file parsers [36]; BankerToolBench gives its judge the same tools as the evaluated agent, so it can run code to check spreadsheet formulas [281]; and StartupBench runs a separate judge session per rubric item, which agreed more often with domain experts than a single holistic judge [280]. Agent judges reach evidence scattered across files, but extra tool execution can cause missed checks or unstable judgments.

These methods are often combined: WildClawBench pairs deterministic checks and side-effect audits with an LLM or VLM judge for reports and images [311], and CRMArena-Pro scores answers by exact match while a GPT-4o judge assesses confidentiality awareness [302]. For open-ended professional work, expert judgment remains the reference; in Remote Labor Index, trained evaluators and subject experts decide, as a reasonable client would, whether an AI deliverable completes the project at least as well as the human one [288]. Table 10 links the targets, dimensions, methods, and metrics of each benchmark.

Table 10 Evaluation targets, evaluation dimensions, evaluation methods, and model performance metrics in work-agent benchmarks. Metrics summarize only the requirements covered by the stated checks or rubrics.

Benchmark	Level	Inspected work result	Main evaluation dimensions	Method	Model performance metrics
Finance & accounting					
BankerToolBench [281] Paper Code Data	Artifact	financial models, presentations, and professional memoranda	Financial correctness, professional requirements, delivery quality	Agent	Weighted Rubric Score
ConvFinQA [98] Paper Code Data	Answer	multi-turn financial question answering	Numerical correctness, contextual reasoning	Rule	Execution Accuracy
Finance Agent Benchmark [289] Paper Code Website	Answer	financial retrieval and research answers	Conclusion correctness, evidence support	Rule	Accuracy
Finance Agent v2 [290] Code Website	Answer	financial research results	Answer correctness, weighted checks with dealbreakers	LLM	Partial Credit, All-Pass
FinQA [21] Paper Code Data	Answer	numerical answer and reasoning program	Computational correctness, program correctness	Rule	Execution Accuracy, Program Accuracy
FrontierFinance [282] Paper Code Data Website	Answer	long-form investment research answers	Source-attributed factual accuracy, analytical completeness	LLM	Macro-averaged Rubric Qualification Rate (all and must-have rubrics), Cost, Latency
TAT-QA [20] Paper Code Data Website	Answer	table and text question answering	Answer correctness, textual and numerical matching	Rule	Exact Match, F1
Law & legal services					
Harvey LAB [291] Code Data	Artifact	legal documents and professional deliverables	Legal professional requirements, content completeness	LLM	Task Pass Rate, Criteria Pass Rate
LawBench [292] Paper Code Data	Answer	legal question answering and judgment	Legal understanding, reasoning, and text-processing correctness	Rule	Accuracy, F1, ROUGE-L, and other task metrics
Legal Research (vals) [293] Code Website	Artifact	legal research results	Research correctness, professional requirement satisfaction	LLM	All-Pass Accuracy, Weighted Partial Credit
LegalBench [101] Paper Code Data	Answer	legal labels and answers	Correctness of legal judgment	Rule	Accuracy, Balanced Accuracy
Healthcare & medicine					
HealthAgentBench [294] Paper Code Data Website	Workflow	medical analysis and tool execution	Task completion, correctness of medical results	Rule + LLM	Task Success Rate
MedAgentBench [116] Paper Code	Environment state	FHIR medical records	Query correctness, record-operation correctness	Rule	Task Success Rate
MedCode [295] Website	Answer	clinical coding	Coding accuracy, agreement with reference codes	Rule	Coding Accuracy
MedScribe [296] Website	Artifact	clinical notes and SOAP documents	Clinical information completeness, documentation quality	LLM	Rubric Score
MLCR-AA [297] Code Data Website	Answer	long medical record question answering	Completeness, no contradiction with the source record	LLM	Pass Rate, Completeness, Accuracy
PSEBench [298] Paper	Workflow	patient-safety event triage	Triage correctness, clause identification, evidence grounding	Rule + LLM	Verdict Accuracy, Clause Accuracy, Evidence Citation F1
Office & document work					
DocVQA [22] Paper Data Website	Answer	document image question answering	Answer correctness	Rule	ANLS

Continued on next page

Table 10 (continued)

Benchmark	Level	Inspected work result	Main evaluation dimensions	Method	Model performance metrics
MMLongBench-Doc [299] Paper Code Data Website	Answer	long-document question answering	Long-document understanding, answer correctness	LLM + Rule	Overall Accuracy, Overall F1
OdysseyBench [300] Paper	Workflow	cross-application office tasks	Task completion, correctness of file and application state	Rule	Pass Rate
OfficeBench [76] Paper Code Data	Workflow	cross-application Office operations	Task completion, state and file correctness	Rule	Pass Rate
OfficeQA Pro [145] Paper Code Data Website	Answer	professional PDF question answering	Answer correctness	Rule	Exact Match Accuracy
OmegaUse-OfficeVal [301] Data	Artifact	Office file deliverables	Content correctness, structure, and delivery requirements	Rule	Score, Time-Weighted Score, Price-Weighted Score
PPTC [112] Paper Code Data	Artifact	native PPTX	Instruction satisfaction, object attributes and positional relations	Rule	Instruction Accuracy, Session Accuracy
PPTEval / PPTAgent [5] Paper Code	Artifact	presentations	Content quality, design quality, coherence	LLM	Content, Design, and Coherence Scores
PresentBench [17] Paper Code Data Website	Artifact	presentations	Content quality, visual presentation	LLM	Overall Score, Dimension Scores
SpreadsheetBench [56] Paper Code Data Website	Artifact	Excel workbooks	Computational behavior, correctness of target results	Rule	Execution Accuracy
Customer service & commerce					
CRMArena [148] Paper Code Data	Answer	answers to CRM queries	Answer correctness	Rule	Exact Match, F1, pass^k
CRMArena-Pro [302] Paper Code Data	Answer	answers in single- and multi-turn CRM tasks	Answer correctness, confidentiality awareness	Rule + LLM	Task Completion, Confidentiality Awareness
E-Commerce Bench [33] Paper Code Data Website	Workflow	long-horizon e-commerce operation	Operating results, business decisions, execution performance	Rule	Year-end Total Assets
τ^2-bench [30] Paper Code Data	Workflow	customer service and device operation	Goal completion, policy compliance, repeat stability	Rule	Task Success, pass^k
τ-bench [29] Paper Code	Workflow	customer service and database state	Target state, policy compliance, repeat stability	Rule	Task Success, pass^k
Enterprise operations					
AutomationBench [140] Paper Code Website	Workflow	SaaS automation tasks	Goal completion, constraint compliance	Rule	Task Success Rate
CI-Work [303] Paper Code	Answer	agent's final information-sharing action	Conveyance of essential information, leakage of sensitive information	LLM	Conveyance Rate, Leakage Rate, Violation Rate
EnterpriseBench [304] Paper Code Data Website	Workflow	enterprise question answering and interactive decisions	Information correctness, decision results, task completion	Rule + LLM	Task accuracy and interactive-decision scores
EnterpriseOps-Gym [139] Paper Code Data Website	Environment state	enterprise databases and service state	Target state, integrity constraints, side effects	Rule	Task Success Rate
FieldWorkArena [305] Paper Code Data	Answer	judgments over field images and video	Answer correctness, numerical accuracy	LLM + Rule	Accuracy Rate (Correctness and Numerical Scores)

Continued on next page

Table 10 (continued)

Benchmark	Level	Inspected work result	Main evaluation dimensions	Method	Model performance metrics
TheAgentCompany [31] Paper Code Website	Workflow	simulated enterprise collaboration tasks	Intermediate objectives, final completion	Rule + LLM	Task Completion Rate, Checkpoint Score
WorkArena [9] Paper Code Data Website	Environment state	ServiceNow application	Target state, task completion	Rule	Task Success Rate
WorkArena++ [133] Paper Code Data Website	Workflow	complex ServiceNow tasks	Multi-step task completion, state correctness	Rule	Task Success Rate
WorkBench [114] Paper Code Data	Environment state	sandbox database state	Target state, side effects	Rule	Accuracy, Side-Effect Rate
Software engineering					
SWE-bench [16] Paper Code Data Website	Artifact	code patch and repository	Defect-repair correctness, regression behavior	Rule	% Resolved / Issue Resolution Rate
Terminal-Bench 2.0 [122] Paper Code Website	Environment state	final container state	Task completion, execution correctness	Rule	Resolution Rate
Data science & data engineering					
DSBench [286] Paper Code Data Website	Answer and artifact	data-analysis answers and prediction files	Analytical correctness, predictive performance	Rule + LLM	Analysis Accuracy, Relative Performance Gap
Spider 2.0 (agentic) [154] Paper Code Website	Workflow	database engineering tasks	Query execution, correctness of workflow results	Rule	Success Rate
Scientific research					
DiscoveryBench [287] Paper Code Data	Answer	scientific hypotheses and analytical conclusions	Correctness of scientific discovery, hypothesis matching	LLM	Hypothesis Matching Score
ScienceAgentBench [10] Paper Code Data Website	Artifact	scientific programs and computed results	Scientific task completion, code executability	Rule	Success Rate, Valid Execution Rate
Computer-aided design & 3D graphics					
CADBench (BlenderLLM) [85] Paper Code Data	Artifact	Blender scripts and 3D models	Object attributes, spatial relations, instruction following, script validity	LLM + Rule	Attribute Score, Spatial Score, Instruction Score, Average Score, Syntax Error Rate
CADBench (MIT) [86] Paper Code Data Website	Artifact	CAD programs and generated geometry	Geometric fidelity, executability, program compactness	Rule	IoU, SIoU, Chamfer Distance, Valid Shape Rate, Token Count, Operation Count
BlenderBench (VIGA) [130] Paper Code Data Website	Artifact	Blender scenes and rendered results	Rendering consistency, spatial relations, visual quality, task completion	Rule + LLM	Photometric Loss, N-CLIP Score, VLM Score
BenchCAD [283] Paper Code Data Website	Artifact	CadQuery programs and edited solids	Geometric fidelity, essential-operation use, executability, edit fidelity	Rule	Execution Rate, Voxel IoU, Chamfer and Hausdorff Distance, Feature-F1, Essential-Op Recall, Normalized Edit Accuracy
General digital interaction					
AgentIF-OneDay [306] Paper Data Website	Artifact	general task deliverables	Instruction following, degree of requirement satisfaction	LLM	Average Normalized Rubric Score
AndroidWorld [27] Paper Code Data Website	Environment state	Android applications	Target state, task completion	Rule	Task Success Rate
AppWorld [37] Paper Code Website	Environment state	multi-application databases	Goal completion, preservation of non-target state	Rule	Task Goal Completion, Scenario Goal Completion

Continued on next page

Table 10 (continued)

Benchmark	Level	Inspected work result	Main evaluation dimensions	Method	Model performance metrics
MCP-Atlas [307] Paper Code Data	Answer	final answers to cross-server tool tasks	Claim correctness, coverage of necessary facts	LLM	Pass Rate, Mean Coverage
MMTB [308] Paper Code Data Website	Workflow	multi-type tool calling	Tool selection, arguments, and interaction correctness	Rule	Action-type / Layer Accuracy, Multi-turn Accuracy
MultiAgentBench [309] Paper Code Data	Workflow	multi-agent collaboration	Task milestones, task outcome, coordination quality	LLM + Rule	Task Score, Milestone KPI, Coordination Score
Odysseys [310] Paper Website	Workflow	long-horizon web and file tasks	Item-wise requirement satisfaction, complete completion	LLM	Rubric Average, Perfect Rate
OSWorld [7] Paper Code Data Website	Environment state	desktop and file state	Target state, task completion	Rule	Task Success Rate
OSWorld 2.0 [32] Paper Code Data Website	Workflow	cross-application desktop tasks with final files and state	Long-horizon completion, target-state correctness, cross-step constraints	Rule	Binary Task Completion, Partial Score
LOCA-bench (up to 256K) [150] Paper Code Website	Workflow	long-context tool tasks and environment state	Tool-operation correctness, context use, task completion	Rule	Task Accuracy (grouped by context length), Average Accuracy
ToolSandbox [127] Paper Code	Workflow	tool calls and state milestones	Task completion, state correctness, constraint compliance	Rule	Milestone-based Task Score
VisualWebArena [284] Paper Code Data Website	Environment state	visual website state and answers	Goal completion, correctness of visual interaction	Rule + LLM	Task Success Rate
WebArena [6] Paper Code Data Website	Environment state	website state and answers	Goal completion, answer correctness	Rule	Task Success Rate
WildClawBench [311] Paper Code Data Website	Workflow	CLI and multimodal tool tasks	Execution correctness, state and constraint compliance	Rule + LLM	Overall Score
Windows Agent Arena [312] Paper Code Website	Environment state	Windows applications and files	Target state, task completion	Rule	Task Success Rate
Cross-domain professional work					
$OneMillion-Bench [313] Paper Code Data	Answer	expert-level text responses	Multidimensional professional quality, requirement satisfaction	LLM	Expert Score, Pass Rate, Economic Value
APEX-Agents [157] Paper Code Data	Workflow	multi-application professional tasks	Task completion, satisfaction of itemized requirements	LLM	Pass@1, Pass@8, Pass^8, Mean Score
Agents' Last Exam [285] Paper Data	Workflow	occupational software tasks and deliverables	Complete task completion, professional acceptance, degree of partial completion	Rule	Pass Rate, Score
CentaurBench [314] Paper Code Data Website	Artifact	deliverables produced directly or by a model-guided worker model	Relative work quality, quality of guidance to a weaker model	LLM	Mean Rank, Automation Rank, Augmentation Rank
CL-Bench [315] Paper Code Data Website	Answer	context-learning tasks	Context understanding, rule compliance, task completion	LLM	Solving Rate

Continued on next page

Table 10 (continued)

Benchmark	Level	Inspected work result	Main evaluation dimensions	Method	Model performance metrics
GDP.pdf [91] Paper Code Data Website	Answer	professional PDF tasks	Document understanding, degree of requirement satisfaction	LLM	Strict All-Pass Rate, Graded Rubric Score
GDPval [279] Paper Data Website	Artifact	occupational professional deliverables	Professional quality, relative acceptability	Expert assessment	Win Rate, Win-or-Tie Rate
JOBBench [316] Data	Artifact	deliverables built from multi-file workspaces	Satisfaction of chained requirements, delivery completeness	LLM	Rubric Score
OmniaBench [317] Paper Code Data Website	Workflow	multi-type executable tasks	Goal completion, program and state correctness	Rule + LLM	Overall Pass@1, Route-level Scores
PRBench [80] Paper Code Data Website	Answer	professional task responses	Professional correctness, completeness, criterion satisfaction	LLM	Weighted Rubric Score, Min-Normalized Score
Remote Labor Index [288] Paper Code Data Website	Artifact	end-to-end remote work products	Professional acceptance, degree of automated completion	Expert assessment	Automation Rate
Workspace-Bench [36] Paper Code Data Website	Workflow	multi-file workspaces	Requirement satisfaction, task completion, correctness of dependency relations	Agent + Rule	Rubric Pass Rate, Task Completion Rate, Node F1, Edge F1
StartupBench [280] Paper Website	Artifact	professional workspaces and complete delivery tasks	Importance-tiered requirement satisfaction, delivery completeness	Agent	Importance-Weighted Mean Task Score, Success Rate (score $\geq$ 90)

5.3 Evaluation Metrics

Evaluation methods judge each task; performance metrics aggregate these judgments across tasks or samples. Benchmarks use three kinds: task outcome scores, relative comparison metrics, and weighted performance metrics. All measure result quality or degree of completion; resource metrics such as cost, tokens, and latency are discussed in Section 5.5.1.

Task outcome scores. These measure whether, or how far, a result meets the acceptance conditions. The most common is the share of successful tasks; in τ-bench, a task succeeds only if the final database matches the ground truth and the agent conveys the required information [29]. Answer-oriented benchmarks use accuracy and matching scores, such as accuracy, F1, or ROUGE-L in LawBench, depending on the task [292]. Rubric scores differ in strictness: GDP.pdf ranks models by a strict pass rate that requires every criterion, alongside a mean rubric score [91]. Some outcomes are continuous, such as year-end total assets after a simulated year of store operation in E-Commerce Bench [33]. Over repeated attempts, pass@k requires at least one of k attempts to succeed, and pass^k, introduced by τ-bench, requires all of them [29]; APEX-Agents reports both Pass@8 and Pass^8 [157]. Because they differ in unit and strictness, these scores are not directly comparable across benchmarks.

Relative comparison metrics. Without a unique reference answer, quality can be measured by preferences, win rates, or rankings among candidates. GDPval has occupational experts blindly compare model and professional deliverables, reporting how often the model's is rated better or as good [279]. CentaurBench turns pairwise judgments from a panel of LLM judges into rankings, separately for automation, where the model produces the deliverable, and augmentation, where it guides a weaker worker model [314]. Such results depend on what is compared and how, and cannot be read apart from the reference set.

Weighted performance metrics. These aggregate results with explicit weights on criteria or on tasks. PRBench weights each expert criterion with an integer from -10 to 10, so that undesirable content lowers the score [80]; OmegaUse-OfficeVal weights tasks by recorded human labor time or a task-price proxy [301].

Criterion weights reflect how much each requirement matters and task weights how much each task is worth; both remain performance scores, not measures of economic return.

Table 11 Categories of model performance metrics in work-agent benchmarks, with their units, aggregation, and required qualifications

Metric category	Specific metrics, units, and aggregation	What it measures	Required qualification
Task Outcome Score	*Success rates:* Task Success Rate, pass@k, pass^k, Pass Rate, and similar measures counting the proportion of successes or the success probability under repeated execution. *Accuracy and matching:* Accuracy, Exact Match, Execution Accuracy, F1, ANLS, IoU, Chamfer Distance, and similar measures computed against reference results. *Quality and partial credit:* Rubric Score, Overall Score, Partial Score, and similar measures aggregating item-level requirements or partial completion. *Continuous outcomes:* final operating or work output counted according to the task definition	How far the model satisfies the task objective, the acceptance requirements, and the delivery quality standard	Acceptance conditions, matching units, and partial-credit rules must be stated. Representative benchmarks include SWE-bench [16] Paper Code Data Website FinQA [21] Paper Code Data DocVQA [22] Paper Data Website PresentBench [17] Paper Code Data Website Workspace-Bench [36] Paper Code Data Website OSWorld 2.0 [32] Paper Code Data Website Agents' Last Exam [285] Paper Data
Relative Comparison Metrics	Pairwise Preference, Win Rate, Tie Rate, Mean Rank, and similar measures aggregated through pairwise comparison, preference statistics, or model ranking	Model quality relative to a designated reference work product or to other models	Comparison objects, judging protocol, and ranking rules must be stated. Representative benchmarks include GDPval [279] Paper Data Website CentaurBench [314] Paper Code Data Website
Weighted Performance Metrics	Importance-Weighted Score, Time-Weighted Score, Price-Weighted Score, and similar measures that aggregate outcome scores using the importance of within-task requirements or explicit weights across tasks	Composite model performance after accounting for requirement importance or differences in task value	The source of the weights, the level at which they act, and the normalization procedure must be stated. Representative benchmarks include BankerToolBench [281] Paper Code Data StartupBench [280] Paper Website OmegaUse-OfficeVal [301] Data

Table 11 summarizes the computation and the interpretive scope of the three classes of metric. Overall, a metric should align with the object and the quality requirements actually accepted, and should state its success threshold, partial credit, comparison baseline, and the source of its weights, so that scores from different benchmarks are not treated as fully equivalent scales of capability.

5.4 Open-Source Harnesses and Execution Environments

Evaluating a work agent depends not only on the model and the task but also on the harness configuration. An agent harness handles context management, tool invocation, and execution control, while an evaluation harness handles task initialization, run recording, and result verification. OpenHands, SWE-agent, Codex, and Claude Code supply agent execution mechanisms [3, 11, 12, 152]; BrowserGym, the SWE-bench harness, Harbor, and Inspect AI support standardized task execution and evaluation [16, 320–322].

The infrastructure of a benchmark is closely tied to its work objects. Software-engineering tasks depend on code repositories and test environments, web and desktop tasks on resettable interactive environments,

and enterprise tasks check business state through APIs and databases, while office and design tasks require verifying native files, computed results, and the actual deliverables. Table 12 summarizes the execution environments and verification mechanisms of representative benchmarks.

Table 12 Harnesses and execution environments of representative work-agent benchmarks.

Task type	Representative benchmarks	Agent harness / execution	Evaluation environment and verification
Software engineering	SWE-bench [16] Paper Code Data Website SWE-Lancer [153]	SWE-agent [3] Paper Code Website OpenHands [152] Paper Code Codex [11] Docs Repo	Docker, code repositories, and test programs
Terminal tasks	Terminal-Bench 2.0 [122] Paper Code Website	Claude Code [12] Docs Repo Codex [11] Docs Repo Pi [323] Code	Harbor [321] Code Website isolated containers and verification scripts
Web interaction	WebArena [6] Paper Code Data Website WorkArena [9] Paper Code Data Website VisualWebArena [284] Paper Code Data Website	Browser agents	BrowserGym [320] Paper Code Website AgentLab, and website state checks
Desktop and mobile	OSWorld [7] Paper Code Data Website OSWorld 2.0 [32] Paper Code Data Website AndroidWorld [27] Paper Code Data Website Windows Agent Arena [312] Paper Code Website	GUI and computer-use agents	Virtual machines, emulators, and application-state verification
API and business operations	AppWorld [37] Paper Code Website τ-bench [29] Paper Code τ^2-bench [30] Paper Code Data	Tool-calling agents	Application APIs, user simulators, and database verification
Enterprise operations	CRMArena [148] Paper Code Data AutomationBench [140] Paper Code Website EnterpriseOps-Gym [139] Paper Code Data Website	API and MCP agents	Enterprise application sandboxes, business state, and constraint checks
Tools and context	MCP-Atlas [307] Paper Code Data ToolSandbox [127] Paper Code LOCA-bench [150] Paper Code Website	MCP and tool agents	Stateful tools, context configuration, and task verification

Continued on next page

Table 12 (continued)

Task type	Representative benchmarks	Agent harness / execution	Evaluation environment and verification
Office artifacts	SpreadsheetBench [56] Paper Code Data Website PPTC [112] Paper Code Data PresentBench [17] Paper Code Data Website	File, script, or GUI agents	Workbook computation, file parsing, and rendering
Integrated office work	OfficeBench [76] Paper Code Data OmegaUse-OfficeVal [301] Data	GUI, API, or hybrid execution	Office workspaces and native-artifact inspection
3D design	CADBench (BlenderLLM) [85] Paper Code Data CADBench (MIT) [86] Paper Code Data Website BlenderBench (VIGA) [130] Paper Code Data Website BenchCAD [283] Paper Code Data Website	Blender and CAD agents	Program execution, geometric comparison, and rendering assessment
Enterprise collaboration	TheAgentCompany [31] Paper Code Website Workspace-Bench [36] Paper Code Data Website	Multi-tool workspace agents	Enterprise services, file dependencies, and task checkpoints
Professional delivery	StartupBench [280] Paper Website BankerToolBench [281] Paper Code Data APEX-Agents [157] Paper Code Data	Professional workspace agents	File inspection, professional rubrics, and agentic evaluation
Occupational work	GDPval [279] Paper Data Website Agents' Last Exam [285] Paper Data Remote Labor Index [288] Paper Code Data Website	Configurable professional task agents	Professional work-product submission and task acceptance
General evaluation	AgentBench [275] HAL [324] Harness-Bench [325] Paper	Various model–harness configurations	Unified task environments, execution traces, and result scoring

Recent work has begun to examine the effect of the harness on evaluation results directly. HAL supports cross-benchmark comparison of different models and agent scaffolds [324]; Harness-Bench further analyzes how model–harness combinations affect task performance and execution behavior [325]. These studies indicate that benchmark scores should be interpreted together with the execution framework, the tool configuration, and the resource budget, rather than attributed entirely to the underlying model.

5.5 Performance, Cost, and Economic Value

Task success rates and delivery quality measure what an agent can do, but they do not capture its execution cost or its actual economic value. Recent work extends the evaluation of work agents along three levels: the performance–cost trade-off, the value of the professional work itself, and the utility of deployment.

5.5.1 Trading Performance against Execution Cost

Task performance and execution cost do not always move together. HAL compares task scores and invocation cost jointly, so that a Pareto frontier can identify competitive system configurations under different cost constraints [324]. The analysis applies both to model comparison and to complete agent systems.

HarnessTax further finds that when the same model is run under different harnesses, invocation cost can differ substantially even when task success rates are close [326]. Harness-Bench analyzes how the execution configuration affects task performance [325], and automatic adaptation of instructions, tools, and the execution flow improves the cost efficiency of some smaller models [327]. These results show that the performance–cost trade-off depends not only on the choice of model but also on the design of the harness.

5.5.2 The Value of Professional Work

Execution cost reflects the resources needed to complete a task, whereas economic value also depends on the task itself. SWE-Lancer connects software-engineering tasks to real market pay [153]; GDPval has occupational experts compare professional deliverables [279]; and Remote Labor Index evaluates end-to-end automation using actual paid remote projects [288].

OmegaUse-OfficeVal further combines office-artifact quality, human completion time, and a task price proxy [301], while a task-completion time horizon uses human task duration to characterize the scale of work an agent can complete [63]. The task distribution of existing agent benchmarks also diverges from the real labor market [328]. Evaluating work value therefore has to account for task representativeness and professional acceptance, and annotation pay or human working hours should not be equated directly with actual economic return.

5.5.3 Reliable Deployment and Agentic ROI

High task performance and low invocation cost do not necessarily produce a positive return on deployment. The position that the real barrier to agent usability is agentic ROI argues that an agent's practical utility depends on delivery quality, the human time saved, and the cost of use, including the effort users spend describing, checking, and revising the work [329].

READY further brings human oversight into reliability and cost evaluation, finding that agents with comparable autonomous accuracy may require different amounts of human assessment to meet the same deployment requirement [330]. Agentic ROI therefore supplies a conceptual view of user utility, and READY an empirical complement in the form of joint human–machine cost under a reliability constraint. Actual deployment return must also account for failure repair, system integration, and long-term maintenance.

Overall, the economic evaluation of work agents needs to distinguish the resource cost of executing a task, the potential economic value of the work produced, and the actual deployment return once human oversight is taken into account. The performance–cost Pareto frontier, the harness tax, and agentic ROI complement conventional task-performance evaluation from the perspectives of system efficiency, execution configuration, and user utility respectively. Section 6 develops how these systems can extend from completing assigned tasks to carrying responsibility as workspaces and requirements change.

6 Frontiers: Toward Next-Generation Work Agents

A manager asks an agent to prepare a report for a recurring business meeting. Completing the first report requires finding the right records, resolving discrepancies, and explaining the results. Taking responsibility for later reports means following revisions, obtaining missing information, and acting before each deadline. The manager may also ask for a recommendation that reflects how a trusted analyst would weigh new evidence. Other teams must confirm inputs, resolve shared assumptions, and approve changes before the report can be used.

These requests extend delegation in three directions. Sustaining the manager's goals motivates *personal work agents*; representing the analyst's judgment motivates *human digital twins*; coordinating contributions

toward the report motivates *organizational work systems*. In each case, the agent must keep work moving as information and responsibilities change. Experience from one reporting cycle can improve the next, while changes to the method may affect results that others have already used.

We develop this agenda through four questions. Section 6.1 examines what people can delegate in these applications. Section 6.2 identifies the advances in environment, persistence, and verification they require. Section 6.3 asks how execution experience can improve the agent while existing work continues. Section 6.4 asks under which work conditions this delegation is beneficial. Figure 9 maps the connections, and Table 13 summarizes the research questions.

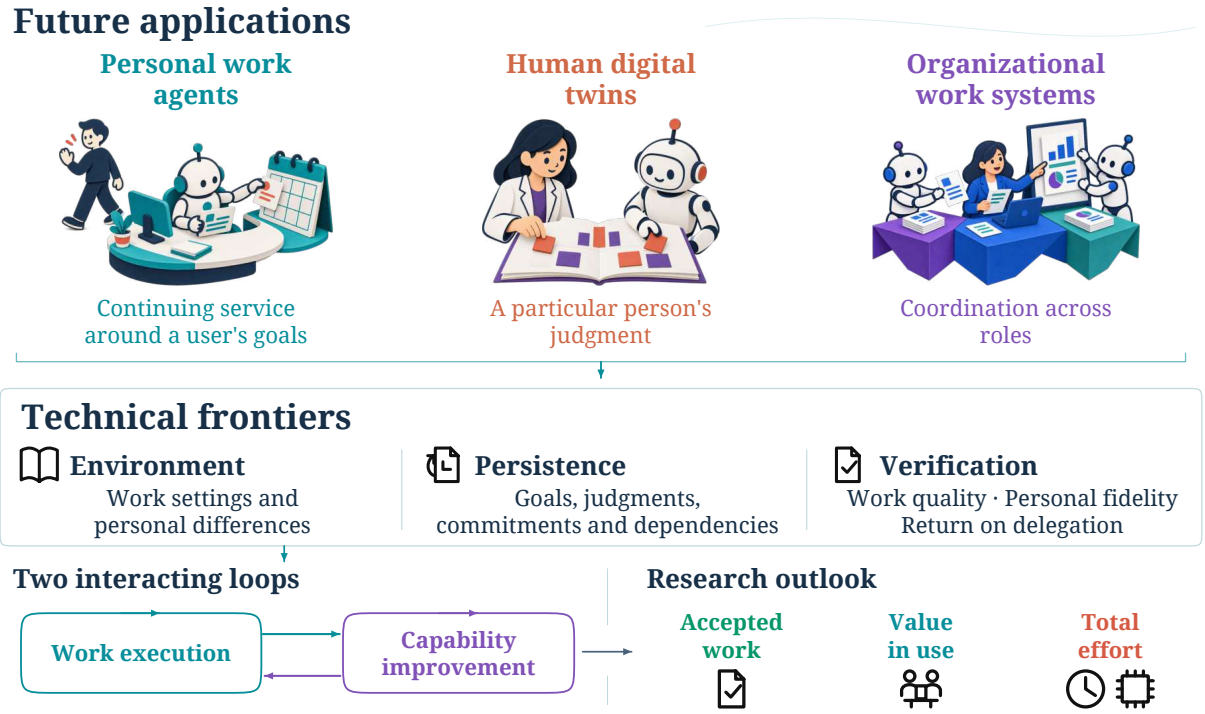


Figure 9 Research directions for next-generation work agents. Continuing service around a user's goals, representation of a particular person's judgment, and coordination across organizational roles motivate advances in environment, persistence, and verification. Interacting execution and improvement loops support these responsibilities, while accepted work, value in use, and total effort guide the assessment of beneficial delegation.

6.1 Future Applications: From Assistance to Delegated Work

The three applications concern different parts of the same reporting process: the manager's continuing goals, the analyst's professional judgment, and the shared delivery. Each places different demands on what the agent must remember, decide, and verify.

6.1.1 Personal Work Agents

A personal work agent helps a user keep an unfinished goal moving across requests. For the manager, this means retaining the purpose of the meeting, gathering materials as they become available, and following up on missing inputs before the deadline. When new information arrives, the agent must decide whether to advance an agreed task, ask the user for a decision, or wait [331]. Proactive-agent research studies anticipation and intervention timing [332, 333]; VibeLifeBench and PAST-Bench examine continuing assistance and gains from retained experience [34, 334].

Such service begins with helping the user specify the work. A manager may know which decision the report should support without knowing which records or analyses it requires. The agent can clarify the decision,

Table 13 Research agenda: what people can delegate, which technical advances support it, how execution and learning interact, and where delegation is beneficial.

Research direction	Open question	Evidence of progress
6.1 Future applications: what can people delegate?		
Personal work agents (§6.1.1)	How can an agent advance unfinished goals between requests?	Timely progress on agreed goals with less repeated specification and intervention.
Human digital twins (§6.1.2)	How can an agent reflect a particular person's judgment in new situations?	Agreement with that person's decisions under comparable information, assessed separately from task quality.
Organizational work (§6.1.3)	How can roles, decision rights, and dependencies be coordinated toward a joint delivery?	Completed handoffs and end-to-end deliveries with less rework caused by coordination failures.
6.2 Technical frontiers: what makes that delegation possible?		
Environment (§6.2.1)	Which work settings and personal differences must practice represent?	Transfer across work settings and correspondence to individual judgments in held-out situations.
Persistence (§6.2.2)	How should goals, personal judgments, commitments, and feedback guide continuing work?	Timely action, retained obligations, and updates consistent with the current basis of delegation.
Verification (§6.2.3)	What evidence establishes work quality, personal fidelity, and return on delegation separately?	Separate measures of deliverable quality, person-specific decision agreement, and benefit relative to cost.
6.3 Interacting loops: how can work continue while capability improves?		
Work execution (§6.3.1)	How can responsibility survive interruptions and changes of executor?	Resumed work that preserves confirmed effects, pending dependencies, and agreements.
Capability improvement (§6.3.2)	Which experience should change memory, skills, harness, or policy?	Gains on independent tasks and work settings from targeted updates.
Coupling the loops (§6.3.3)	Which update scope and timing yield gains after rework and coordination costs?	More accepted and used work per total effort, including repair of affected ongoing work.
6.4 Research outlook: where does the system expand useful work?		
Value under openness and persistence (§6.4)	Under which discovery and continuity demands does delegation pay off?	A wider range of work completed and used with sustainable demands on people and resources.

identify missing inputs, and turn the agreed goal into tasks for specialist executors. Magentic-One's progress tracking and AutoGen's configurable agent conversations provide mechanisms for coordinating those tasks [53, 335]. Retaining the open goals, completed work, and events that warrant further action lets the agent resume across reporting cycles with less repeated explanation from the user.

Claude Duet, introduced as an experimental release in October 2026, illustrates this direction in a personal service: it checks connected applications, prepares briefings, and continues delegated tasks while the user is away, notifying the user when work is complete or needs a decision [336]. This makes the interval between requests part of the work: the agent must choose what to advance and when the user's attention is needed.

6.1.2 Human Digital Twins for Professional Work

The manager may need a recommendation that reflects how a particular analyst weighs new evidence. A human digital twin aims to represent that person's judgment across new situations. Two analysts can draw different conclusions from the same records because they use different assumptions or accept different tradeoffs. Professional competence concerns the quality of the work; personal fidelity concerns whose judgment the system represents.

Existing studies address parts of this correspondence. LLM-based twin architectures support context-aware knowledge exchange, agents grounded in interviews and questionnaires model individual behavior, and TwinVoice evaluates fidelity to a person's communication and persona [337–339]. GRACE extends representation into clinical communication through clinician-informed dialogue scripts and retrieved medical knowledge [340]. Extending this correspondence to professional action requires learning how a particular person weighs evidence, handles uncertainty, and chooses among acceptable courses of action.

Such a twin should be evaluated on held-out cases in which the represented person makes or assesses a decision from comparable information. A personalized professional agent that knows the person's preferences provides a comparison for testing whether the twin also captures characteristic judgments.

The twin's recommendations also enter work done by other participants. Authenticated delegation provides mechanisms for scoped authority [341], while handoff studies show how constraints can be lost when context is transferred [342]. Passing the analysis to the manager's agent therefore requires retaining both its assumptions and the authority under which it was produced.

6.1.3 Organizational Work Systems

Organizational work joins contributions into a delivery that other people can use. The analyst's forecast must reflect assumptions agreed with other teams, reach the manager's report, and arrive before the meeting. MetaGPT provides an example of coordination through structured roles, procedures, and intermediate artifacts in software development [54]. TheAgentCompany places agents in shared applications with simulated colleagues [31]. Failure analyses and controlled scaling studies show how communication structure, verification, and task dependencies affect multi-agent execution [343, 344].

The central challenge is coordinating role responsibilities, decision rights, and interdependent contributions toward a joint delivery. A revised sales estimate may require an analyst to update the forecast, another team to confirm an assumption, and a manager to authorize release. The system must determine who is responsible for each unresolved dependency, which decisions can proceed in parallel, and how revisions affect work already passed to another participant. Human–agent teamwork research and WORKBank's occupational preferences inform how people choose to share those responsibilities [345, 346]. Progress requires dependable handoffs and conflict resolution that advance the overall delivery while limiting avoidable rework.

6.2 Technical Frontiers for Sustained Delegation

The three applications place different demands on sustained delegation: personal agents must advance goals between requests, digital twins must remain faithful to a particular person's judgment, and organizational systems must coordinate work across roles. **Environment** research asks how to represent situations that exercise these abilities. **Persistence** asks how goals, personal judgments, commitments, and dependencies remain available and applicable over time. **Verification** asks which evidence establishes progress on the delegated responsibility, correspondence to the represented person, and the quality and value of the resulting work.

6.2.1 Environment: Learning across Realistic Work Worlds

Environment research must decide what work to represent, which features make the experience realistic, how to run it, and where additional practice is most useful.

Occupational coverage and transfer. A teacher, analyst, and procurement specialist may all produce presentations while relying on different knowledge and standards. O*NET organizes occupational requirements; GDPval and the Remote Labor Index connect professional tasks to deliverables; WORKBank adds preferences about delegation [279, 288, 346, 347]. Together with analyses of occupational exposure [348], these resources can guide coverage across domains, occupations, and workflows. Testing held-out combinations would reveal what transfers: reconciling inconsistent records may reuse a common procedure, while deciding why a discrepancy matters may require knowledge of the new profession. Those gaps then guide the construction of further experience.

For digital twins, environments should also vary the represented person within the same occupation and task. The same reporting problem may lead two analysts to different choices because they use different assumptions or commitments. Holding the available evidence and authorization constant where feasible would isolate differences in judgment; varying authorization separately would test whether actions remain within the delegated role.

Decision-relevant realism. Within each kind of work, an environment should preserve the relationships that affect decisions. Workspace-Bench makes file dependencies explicit; business and workspace synthesis methods construct connected entities and evidence relations [36, 143, 163]. Generating workspaces through successive episodes of activity could add the histories behind these relationships. Earlier reports would establish conventions, revisions would create competing versions, and parallel workflows would leave plausible but irrelevant materials. Shared-state settings such as τ^2-bench and Collaborative Gym add participants who change the environment [30, 141], while simulation-to-reality studies examine transfer to real interactions [349]. Varying source ambiguity, dependency depth, and incomplete updates would then exercise the decisions needed to clarify and repair work.

Execution infrastructure. These environments need to support repeated practice and continuation from interrupted work [350]. BrowserGym provides common web observations and actions, while Agent Lightning connects harness execution to policy training [260, 320, 351]. DSec separates stateful rollouts from preemptible GPU training [352], and AIOpsLab composes services, workloads, faults, and telemetry for operational tasks [123]. Building on these mechanisms, a work environment could package the application versions, source materials, business state, and events needed to reproduce an episode, together with checks for its outcome. Saved states and records of completed effects would let training and evaluation resume the same unfinished work.

Practice allocation. Once environments are available, the remaining question is which experiences deserve the execution budget. Repository environments provide executable practice [353, 354]; Skill2Env targets capability gaps, and AgentGym-RL progressively expands interaction horizons [355, 356]. For professional work, an expensive episode involving clarification and coordination may teach more than many easily scored actions. Comparing learning gains and transfer across these choices would turn the construction methods in Section 4.1 into a curriculum guided by the work agents still struggle to perform.

6.2.2 Persistence: Memory, Initiative, and Adaptation

Realistic environments expose changes that unfold between interactions. Handling them requires maintaining an open responsibility: work that remains due even when no agent is currently running. The reporting deadline, pending source confirmation, and authority to release the result must survive the pause. This creates three connected needs: remembering what still applies, deciding when to act, and learning from repeated work.

Memory grounded in current work. The agent needs to retrieve relevant decisions and procedures and determine whether they still apply. Personalized workflow prediction studies which procedures fit a user's context [357]. Memory management and temporal representations support retention and revision [234, 358–361]; A-MEM links and revises notes, while Memory-R1 learns memory operations and selection from downstream rewards [362, 363]. StreamMemBench exposes gaps between storing evidence, using it, and reusing feedback [364]. In the report, a convention may expire when the audience changes while the reconciliation procedure remains useful.

Memory should also distinguish a user's goals and preferences from a represented person's judgments, existing commitments, and authorization. Linking each item to the relevant person, role, source, and period of applicability would help the agent distinguish a preference about presentation from a commitment governing the analysis or the authority needed to release it.

Initiative guided by responsibilities. Retained obligations give the agent a reason to act between requests. Proactive Agent learns from activity traces and judgments of proposed assistance; Proactivity-Gym studies intervention timing and trust [332, 333]. VibeLifeBench examines needs that become visible through inspecting external state [34], and temporal and asynchronous environments provide settings for choosing when to observe, ask, act, or wait [128, 365, 366]. A revised sales record may require several reports to be updated, while an approaching deadline may warrant asking for a missing input. The agent also needs to know which decisions it has been delegated. Collecting a missing input, recommending a conclusion, approving its release, and accepting a new commitment involve different decision rights. It can advance tasks covered by the current delegation and seek a decision when a new choice or commitment lies outside that scope.

Learning from repeated work. Repeated reporting cycles supply experience for improving source selection, analysis, and explanations. Reflection, experience abstraction, skill libraries, and workflow memory retain reusable lessons [60, 367–369]; SkillRL connects skill evolution to policy learning, and PAST-Bench measures gains from retained experience [265, 334].

Feedback can revise the current report, the procedure used for later reports, or the representation of the analyst. A corrected source value repairs the current result; repeated reconciliation errors motivate a reusable check; feedback from the analyst reveals how that person weighs evidence. Retaining which part of the work each correction concerns gives the agent a basis for later adaptation. Work quality and personal fidelity require separate assessment. Section 6.3.3 examines how procedural updates affect work already in progress.

6.2.3 Verification: Work Quality, Personal Fidelity, and Return on Delegation

Continuing service raises three evaluation questions: does the delivery meet its requirements, does a digital twin reflect the represented person's judgment, and does delegation yield a useful return? These questions require separate evidence about work quality, personal fidelity, and benefit relative to human and computational effort. Verification must follow each as the work and its use evolve.

Task and deliverable acceptance. A report may contain correct numbers yet fail to inform the decision it is meant to support. Professional acceptance therefore combines direct checks with judgments about purpose and audience. PresentBench uses slide-specific checklists, PRBench elicits expert criteria, and GDPval uses expert comparisons of occupational deliverables [17, 80, 279]. Agent-as-a-Judge provides mechanisms for inspecting files and execution evidence against requirements [15]. The research task is to connect these checks to the brief: determine which criteria transfer across occupations, which require expert input, and which requirements need clarification. Variation across web benchmarks and evidence of judge bias make consistency across these settings an important evaluation target [370–373].

Personal decision fidelity. A twin's fidelity is assessed against decisions elicited from the represented person on held-out cases under comparable information. For the same report, two analysts may recommend different forecasts because they weigh uncertainty differently. Both recommendations may meet professional standards, yet only one may reflect the represented person's judgment. Conversely, a faithful prediction can reproduce that person's mistake. Evaluation should therefore report decision agreement separately from deliverable quality, record uncertainty or changes in the person's position, and check execution against the agreed authority. This distinguishes whose judgment the twin represents from the quality of the resulting work.

Return on delegation. Measuring the effort of delegation requires tracing the whole delivery process. Agentic ROI considers quality relative to a human baseline, time saved after human interaction, and monetary cost [329]. The relevant effort includes specification, supervision, correction, and integration into later work. Computational cost also depends on the harness: HarnessTax reports similar success rates at substantially different API costs for the same model across harnesses on SWE-bench Lite and Terminal-Bench 2.0 [326]. Recording these costs by stage would reveal where changes to the model or harness could reduce the effort of obtaining an accepted result.

The benefit side requires following what happens to that result. For the report, useful evidence includes whether it supported discussion at the meeting and reduced the work needed to reach a decision. Such benefits depend on the people and process using the artifact. A natural experiment on Change.org found changes in petition language without improved platform-level outcomes after AI writing assistance became available [374]. Comparisons with alternative ways of completing the same work are therefore needed to estimate the benefit attributable to delegation. These observations complement acceptance checks and supply the evidence used in Section 6.4.

Evidence over time. New source data may invalidate an accepted analysis, while later use may reveal a missing requirement. StateGuard represents constraints, versioned variables, and conclusions in a validity graph [375]. Linking acceptance checks to the source, artifact, and requirement versions they concern would help identify which results need rechecking. Linking later feedback to the delivery and its use would show

which procedures need improvement. These two paths lead directly to the interaction between ongoing execution and learning.

6.3 Two Interacting Loops: Delivering Work and Improving Capability

Execution and learning share the same workspace. A new analysis procedure may improve future reports while changing an estimate that another team has already used. Introducing it requires tracing how its effects propagate through intermediate results, downstream decisions, and ongoing commitments. The *work execution loop* advances current responsibilities and records what happened. The *capability improvement loop* learns from that experience and changes how later work is done. Their connection supplies the evidence for choosing the scope and timing of updates while work remains active. Figure 10 shows how updates return to ongoing execution.

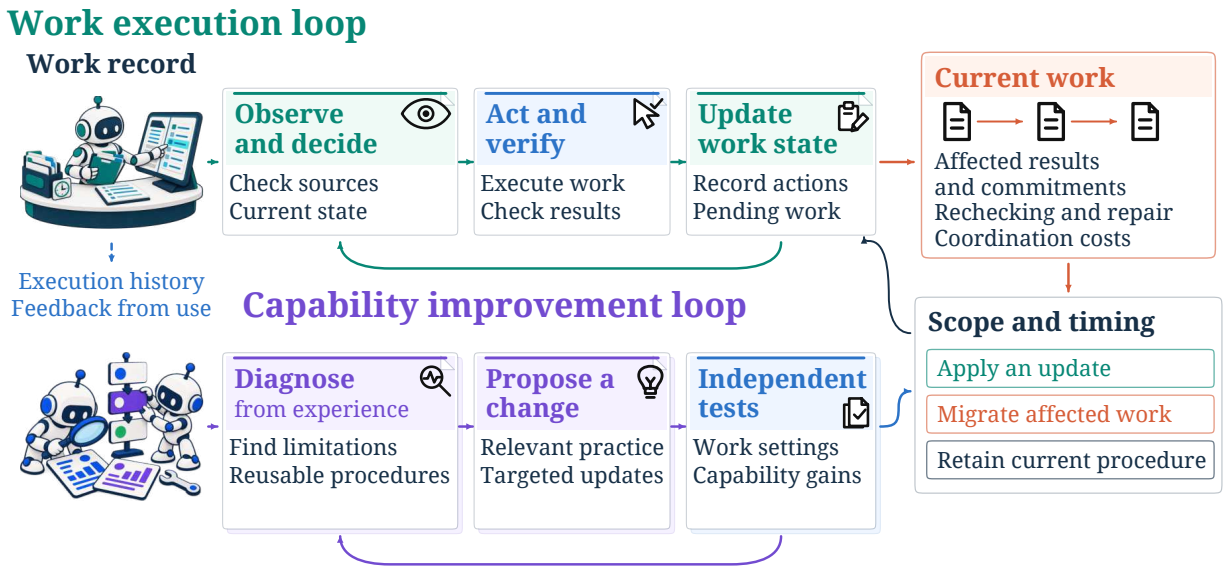


Figure 10 Work execution and capability improvement during continuing service. A persistent work record retains agreed requirements, source versions, unfinished work, and completed actions. Execution history and feedback from downstream use inform targeted updates. Independent tests assess capability gains, while checks on current work identify affected results and commitments. This evidence guides whether to apply an update, migrate affected work, or retain the current procedure. The benefit of an update depends on its gains relative to the resulting rework and coordination costs.

6.3.1 The Work Execution Loop

The execution loop needs a record that lasts across agent runs. A *persistent work record* can hold the report's agreed requirements, source versions, pending approvals, and completed actions; case handling provides a precedent for such enduring records [376]. When a revised sales estimate arrives, the agent identifies the affected analysis, updates it, checks the result, and records what remains due. Approval closes that delivery while leaving the next reporting cycle active.

Interruptions make the relationship between this record and the workspace especially important. An external action may complete before the agent retains its confirmation [377, 378]. On resuming, the agent needs to inspect the actual effect before deciding to retry or repair, using the execution-state distinctions in Section 2.3.4. Provenance and compensating operations provide mechanisms for reconstructing and correcting these effects [379–382]. The resulting history supports both the next participant taking over the work and the improvement loop diagnosing how it was performed.

6.3.2 The Capability Improvement Loop

The improvement loop uses execution history and feedback from downstream use to identify reusable procedures and changes that could improve later work. If reports repeatedly miss a deadline because the agent discovers inconsistent inputs too late, an earlier source check may be more useful than a stronger drafting model. Other failures may call for professional training, a repaired tool interface, or better timing. The methods in Section 4 provide the update mechanisms; reflection, workflow memory, skill evolution, and harness-aware training offer ways to retain and learn from the experience [60, 260, 265, 369].

The loop then constructs relevant practice, proposes a change, and tests it in independent work settings. For the earlier source check, evaluation would examine whether it catches discrepancies and reduces later rework across different reports. Independent output checks against shared task requirements make gains comparable across changes to both the agent and its grader [383–385]. Environment construction supplies practice that exercises the missing ability; verification tests whether the proposed update improves the resulting work. The remaining question is how to apply a successful update to work already in progress.

6.3.3 Coupling Learning to Open Responsibilities

Continual learning studies how updates preserve performance on earlier tasks [386]. During service, other outputs or participants may already depend on results produced by the method being updated. The research problem is to trace the update's effects through these dependencies. In the reporting example, records should link source versions and analytical assumptions to calculations, conclusions, approvals, and downstream decisions. A new forecasting method may change a budget estimate already used by another team; an additional source check may leave that estimate intact.

For a digital twin, these dependencies also include the represented person's assumptions and subsequent choices. Evaluation therefore follows both task quality and personal fidelity, using the separate assessments in Section 6.2.3.

These links define the scope of an update: which results to recompute, which checks to repeat, and which handoffs to reopen. Studies should compare changes to a single check, a reusable procedure, and the full model–harness configuration, measuring how far their effects propagate. This connects capability gains to the repair and coordination needed to put them into use. Audits of self-evolving financial agents further motivate examining how learning changes behavior and its effects [387].

This makes *net benefit during service* a concrete research target. Studies should vary update scope and timing while tasks remain unfinished, measuring gains in accepted and used work alongside training, inference, rechecking, repair, and human coordination costs over the same period. Comparisons with retaining the current procedure or updating only between deliveries would show when those gains repay the cost of change. The goal is to identify the work conditions and update granularity under which continual improvement pays off, including settings where a stable task template remains the more productive choice.

6.4 Research Outlook: Useful Work under Openness and Persistence

The systems above should expand the range of work that people can usefully delegate. Openness and persistence, introduced in Section 2, provide two independent dimensions along which to test that expansion. Preparing a single report in an unfamiliar organization stresses discovery of sources, relationships, and authority. Preparing recurring reports in a fixed environment stresses continuity across revisions and interruptions. Combining the two asks whether an agent can learn its way around changing workspaces while keeping existing work moving. Figure 11 contrasts these settings and the evidence needed to assess them.

These conditions create different opportunities and costs. Persistent service can reuse context and procedures across deliveries, but it also requires continued observation, maintenance, and recovery. Open workspaces offer access to tasks whose inputs were never fully specified, while adding discovery and clarification effort. Studies can vary these demands separately and together, using the evidence in Section 6.2.3 and the measures in Section 5.3 over the full period of delegation. The resulting comparisons would show where agents reduce the effort of existing work and where they make previously inaccessible work feasible.

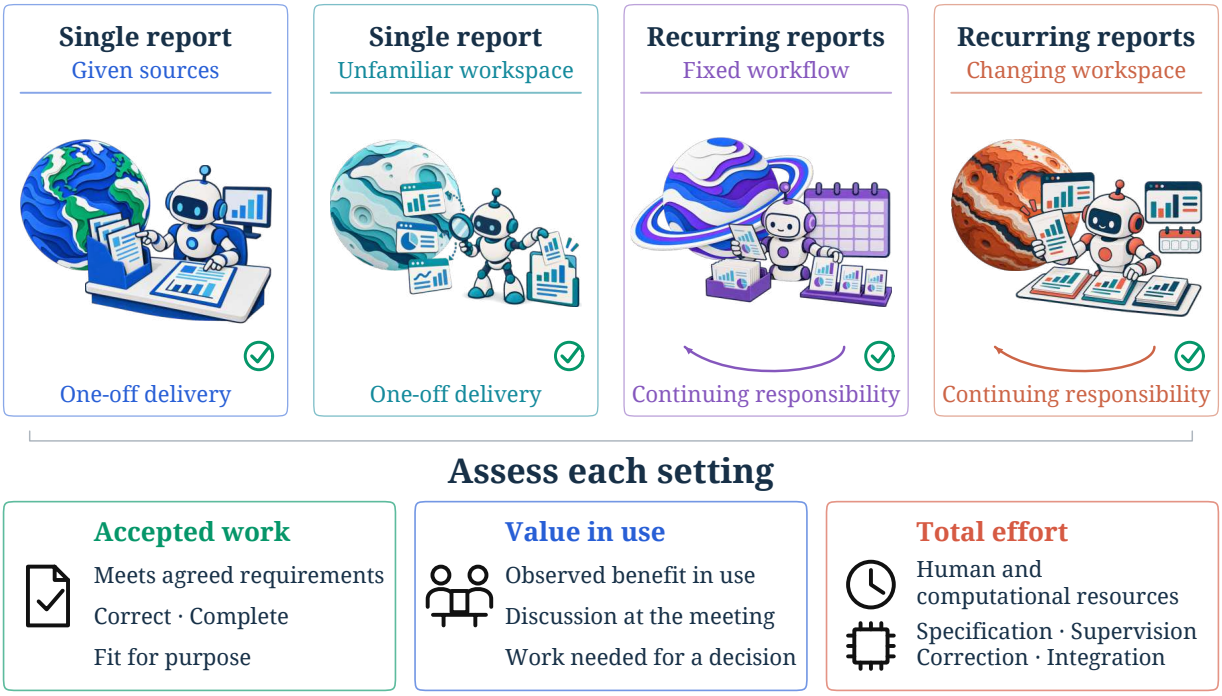


Figure 11 Assessing beneficial delegation across work conditions. The four scenarios illustrate variation in work openness and persistence: single reports from given sources or in an unfamiliar workspace, and recurring reports within a fixed workflow or a changing workspace. Across all four settings, accepted work, value in use, and total human and computational effort provide complementary evidence for assessing delegation.

The division of work between people and agents is part of this comparison. In a product-innovation experiment, individuals with AI assistance matched human teams without AI and produced proposals spanning technical and commercial perspectives [388]. In a study of experienced developers, the tested AI tools increased completion time [389]. Alongside experiments on professional writing [390], these findings motivate testing delegation across tasks and working arrangements. Changes to processes and skills can determine whether local gains improve overall productivity [391, 392]. For the reporting process, faster analysis matters when assessment and integration can turn it into a timely decision.

This returns the agenda to the two loops. As execution improves, assessment, integration, or human decisions may become the limiting stage [393]. The execution loop must organize work around that stage, and the improvement loop must address the capability or coordination that limits the next delivery. Progress is a wider range of professional work completed, accepted, and used with sustainable demands on people and resources.

7 Conclusion

Reliable digital work depends on keeping task requirements, actions, and outcomes connected as work unfolds. The system view developed here makes these connections visible across tasks, domains, and interfaces. As work expands from returning an answer or making a local change to coordinating workflows and related workspace objects, agents must track more dependencies and use the results of one step to guide the next. Openness makes discovery part of this process; persistence carries responsibility across stages and events. The five core capabilities provide a way to understand where the process succeeds and where it breaks down.

Work execution supplies observations, procedures, corrections, and feedback that can improve subsequent work. Training can develop reusable knowledge and action patterns, while harness changes can improve access to evidence, execution control, and retained state. Model–harness optimization coordinates these updates as

the model and the work change. Evaluation checks whether the resulting system produces answers, artifacts, state changes, and workflow results that meet their requirements. Distinguishing the inspected result, the assessed properties, the judging method, and the reported metric helps identify which part of the work an improvement actually advances.

Sustained delegation brings execution and learning together. An agent must respond to new information and improve its methods while completing work under existing agreements. Gains on new tasks must therefore be considered alongside the repair of dependent work and the effort of introducing updates. Personal work agents, human digital twins, and organizational work systems bring different tests of this capability: progress on continuing goals, fidelity to a particular person's judgment, and coordination toward a joint delivery. Advances in realistic environments, persistent coordination, and verification can widen the conditions under which these systems are useful. Their practical value depends on the work people accept and use, the benefits it brings, and the human and computational effort required to obtain it.

References

[1] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the Opportunities and Risks of Foundation Models. arXiv preprint arXiv:2108.07258, 2021. URL https://arxiv.org/abs/2108.07258.

[2] Tong Zheng, Xidong Wu, Zheng Zhang, Zhankui He, Chaoyi Zhang, Benjamin Coleman, Ruoqiao Wei, Di Bai, Haolin Liu, Rui Liu, et al. Dream-rsi: Recursive self-improvement through evolving worlds. arXiv preprint arXiv:2609.14858, 2026.

[3] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In Advances in Neural Information Processing Systems, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf.

[4] Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and ZHAO-XIANG ZHANG. SheetCopilot: Bringing Software Productivity to the Next Level through Large Language Models. In Advances in Neural Information Processing Systems, volume 36, pages 4952–4984. Curran Associates, Inc., 2023. doi: 10.52202/075280-0220. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/0ff30c4bf31db0119a6219e0d250e037-Paper-Conference.pdf.

[5] Hao Zheng, Xinyan Guan, Hao Kong, Wenkai Zhang, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 14402–14418. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.728. URL https://aclanthology.org/2025.emnlp-main.728/.

[6] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents. In International Conference on Learning Representations, pages 15585–15606, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/4410c0711e9154a7a2d26f9b3816d1ef-Paper-Conference.pdf.

[7] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In Advances in Neural Information Processing Systems, volume 37, pages 52040–52094. Curran Associates, Inc., 2024. doi: 10.52202/079017-1650. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/5d413e48f84dc61244b6be550f1cd8f5-Paper-Datasets_and_Benchmarks_Track.pdf.

[8] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, et al. CogAgent: A Visual Language Model for GUI Agents. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024. URL https://arxiv.org/abs/2312.08914.

[9] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks? In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 11642–11662. PMLR, 2024. URL https://proceedings.mlr.press/v235/drouin24a.html.

[10] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. In International Conference on Learning Representations, pages 96934–96990, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html.

[11] OpenAI. Codex Manual, n.d. URL https://developers.openai.com/codex/codex-manual.md. Accessed October 22, 2026.

[12] Anthropic. Claude Code Overview, n.d. URL https://code.claude.com/docs/en/overview. Accessed October 22, 2026.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In International Conference on Learning Representations, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[14] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive Architectures for Language Agents. Transactions on Machine Learning Research, 2024. URL https://arxiv.org/abs/2309.02427.

[15] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-Judge: Evaluate Agents with Agents. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 80569–80611. PMLR, 2025. URL https://proceedings.mlr.press/v267/zhuge25a.html.

[16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-world Github Issues? In International Conference on Learning Representations, pages 54107–54157, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf.

[17] Xin-Sheng Chen, Jiayu Zhu, Pei-lin Li, et al. PresentBench: A Fine-Grained Rubric-Based Benchmark for Slide Generation. arXiv preprint, 2026. URL https://arxiv.org/abs/2603.07244.

[18] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pages 2383–2392. Association for Computational Linguistics, 2016. doi: 10.18653/v1/D16-1264. URL https://aclanthology.org/D16-1264/.

[19] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pages 2369–2380. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

[20] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 3277–3287. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.acl-long.254. URL https://aclanthology.org/2021.acl-long.254/.

[21] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. FinQA: A Dataset of Numerical Reasoning over Financial Data. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 3697–3711. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.300. URL https://aclanthology.org/2021.emnlp-main.300/.

[22] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. DocVQA: A Dataset for VQA on Document Images. In 2021 IEEE Winter Conference on Applications of Computer Vision (WACV), pages 2199–2208. IEEE, 2021. doi: 10.1109/wacv48630.2021.00225. URL https://doi.org/10.1109/wacv48630.2021.00225.

[23] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning. In Findings of the Association for Computational Linguistics: ACL 2022, pages 2263–2279. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-acl.177. URL https://aclanthology.org/2022.findings-acl.177/.

[24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In Advances in Neural Information Processing Systems, volume 36, pages 68539–68551, 2023. doi: 10.52202/075280-2997. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

[25] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 3102–3116. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL https://aclanthology.org/2023.emnlp-main.187/.

[26] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In Advances in Neural Information Processing Systems, volume 37, pages 126544–126565, 2024. doi: 10.52202/079017-4020. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html.

[27] Chris Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. In International Conference on Learning Representations, pages 406–441, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/01a83bc2f2732a58e6aa731e659e7101-Abstract-Conference.html.

[28] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. In International Conference on Learning Representations, pages 32593–32627, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/8a0d3ae989a382ce6e50312bc35bf7e1-Paper-Conference.pdf.

[29] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In International Conference on Learning Representations, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.

[30] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. In Proceedings of the 43rd International Conference on Machine Learning, volume 306 of Proceedings of Machine Learning Research, pages 6736–6782. PMLR, 2026. URL https://proceedings.mlr.press/v306/barres26a.html.

[31] Frank (Fangzheng) Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. In Advances in Neural Information Processing Systems, volume 38, 2025. doi: 10.52202/085713-0315. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/0d744742f6fac4d1134c019b7cef3c8a-Abstract-Datasets_and_Benchmarks_Track.html.

[32] Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, et al. OSWorld 2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. arXiv preprint arXiv:2606.29537, 2026. URL https://arxiv.org/abs/2606.29537.

[33] Wei Fan, Xinjie Shen, Xudong Guo, et al. E-Commerce Bench: Evaluating LLM Agents on Long-Horizon Autonomous Business Operation. arXiv preprint arXiv:2608.30730, 2026. URL https://arxiv.org/abs/2608.30730.

[34] Xiaohongshu Dots Studio and Evolvent AI. VibeLifeBench: Can your life agent be proactive and persistent in a living world? arXiv preprint arXiv:2608.10875, 2026. URL https://arxiv.org/abs/2608.10875.

[35] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In Forty-second International Conference on Machine Learning, 2025.

[36] Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Weizheng Wang, Hongzhang Huang, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, GuoLiang Li, Jihua Kang, and Fan Wu. Workspace-Bench 1.0: Benchmarking AI Agents on Workspace Tasks with Large-Scale File Dependencies. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139133. Accepted for publication.

[37] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 16022–16076. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.850. URL https://aclanthology.org/2024.acl-long.850/.

[38] Tao Yu, Hao Wang, Changyu Li, Shenghua Chai, Minghui Zhang, Zhongtian Luo, Yuxuan Zhou, Haopeng Jin, Zhaolu Kang, Jiabing Yang, YiFan Zhang, Xinming Wang, Hongzhu Yi, Zheqi He, Jing-Shu Zheng, Xi Yang, Yan Huang, and Liang Wang. Beyond the All-in-One Agent: Benchmarking Role-Specialized Multi-Agent Collaboration in Enterprise Workflows, 2026. URL https://arxiv.org/abs/2605.08761. Preprint; arXiv.

[39] Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik Narasimhan, and Victor Barres. τ-Knowledge: Evaluating Conversational Agents over Unstructured Knowledge. arXiv preprint arXiv:2603.04370, 2026. doi: 10.48550/arXiv.2603.04370. URL https://arxiv.org/abs/2603.04370.

[40] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for General AI Assistants. In International Conference on Learning Representations, pages 9025–9049, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/25ae35b5b1738d80f1f03a8713e405ec-Paper-Conference.pdf.

[41] Houxing Ren, Mingjie Zhan, Zimu Lu, Ke Wang, Yunqiao Yang, Haotian Hou, and Hongsheng Li. Towards Robust Real-World Spreadsheet Understanding with Multi-Agent Multi-Format Reasoning. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1906–1933. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.86. URL https://aclanthology.org/2026.acl-long.86/.

[42] Haoyu Dong, Jianbo Zhao, Yuzhang Tian, Junyu Xiong, Mengyu Zhou, Yun Lin, José Cambronero, Yeye He, Shi Han, and Dongmei Zhang. Encoding Spreadsheets for Large Language Models. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 20728–20748. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.1154. URL https://aclanthology.org/2024.emnlp-main.1154/.

[43] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement. In Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pages 1592–1604. ACM, 2024. doi: 10.1145/3650212.3680384. URL https://doi.org/10.1145/3650212.3680384.

[44] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In International Conference on Learning Representations, pages 47832–47850, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/d191ba4c8923ed8fd8935b7c98658b5f-Paper-Conference.pdf.

[45] Yibin Chen, Yifu Yuan, Zeyu Zhang, YAN ZHENG, Jinyi Liu, Fei Ni, Jianye HAO, Hangyu Mao, and Fuzheng Zhang. SheetAgent: Towards a Generalist Agent for Spreadsheet Reasoning and Manipulation via Large Language Models. In THE WEB CONFERENCE 2025, 2025. URL https://openreview.net/forum?id=AKy9ryZwi3.

[46] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable Code Actions Elicit Better LLM Agents. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 50208–50232. PMLR, 2024. URL https://proceedings.mlr.press/v235/wang24h.html.

[47] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+p: Empowering large language models with optimal planning proficiency. arXiv preprint arXiv:2304.11477, 2023. URL https://arxiv.org/abs/2304.11477.

[48] Guancheng Zeng, Xueyi Chen, Jiawang Hu, Shaohua Qi, Yaxuan Mao, Zhantao Wang, Yifan Nie, Shuang Li, Qiuyang Feng, Pengxu Qiu, Yujia Wang, Wenqiang Han, Linyan Huang, Gang Li, Jingjing Mo, and Haowen Hu. Routine: A structural planning framework for llm agent system in enterprise, 2025. URL https://arxiv.org/abs/2507.14447.

[49] Hanchao Liu, Rongjun Li, Weimin Xiong, Ziyu Zhou, and Wei Peng. WorkTeam: Constructing Workflows from Natural Language with Multi-Agents. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track), pages 20–35. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-industry.3. URL https://aclanthology.org/2025.naacl-industry.3/.

[50] Thomas W. Malone and Kevin Crowston. The interdisciplinary study of coordination. ACM Computing Surveys, 26(1):87–119, 1994. doi: 10.1145/174666.174668.

[51] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Robert Tang, Xiangtao Lu, Xiawu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Yongxin Ni, Zhibin Gou, Zongze Xu, Yuyu Luo, and Chenglin Wu. Data Interpreter: An LLM Agent for Data Science. In Findings of the Association for Computational Linguistics: ACL 2025, pages 19796–19821. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.1016. URL https://aclanthology.org/2025.findings-acl.1016/.

[52] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents. In Second Conference on Language Modeling, 2025. URL https://openreview.net/forum?id=zg5is4GJ3R.

[53] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In First Conference on Language Modeling, 2024. URL https://openreview.net/forum?id=BAakY1hNKS.

[54] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, zili wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In International Conference on Learning Representations, pages 23247–23275, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf.

[55] Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Li Ming. InfiAgent: An Infinite-Horizon Framework for General-Purpose Autonomous Agents. In Findings of the Association for Computational Linguistics: ACL 2026, pages 35884–35894. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1787. URL https://aclanthology.org/2026.findings-acl.1787/.

[56] Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. In Advances in Neural Information Processing Systems, volume 37, pages 94871–94908, 2024. doi: 10.52202/079017-3007. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ac840df270ac537dd74530a15c332684-Abstract-Datasets_and_Benchmarks_Track.html.

[57] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2305.11738.

[58] Hao Zheng, Guozhao Mo, Xinru Yan, Qianhao Yuan, Wenkai Zhang, Xuanang Chen, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. DeepPresenter: Environment-Grounded Reflection for Agentic Presentation Generation. In Findings of the Association for Computational Linguistics: ACL 2026, pages 31545–31558. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.1578. URL https://aclanthology.org/2026.findings-acl.1578/.

[59] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In Advances in Neural Information Processing Systems, volume 36, pages 46534–46594, 2023. doi: 10.52202/075280-2019. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.

[60] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, volume 36, pages 8634–8652, 2023. doi: 10.52202/075280-0377. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[61] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 62138–62160. PMLR, 2024. URL https://proceedings.mlr.press/v235/zhou24r.html.

[62] Guanting Dong, Xiaoshuai Song, Yuyang Hu, et al. Towards Long-Horizon Agents: A Survey — Foundation, Evolution, Harness, Optimization, Application, and Frontier, 2026. URL https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents.

[63] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, Ryan Bloom, Thomas Broadley, Haoxing Du, Brian Goodrich, Nikola Jurkovic, Luke Miles, Seraphina Nix, Tao Lin, Neev Parikh, David Rein, Lucas Jun Koba Sato, Hjalmar Wijk, Daniel Ziegler, Elizabeth Barnes, and Lawrence Chan. Measuring AI Ability to Complete Long Software Tasks. In Advances in Neural Information Processing Systems, volume 38, pages 92213–92266, 2025. doi: 10.52202/085713-3086.

URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/85069585133c4c168c865e65d72e9775-Abstract-Conference.html.

[64] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 66772–66832. PMLR, 2025. URL https://proceedings.mlr.press/v267/wijk25a.html.

[65] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In International Conference on Learning Representations, pages 86809–86836, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html.

[66] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 13851–13870. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.747. URL https://aclanthology.org/2024.acl-long.747/.

[67] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In International Conference on Learning Representations, 2025. URL https://arxiv.org/abs/2410.05243.

[68] Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. GUI-Actor: Coordinate-Free Visual Grounding for GUI Agents. In Advances in Neural Information Processing Systems, volume 38, pages 15101–15128, 2025. doi: 10.52202/085713-0508. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/16130af940e9dabb43c726119bd3b42e-Abstract-Conference.html.

[69] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for Pure Vision Based GUI Agent. arXiv preprint arXiv:2408.00203, 2024. URL https://arxiv.org/abs/2408.00203.

[70] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, et al. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 40, pages 17608–17616, 2026.

[71] Zhengxi Lu, Jiabo Ye, Fei Tang, Yongliang Shen, Haiyang Xu, Ziwei Zheng, Weiming Lu, Ming Yan, Fei Huang, Jun Xiao, et al. Ui-s1: Advancing gui automation via semi-online reinforcement learning. arXiv preprint arXiv:2509.11543, 2025.

[72] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9313–9332, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.505. URL https://aclanthology.org/2024.acl-long.505/.

[73] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 6864–6890. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.371. URL https://aclanthology.org/2024.acl-long.371/.

[74] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. In ICLR 2024 Workshop on Large Language Model Agents, 2024. URL https://openreview.net/forum?id=3WWFrg8UjJ.

[75] Huawei Cloud. Huawei Cloud OfficeAce Office Agent Opens for Invitation Testing, 2026. URL https://www.huaweicloud.com/news/2026/20260416163440660.html. Official product announcement, April 16, 2026. Accessed October 26, 2026.

[76] Zilong Wang, Yuedong Cui, Li Zhong, et al. OfficeBench: Benchmarking Language Agents across Multiple Applications for Office Automation. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.19056.

[77] Junjie Xing, Yeye He, Mengyu Zhou, Haoyu Dong, Shi Han, Lingjiao Chen, Dongmei Zhang, Surajit Chaudhuri, and H. V. Jagadish. MMTU: A Massive Multi-Task Table Understanding and Reasoning Benchmark. In NeurIPS 2025 Datasets and Benchmarks Track, 2025. URL https://papers.neurips.cc/paper_files/paper/2025/hash/63b3985ad1dade84a34d581590e5f608-Abstract-Datasets_and_Benchmarks_Track.html.

[78] Renze Lou, Hanzi Xu, Sijia Wang, Jiangshu Du, Ryo Kamoi, Xiaoxin Lu, Jian Xie, Yuxuan Sun, Yusen Zhang, Jihyun Janice Ahn, Hongchao Fang, Zhuoyang Zou, Wenchao Ma, Xi Li, Kai Zhang, Congying Xia, Lifu Huang, and Wenpeng Yin. AAAR-1.0: Assessing AI's Potential to Assist Research. In ICML 2025, 2025. URL https://proceedings.mlr.press/v267/lou25c.html.

[79] Yitao Long, Tiansheng Hu, Yilun Zhao, Arman Cohan, and Chen Zhao. FinLFQA: Evaluating Attributed Text Generation of LLMs in Financial Long-Form Question Answering. In Findings of EMNLP 2025, 2025. URL https://aclanthology.org/2025.findings-emnlp.908/.

[80] Afra Feyza Akyürek, Advait Gosai, Chen Bo Calvin Zhang, Vipul Gupta, Jaehwan Jeong, Anisha Gunjal, Tahseen Rabbani, Maria Mazzone, David Randolph IV, Mohammad Mahmoudi Meymand, Gurshaan Chattha, Paula Rodriguez, Diego A. Mares Buendia, Pavit Singh, Michael Liu, Subodh Chawla, Peter Cline, Lucy Ogaz, Ernesto Gabriel Hernández Montoya, Zihao Wang, Pavi Bhatter, Marcos Ayestaran, Bing Liu, and Yunzhong He. PRBench: Large-Scale Expert Rubrics for Evaluating High-Stakes Professional Reasoning. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 42297–42325. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1958. URL https://aclanthology.org/2026.acl-long.1958/.

[81] Zhaowei Wang, Wenhao Yu, Xiyu Ren, Jipeng Zhang, Yu Zhao, Rohit Saxena, Liang Cheng, Ginny Wong, Simon See, Pasquale Minervini, Yangqiu Song, and Mark Steedman. MMLongBench: Benchmarking Long-Context Vision-Language Models Effectively and Thoroughly. In NeurIPS 2025 Datasets and Benchmarks Track, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/9c8712a9e6d34d60edb8c4c980d4a0f2-Paper-Datasets_and_Benchmarks_Track.pdf.

[82] Jin Huang, Silviu Cucerzan, Sujay Kumar Jauhar, and Ryen W. White. Idea2Plan: Exploring AI-Powered Research Planning. arXiv preprint arXiv:2510.24891, 2025. URL https://arxiv.org/abs/2510.24891.

[83] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. TravelPlanner: A Benchmark for Real-World Planning with Language Agents. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 54590–54613. PMLR, 2024. URL https://proceedings.mlr.press/v235/xie24j.html.

[84] Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Benchmarking Agentic Workflow Generation. In ICLR 2025, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/adbe936993aa7cf41e45054d8b72f183-Abstract-Conference.html.

[85] Yuhao Du, Shunian Chen, Wenbo Zan, Peizhao Li, Mingxuan Wang, Dingjie Song, Bo Li, Yan Hu, and Benyou Wang. BlenderLLM: Training Large Language Models for Computer-Aided Design with Self-improvement. arXiv preprint arXiv:2412.14203, 2024. URL https://arxiv.org/abs/2412.14203.

[86] Anna C. Doris, Jacob Thomas Sony, Ghadi Nehme, Era Syla, Amin Heyrani Nobari, and Faez Ahmed. CADBench: A Multimodal Benchmark for AI-Assisted CAD Program Generation. arXiv preprint arXiv:2605.10873, 2026. URL https://arxiv.org/abs/2605.10873.

[87] Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. TaPas: Weakly Supervised Table Parsing via Pre-training. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 4320–4333. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.398. URL https://aclanthology.org/2020.acl-main.398/.

[88] Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Sun. TableLlama: Towards Open Large Generalist Models for Tables. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 6024–6044. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.naacl-long.335. URL https://aclanthology.org/2024.naacl-long.335/.

[89] Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, Zhiyuan Zhao, Rui Xu, Kaiwen Liu, Yuan Qu, Fukai Shang, et al. MinerU: An Open-Source Solution for Precise Document Content Extraction. arXiv preprint arXiv:2409.18839, 2024. URL https://arxiv.org/abs/2409.18839.

[90] Ahmed Masry, Mohammed Saidul Islam, Mahir Ahmed, Aayush Bajaj, Firoz Kabir, Aaryaman Kartha, Md Tahmid Rahman Laskar, Mizanur Rahman, Shadikur Rahman, Mehrad Shahmohammadi, Megh Thakkar, Md Rizwan Parvez, Enamul Hoque, and Shafiq Joty. ChartQAPro: A More Diverse and Challenging Benchmark for Chart Question Answering. In Findings of ACL 2025, 2025. URL https://aclanthology.org/2025.findings-acl.978/.

[91] Suhaas Garre, Emily Ritchie, Sushant Mehta, and Edwin Chen. GDP.pdf: Benchmarking Grounded Multimodal Reasoning over Professional PDF Documents. arXiv preprint arXiv:2607.11192, 2026. URL https://arxiv.org/abs/2607.11192.

[92] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.183. URL https://aclanthology.org/2025.acl-long.183/.

[93] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A New Benchmark for Financial Question Answering. arXiv preprint, 2023. URL https://arxiv.org/abs/2311.11944.

[94] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3119–3137. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

[95] Hongchen Wei, Yuanzhe Wang, Bei Liu, Yifan Yang, Qi Dai, Ruichun Ma, Kai Qiu, Yunsheng Li, Dongdong Chen, Chong Luo, Zhenzhong Chen, and Baining Guo. XL-DocBench: Benchmarking Evidence-Grounded Extra-Long Document Understanding. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139062. Accepted for publication.

[96] Rubèn Tito, Dimosthenis Karatzas, and Ernest Valveny. Hierarchical multimodal transformers for Multipage DocVQA. Pattern Recognition, 144:109834, 2023. doi: 10.1016/j.patcog.2023.109834. URL https://doi.org/10.1016/j.patcog.2023.109834.

[97] Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, Seunghyun Yoon, Ryan A. Rossi, and Franck Dernoncourt. PDFTriage: Question Answering over Long, Structured Documents. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pages 153–169. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-industry.13. URL https://aclanthology.org/2024.emnlp-industry.13/.

[98] Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. ConvFinQA: Exploring the Chain of Numerical Reasoning in Conversational Finance Question Answering. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pages 6279–6292. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.421. URL https://aclanthology.org/2022.emnlp-main.421/.

[99] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. In International Conference on Machine Learning, pages 10764–10799, 2023. URL https://proceedings.mlr.press/v202/gao23f.html.

[100] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. Transactions on Machine Learning Research, 2023. URL https://arxiv.org/abs/2211.12588.

[101] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Aditya K, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory Dickinson, Haggai Porat, Jason Hegland, Jessica Wu, Joe Nudell, Joel Niklaus, John Nay, Jonathan Choi, Kevin Tobia, Margaret Hagan, Megan Ma, Michael Livermore, Nikon Rasumov-Rahe, Nils Holzenberger, Noam Kolt, Peter Henderson, Sean Rehaag, Sharad Goel, Shang Gao, Spencer Williams, Sunny

Gandhi, Tom Zur, Varun Iyer, and Zehua Li. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In Advances in Neural Information Processing Systems, volume 36, pages 44123–44279, 2023. doi: 10.52202/075280-1915. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/89e44582fd28ddfea1ea4dcb0ebbf4b0-Abstract-Datasets_and_Benchmarks.html.

[102] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A Dataset for Biomedical Research Question Answering. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 2567–2577. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1259. URL https://aclanthology.org/D19-1259/.

[103] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to Explain: Multimodal Reasoning via Thought Chains for Science Question Answering. In Advances in Neural Information Processing Systems, volume 35, pages 2507–2521. Curran Associates, Inc., 2022. doi: 10.52202/068431-0182. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/11332b6b6cf4485b84afadb1352d3a9a-Paper-Conference.pdf.

[104] Philippe Laban, Tobias Schnabel, and Jennifer Neville. LLMs Corrupt Your Documents When You Delegate. arXiv preprint arXiv:2604.15597, 2026. URL https://arxiv.org/abs/2604.15597.

[105] Xinyun Chen, Petros Maniatis, Rishabh Singh, Charles Sutton, Hanjun Dai, Max Lin, and Denny Zhou. SpreadsheetCoder: Formula Prediction from Semi-structured Context. In Proceedings of the 38th International Conference on Machine Learning, volume 139 of Proceedings of Machine Learning Research, pages 1661–1672. PMLR, 2021. URL https://proceedings.mlr.press/v139/chen21m.html.

[106] Wei Zhao, Zhitao Hou, Siyuan Wu, Yan Gao, Haoyu Dong, Yao Wan, Hongyu Zhang, Yulei Sui, and Haidong Zhang. NL2Formula: Generating spreadsheet formulas from natural language queries. In Yvette Graham and Matthew Purver, editors, Findings of the Association for Computational Linguistics: EACL 2024, pages 2377–2388, St. Julian's, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-eacl.158. URL https://aclanthology.org/2024.findings-eacl.158/.

[107] Mukul Singh, José Cambronero, Sumit Gulwani, Vu Le, Carina Negreanu, Elnaz Nouri, Mohammad Raza, and Gust Verbruggen. FormaT5: Abstention and Examples for Conditional Table Formatting with Natural Language. Proceedings of the VLDB Endowment, 17(3):497–510, 2023. doi: 10.14778/3632093.3632111. URL https://doi.org/10.14778/3632093.3632111.

[108] Mansi Uniyal, Agamdeep Singh, Ananya Singha, Priyanshu Gupta, Mukul Singh, Gust Verbruggen, Vu Le, and Sumit Gulwani. Back to the Future: A workbook time machine for spreadsheet creation benchmarks. In COLM 2026, 2026. URL https://openreview.net/forum?id=7COs0aR7ra.

[109] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. ACM SIGPLAN Notices, 46(1):317–330, 2011. doi: 10.1145/1925844.1926423. URL https://doi.org/10.1145/1925844.1926423.

[110] Sumit Gulwani and Mark Marron. NLyze: Interactive Programming by Natural Language for SpreadSheet Data Analysis and Manipulation, 2014. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/sigmod14.pdf.

[111] Justin Payan, Swaroop Mishra, Mukul Singh, Carina Negreanu, Christian Poelitz, Chitta Baral, Subhro Roy, Rasika Chakravarthy, Benjamin Van Durme, and Elnaz Nouri. InstructExcel: A Benchmark for Natural Language Instruction in Excel. In Findings of the Association for Computational Linguistics: EMNLP 2023, pages 4026–4043. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-emnlp.265. URL https://aclanthology.org/2023.findings-emnlp.265/.

[112] Yiduo Guo, Zekai Zhang, Yaobo Liang, Dongyan Zhao, and Nan Duan. PPTC Benchmark: Evaluating Large Language Models for PowerPoint Task Completion. In Findings of the Association for Computational Linguistics: ACL 2024, pages 8682–8701. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.514. URL https://aclanthology.org/2024.findings-acl.514/.

[113] Zihao Lin, Wanrong Zhu, Jiuxiang Gu, Jihyung Kil, Christopher Tensmeyer, Lin Zhang, Shilong Liu, Ruiyi Zhang, Lifu Huang, Vlad I. Morariu, and Tong Sun. MiLDEdit: Reasoning-Based Multi-Layer Design Document Editing. arXiv preprint arXiv:2601.04589, 2026. URL https://arxiv.org/abs/2601.04589.

[114] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting. In First Conference on Language Modeling, 2024. URL https://openreview.net/forum?id=4HNAwZFDcH.

[115] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. In Proceedings of the 33rd ACM International Conference on Multimedia, pages 8778–8786. ACM, 2025. doi: 10.1145/3746027.3755688. URL https://doi.org/10.1145/3746027.3755688.

[116] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. MedAgentBench: A Realistic Virtual EHR Environment to Benchmark Medical LLM Agents. arXiv preprint arXiv:2501.14654, 2025. URL https://arxiv.org/abs/2501.14654.

[117] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In Advances in Neural Information Processing Systems, volume 35, pages 20744–20757. Curran Associates, Inc., 2022. doi: 10.52202/068431-1508. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf.

[118] Dan Zhang, Sining Zhoubian, Min Cai, Fengzu Li, Lekang Yang, Wei Wang, Tianjiao Dong, Ziniu Hu, Jie Tang, and Yisong Yue. DataSciBench: An LLM Agent Benchmark for Data Science. In Findings of the Association for Computational Linguistics: ACL 2026, pages 3685–3728. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.181. URL https://aclanthology.org/2026.findings-acl.181/.

[119] Tsu-Jui Fu, William Yang Wang, Daniel McDuff, and Yale Song. DOC2PPT: Automatic Presentation Slides Generation from Scientific Documents. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 36, pages 634–642, 2022. doi: 10.1609/aaai.v36i1.19943. URL https://doi.org/10.1609/aaai.v36i1.19943.

[120] Jiaxin Ge, Zora Zhiruo Wang, Xuhui Zhou, Yi-Hao Peng, Sanjay Subramanian, Qinyue Tan, Maarten Sap, Alane Suhr, Daniel Fried, Graham Neubig, and Trevor Darrell. AutoPresent: Designing Structured Visuals from Scratch. In 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 2902–2911. IEEE, 2025. doi: 10.1109/cvpr52734.2025.00276. URL https://doi.org/10.1109/cvpr52734.2025.00276.

[121] Jian Zhu, Yuzheng Zhang, Zeyao Ma, Bohan Zhang, Armin Schoepf, Daniel Woloch, Peter Yiliu Wang, Guangyu Robert Yang, Samuel Jacob, Siddharth Nagisetty, Abhiram Chundru, Jean Lin, Spencer Mateega, and Jing Zhang. SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139155. Accepted for publication.

[122] Mike Merrill, Alexander Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong (Ryan) Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In International Conference on Learning Representations, pages 40903–40986, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/444a3737adaee10d86ad2ef5f74468e6-Paper-Conference.pdf.

[123] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. In Conference on Machine Learning and Systems, 2025. URL https://arxiv.org/abs/2501.06706.

[124] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Parag Shah, Yujia Bao, and Eugene Siow. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. In International Conference on Learning Representations, pages

97377–97425, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/9e4b14eb6f16fe7b5818a8d633a0606a-Paper-Conference.pdf.

[125] Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers, 2025. URL https://arxiv.org/abs/2508.14704. Preprint; arXiv:2508.14704; verified 2026-10-16.

[126] Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. Dabstep: Data agent benchmark for multi-step reasoning. arXiv preprint arXiv:2506.23719, 2025. URL https://arxiv.org/abs/2506.23719.

[127] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In Findings of the Association for Computational Linguistics: NAACL 2025, pages 1160–1183. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL https://aclanthology.org/2025.findings-naacl.65/.

[128] Yikai Zhang, Siyu Yuan, Caiyu Hu, Kyle Richardson, Yanghua Xiao, and Jiangjie Chen. TimeArena: Shaping efficient multitasking language agents in a time-aware simulation. In Annual Meeting of the Association for Computational Linguistics, 2024. URL https://arxiv.org/abs/2402.05733.

[129] Banghao Chi, Yining Xie, Mingyuan Wu, et al. Spreadsheet-RL: Advancing Large Language Model Agents on Realistic Spreadsheet Tasks via Reinforcement Learning. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.22642.

[130] Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Chenyang Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. Vision-as-Inverse-Graphics Agent via Interleaved Multimodal Reasoning. In Computer Vision – ECCV 2026, volume 17035 of Lecture Notes in Computer Science, pages 557–575. Springer Nature Switzerland, 2026. doi: 10.1007/978-3-032-37464-6_30. URL https://link.springer.com/chapter/10.1007/978-3-032-37464-6_30.

[131] Boxi Yu, Yuzhong Zhang, Zhenfeng Su, Linfu Xi, Yinghan Yao, Shitao Chen, Yang Cao, Liting Lin, Lionel Briand, and Emir Muñoz. CXBench: A Benchmark for Process-Compliant LLM Agents in Regulated Customer-Experience Workflows. arXiv preprint arXiv:2610.25954, 2026. URL https://arxiv.org/abs/2610.25954.

[132] Anthropic. How we built our multi-agent research system, June 2025. URL https://www.anthropic.com/research/multi-agent-research-system. Published June 13, 2025; accessed October 22, 2026.

[133] Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. WorkArena++: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. In Advances in Neural Information Processing Systems, volume 37, pages 5996–6051. Curran Associates, Inc., 2024. doi: 10.52202/079017-0195. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/0b82662b6c32e887bb252a74d8cb2d5e-Paper-Datasets_and_Benchmarks_Track.pdf.

[134] Karthik Ragunath Ananda Kumar and Subrahmanyam Arunachalam. Learning to Present: Inverse Specification Rewards for Agentic Slide Generation. arXiv preprint, 2026. URL https://arxiv.org/abs/2603.16839.

[135] Patrick Tser Jern Kon, Qiuyi Ding, Jiachen Liu, Xinyi Zhu, Jingjia Peng, Jiarong Xing, Yibo Huang, Yiming Qiu, Jayanth Srinivasa, Myungjin Lee, Mosharaf Chowdhury, Matei Zaharia, and Ang Chen. EXP-Bench: Can AI Conduct AI Research Experiments? In ICLR 2026, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/c411f5b2d9c55f1685e72db224ad8b0e-Abstract-Conference.html.

[136] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI's Ability to Replicate AI Research. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 56843–56873. PMLR, 2025. URL https://proceedings.mlr.press/v267/starace25a.html.

[137] Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, Jianwen Luo, Xin Zou, Ruiyi Yang, Wenbo Shi, Yan Gao, Shizhu He, Zuo Wang, Qian Liu, Yang Wang, Ke Wang, Jun Zhao, and Kang Liu. DAComp: Benchmarking Data Agents across the Full Data Intelligence Lifecycle. In ICLR 2026, 2026. URL https://openreview.net/pdf?id=EtzJy9yI5J.

[138] Haolin Chen, Deon Metelski, Leon Qi, Tao Xia, Joonyul Lee, Steve Brown, Kevin Riley, Frank Wang, T. Y. Alvin Liu, Hank Capps, Zeyu Tang, Xiangchen Song, Lingjing Kong, Fan Feng, Tianyi Zeng, Zhiwei Liu, Zixian Ma, Hang Jiang, Fangli Geng, Yuan Yuan, Chenyu You, Qingsong Wen, Hua Wei, Yanjie Fu, Yue Zhao, Carl Yang, Biwei Huang, Kun Zhang, Caiming Xiong, Sanmi Koyejo, Eric P. Xing, Philip S. Yu, and Weiran Yao. CHI-Bench: Can AI Agents Automate End-to-End, Long-Horizon, Policy-Rich Healthcare Workflows? In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139361. Accepted for publication.

[139] Shiva Krishna Reddy Malay, Shravan Nayak, Sagar Davasam, Srinivas Sunkara, and Sai Rajeswar. EnterpriseOps-Gym: Environments and Evaluations for Stateful Agentic Planning and Tool Use in Enterprise Settings. In Proceedings of the 43rd International Conference on Machine Learning, volume 306 of Proceedings of Machine Learning Research, pages 85969–86001. PMLR, 2026. URL https://proceedings.mlr.press/v306/malay26a.html.

[140] Daniel Shepard and Robin Salimans. AutomationBench. arXiv preprint, 2026. URL https://arxiv.org/abs/2604.18934.

[141] Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. Collaborative Gym: A Framework for Enabling and Evaluating Human-Agent Collaboration. In International Conference on Learning Representations, pages 99616–99649, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/a1c8a68e52499c9396854e3f967e37c0-Paper-Conference.pdf.

[142] Microsoft. Microsoft 365 Copilot Architecture and How It Works, 2026. URL https://learn.microsoft.com/en-us/copilot/microsoft-365/microsoft-365-copilot-architecture.

[143] Qisheng Su, Hanchen Wang, Guanru Zhu, Huicheng Jiang, Qiuyinzhe Zhang, Kou Shi, Zhen Fang, Ziao Zhang, Qingnan Ren, Honglin Guo, Zehui Chen, Tao Gui, and Feng Zhao. GraphForge: Training Working Agents with Graph-Anchored Workspace Synthesis. arXiv preprint arXiv:2609.38923, 2026. URL https://arxiv.org/abs/2609.38923.

[144] Xinyu Zhu, Fenyi Liu, Yuzhu Cai, Shuo Tang, Rui Ye, Linfeng Zhang, and Siheng Chen. WorkGenesis: Building the Worlds That Teach Agents to Work. arXiv preprint arXiv:2609.39325, 2026. URL https://arxiv.org/abs/2609.39325.

[145] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An enterprise benchmark for end-to-end grounded reasoning, 2026. URL https://arxiv.org/abs/2603.08655.

[146] Amirhossein Abaskohi, Tianyi Chen, Miguel Muñoz-Mármol, Curtis Fox, Amrutha Varshini Ramesh, Étienne Marcotte, Xing Han Lu, Nicolas Chapados, Spandana Gella, Christopher Pal, Alexandre Drouin, and Issam Laradji. DRBench: A Realistic Benchmark for Enterprise Deep Research. In ICLR 2026, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/0ba520d93c3df592c83a611961314c98-Abstract-Conference.html.

[147] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient Document Retrieval with Vision Language Models. In International Conference on Learning Representations, 2025. URL https://arxiv.org/abs/2407.01449.

[148] Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, Yixin Mao, Huan Wang, Silvio Savarese, Caiming Xiong, Philippe Laban, and Chien-Sheng Wu. CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments. In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 3830–3850. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.194. URL https://aclanthology.org/2025.naacl-long.194/.

[149] Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng LIU, Zhaochen Su, Yiyang Guo, FAN ZHOU, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. In International Conference on Learning Representations, pages 53359–53394, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/5783212d85c205ef823b8974d44872c5-Paper-Conference.pdf.

[150] Weihao Zeng, Yuzhen Huang, and Junxian He. LOCA-bench: Benchmarking language agents under controllable

and extreme context growth. arXiv preprint arXiv:2602.07962, 2026. doi: 10.48550/arXiv.2602.07962. URL https://arxiv.org/abs/2602.07962.

[151] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-Guided LLM Agents for Code Localization. In ACL 2025 main, 2025. URL https://aclanthology.org/2025.acl-long.426/.

[152] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In International Conference on Learning Representations, pages 65882–65919, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.

[153] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can Frontier LLMs Earn $1 Million from Real-World Freelance Software Engineering? In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 44412–44450. PMLR, 2025. URL https://proceedings.mlr.press/v267/miserendino25a.html.

[154] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin SU, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In International Conference on Learning Representations, pages 28691–28735, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/46c10f6c8ea5aa6f267bcdabcb123f97-Abstract-Conference.html.

[155] Honglin Guo, Qi Zhang, Yu Zhang, Weijie Li, Rui Zheng, Zhikai Lei, Qiyuan Peng, Zhiheng Xi, Tao Gui, and Qi Zhang. Agora: An Archive-Grounded Benchmark for Agentic Workplace Document Reasoning, 2026. URL https://arxiv.org/abs/2606.24526. Preprint; arXiv:2606.24526.

[156] Prafulla Kumar Choubey, Xiangyu Peng, Shilpa Bhagavath, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Benchmarking Deep Search over Heterogeneous Enterprise Data. In EMNLP 2025 Industry Track, 2025. URL https://aclanthology.org/2025.emnlp-industry.34/.

[157] Bertie Vidgen, Austin Mann, Abby Fennelly, et al. APEX-Agents. arXiv preprint, 2026. URL https://arxiv.org/abs/2601.14242.

[158] Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, Jiarui Hu, Liming Liu, Jinlong Hou, and Pengfei Liu. daVinci-Env: Open SWE Environment Synthesis at Scale. arXiv preprint arXiv:2603.13023, 2026. URL https://arxiv.org/abs/2603.13023.

[159] Jiarong Zhao, Zhikai Lei, Zhiheng Xi, Rui Zheng, Hang Yan, Jie Zhou, Qin Chen, and Liang He. NexForge: Scaling Agent Capabilities through Requirement-Driven Task Synthesis for LLMs. arXiv preprint arXiv:2607.14186, 2026. URL https://arxiv.org/abs/2607.14186.

[160] Jiazheng Zhang, Long Ma, Yunxian Yang, Zhiheng Xi, Zhikai Lei, Yajie Yang, Chenyang Liao, Enyu Zhou, Yang Nan, Yuchen Tian, Senjie Jin, Yibo Wang, Wei He, Boyang Liu, Jixuan Huang, Xin Guo, Shutong Jiang, Zhezheng Hao, Xinbing Liang, Zhihao Zhang, Changzhi Zhou, Wiggin Zhou, Tao Gui, Qi Zhang, Xuanjing Huang, Clarenceai, and Aiden Adams. Scaling Verifiable Environments for Long-horizon Work Agents. arXiv preprint arXiv:2610.04906, 2026. URL https://arxiv.org/abs/2610.04906.

[161] Junyi Zhou, Yufei Wang, Qiyuan Zhang, Chuhan Wu, Zhengxi Li, Wentao Zhao, Jie Zhou, Le Tian, Xiao Zhou, and Chen Ma. WorkBorne: Discovering Agentic Tasks from Real-World Artifact Transitions. arXiv preprint arXiv:2610.14915, 2026. URL https://arxiv.org/abs/2610.14915.

[162] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent World Model: Infinity Synthetic Environments for Agentic Reinforcement Learning. In Proceedings of the 43rd International Conference on Machine Learning, volume 306 of Proceedings of Machine Learning Research, pages 131469–131511. PMLR, 2026. URL https://proceedings.mlr.press/v306/wang26jh.html.

[163] Minbyul Jeong and Chanwoong Yoon. AgentMercury: Your Agent Can Synthesize Verifiable Environments for Business Scenarios at scale. arXiv preprint arXiv:2608.20634, 2026. URL https://arxiv.org/abs/2608.20634.

[164] Yirong Zeng, Shen You, Jinhang Feng, Yufei Liu, Xiao Ding, Yutai Hou, Hao Cong, Yuxian Wang, Wu Ning, Wang Xu, and Bibo Cai. EnvCraft: Synthesizing Executable Environments in Agentic RL for Claw-like Agent. arXiv preprint arXiv:2609.05576, 2026. URL https://arxiv.org/abs/2609.05576.

[165] Xiao-Wen Yang, Weiyi Xu, Wen Da, Hang Xu, Canwei Li, Hong-Jie You, Pusen Dong, Yucheng Zeng, Zhaokai Luo, Yu-Feng Li, Yao Hu, and Mu Chuan. CompoWorld: Compositional Environment Scaling for General Agents. arXiv preprint arXiv:2609.33665, 2026. URL https://arxiv.org/abs/2609.33665v2. Version 2.

[166] Zhongwei Xie, Yiming Du, Jiaxin Bai, Baixuan Xu, Chunyang Li, Xiyu Ren, Wei Fan, Tianshi Zheng, Weizhi Du, Jingqiao Yu, Xingsu Li, Zhaowei Wang, and Yangqiu Song. CoworkEnv: Synthesizing Training Environments for Long-Horizon Cowork Agents. arXiv preprint arXiv:2610.24960, 2026. URL https://arxiv.org/abs/2610.24960.

[167] Anni Zou, Cheng Chen, Jiaji Deng, Weikang Zhou, Xinji Mai, Yue Wu, Jianing Lin, Lipeng Xie, Lixiong Qin, Qingyue Zhao, Que Shen, Shuchang Tao, Xuchen Pan, Yaliang Li, Yunpeng Zhai, Yushuo Chen, Zhihai Wang, Zhuo Zhang, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. Cowork-Gym: Synthesizing Interactive Work Environments at Scale. arXiv preprint arXiv:2610.17670, 2026. URL https://arxiv.org/abs/2610.17670.

[168] Hang He, Linting Wang, Guanming Liu, Bo Mao, Jinxiu Liu, Yiran Suo, Maosen Zhou, Zhiheng Xi, Guanting Dong, Zhou Shao, Rui Zheng, Jie Zhou, Chengcheng Wan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. WorkRover: Scaling Long-Horizon Cowork Agent Training Via Recursive Environment Evolution. arXiv preprint arXiv:2610.18067, 2026. URL https://arxiv.org/abs/2610.18067.

[169] Ji Zeng, Dayuan Fu, Tiantian Mi, Zhuang Yumin, Yaxing Huang, Xuefeng Li, Lyumanshan Ye, Muhang Xie, Qishuo Hua, Zhen Huang, Mohan Jiang, Hanning Wang, Jifan Lin, Yang Xiao, Jie Sun, Yunze Wu, and Pengfei Liu. daVinci-Dev: Agent-native Mid-training for Software Engineering. In Proceedings of the 43rd International Conference on Machine Learning, volume 306 of Proceedings of Machine Learning Research, pages 153265–153282. PMLR, 2026. URL https://proceedings.mlr.press/v306/zeng26d.html.

[170] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In International Conference on Learning Representations, 2025. URL https://arxiv.org/abs/2412.09605.

[171] Ankush Agarwal, Harsh Vishwakarma, Suraj Nagaje, and Chaitanya Devaguptapu. EnterpriseLab: A Full-Stack Platform for developing and deploying agents in Enterprises. arXiv preprint arXiv:2603.21630, 2026. URL https://arxiv.org/abs/2603.21630.

[172] Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, Ryan Marten, Charlie F. Ruan, Tyler Griggs, Alexander Glenn Shaw, Hritik Bansal, E. Kelly Buchanan, Artem Gazizov, Reinhard Heckel, Chinmay Hegde, Sankalp Jajee, Daanish Khazi, Emmanouil Koukoumidis, Xiangyi Li, Hange Liu, Shlok Natarajan, Harsh Raj, Nicholas Roberts, Ethan Shen, Nishad Singhi, Michael Siu, Ashima Suvarna, Hanwen Xing, Patrick Yubeaton, Robert Zhang, Leon Liangyu Chen, Xiaokun Chen, Steven Dillmann, Saadia Gabriel, Xunyi Jiang, Anurag Kashyap, Boxuan Li, Yein Park, Minh Pham, Sujay Sanghavi, Lin Shi, Ke Sun, Yixin Wang, Zhiwei Xu, Erica Zhang, Siyan Zhao, Wanjia Zhao, Jenia Jitsev, Alex Dimakis, Benjamin Feuer, and Ludwig Schmidt. OpenThoughts-Agent: Data Recipes for Agentic Models. arXiv preprint arXiv:2606.24855, 2026. URL https://arxiv.org/abs/2606.24855.

[173] Dingzirui Wang, Xuanliang Zhang, Keyan Xu, Qingfu Zhu, and Wanxiang Che. CurateEvo: Data-Curation Evolving for Agentic Post-Training. arXiv preprint arXiv:2607.06140, 2026. URL https://arxiv.org/abs/2607.06140.

[174] Maksim Ivanov and Abhijay Rana. Anchor: Mitigating Artifact Drift in Agent Benchmark Generation. In RLEval Workshop at the ACM Conference on AI and Agentic Systems, 2026. URL https://arxiv.org/abs/2605.26321.

[175] Junqi Liu, Yongyang Pan, Zhuosong Jiang, Dongbai Li, Bo Zhang, Xitong Ling, Sheng Wang, Hanrong Ye, Yufan He, Can Zhao, Pengfei Guo, Dong Yang, Andriy Myronenko, Yuyin Zhou, Tianyu Liu, Daguang Xu, and Yucheng Tang. VERA: Scaling Verifiable Environments for Agentic co-Evolution. arXiv preprint arXiv:2610.05923, 2026. URL https://arxiv.org/abs/2610.05923.

[176] Guhong Chen, Chenghao Sun, Cheng Fu, Qiyao Wang, Zhihong Huang, ChaoPeng Wei, Guangxu Chen, Feiteng Fang, Ahmadreza Argha, Bing Zhao, Xander Xu, Qi Han, Hamid Alinejad-Rokny, Qiang Qu, Binhua Li, Shiwen Ni, Min Yang, HU Wei, and Yongbin Li. Beyond Quantity: Trajectory Diversity Scaling for Code Agents. In Findings of the Association for Computational Linguistics: ACL 2026, pages 15676–15691. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.findings-acl.768. URL https://aclanthology.org/2026.findings-acl.768/.

[177] Zhuohang Fan, Beichen Zhang, Yuanfa Li, Changqiao Wu, Wei Liu, Jian Luan, and Weigang Zhang. SEE: Structure-aware Exploring & Exploiting for Long-horizon GUI Agent Trajectory Synthesis. In Proceedings

of the 34th ACM International Conference on Multimedia, 2026. URL https://arxiv.org/abs/2607.18046. Accepted for publication.

[178] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep Reinforcement Learning from Human Preferences. In Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf.

[179] Shangqing Tu, Daniel Zhang-Li, Yucheng Wang, Shiyu Gan, Yanpeng Wang, Huiqiang Rong, Mofei Chen, Shen Yang, Yini Chen, Yinuo Duan, Binglin Liu, Ye He, Danqi Zheng, Zhanxin Hao, Yuxuan Wu, Mengting Tao, Yuqiu Liu, Jifan Yu, Juanzi Li, Bin Xu, Lei Hou, Huiqin Liu, and Yu Zhang. CogEvol: Towards Efficient and Reliable Learning Environment Generation. arXiv preprint arXiv:2608.30968, 2026. URL https://arxiv.org/abs/2608.30968.

[180] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2305.20050.

[181] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. Process Reward Models That Think. Transactions on Machine Learning Research, 2026. URL https://openreview.net/forum?id=FPVCb0WMuN. J2C Certification.

[182] DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv preprint arXiv:2606.19348, 2026. URL https://arxiv.org/abs/2606.19348.

[183] Dongsheng Wang, Natraj Raman, Mathieu Sibue, Zhiqiang Ma, Petr Babkin, Simerjot Kaur, Yulong Pei, Armineh Nourbakhsh, and Xiaomo Liu. DocLLM: A Layout-Aware Generative Language Model for Multimodal Document Understanding. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 8529–8548. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.463. URL https://aclanthology.org/2024.acl-long.463/.

[184] Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table Pre-training via Learning a Neural SQL Executor. In International Conference on Learning Representations, 2022. URL https://arxiv.org/abs/2107.07653.

[185] Kimi Team. Kimi K2: Open Agentic Intelligence. arXiv preprint arXiv:2507.20534, 2025. URL https://arxiv.org/abs/2507.20534.

[186] Kimi Team. Kimi K2.5: Visual Agentic Intelligence. arXiv preprint arXiv:2602.02276, 2026. URL https://arxiv.org/abs/2602.02276.

[187] GLM-5 Team. GLM-5: from Vibe Coding to Agentic Engineering. arXiv preprint arXiv:2602.15763, 2026. URL https://arxiv.org/abs/2602.15763.

[188] Ailin Huang, Ang Li, Aobo Kong, Bin Wang, Binxing Jiao, Bo Dong, Bojun Wang, Boyu Chen, Brian Li, Buyun Ma, et al. Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters. arXiv preprint arXiv:2602.10604, 2026. URL https://arxiv.org/abs/2602.10604.

[189] Soumye Singhal, Joseph Jennings, Ka Shun Shum, Alexander Bukharin, Khushi Bhardwaj, Zirui Liu, Venkat Srinivasan, Jonathan Li, Yash Jain, Kezhi Kong, Markus Kliegl, Olivier Delalleau, Sanjeev Satheesh, and Mostofa Patwary. Priming Base Models for Agentic Post-Training: An Open Midtraining Recipe. arXiv preprint arXiv:2610.22019, 2026. URL https://arxiv.org/abs/2610.22019.

[190] Miao Peng, Qintong Zhang, Nuo Chen, Yuhan Li, Guochen Yan, Xinran Gu, Hongqiu Wu, Hai Wang, Lydell Huang, Wentao Zhang, and Jia Li. TrajLong: Co-Designing Agentic and Long-Context Supervision for Mid-Training. arXiv preprint arXiv:2610.04973, 2026. URL https://arxiv.org/abs/2610.04973.

[191] Yubo Wang, Jiarong Liang, Yuxuan Zhang, Xuye Liu, Cong Wei, Yuyu Zhang, Ping Nie, and Wenhu Chen. Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models. arXiv preprint arXiv:2607.12463, 2026. URL https://arxiv.org/abs/2607.12463.

[192] Qwen Team. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability. arXiv preprint arXiv:2608.30320, 2026. URL https://arxiv.org/abs/2608.30320.

[193] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling Generalized Agent Abilities for LLMs. In Findings of the Association for Computational Linguistics: ACL 2024, 2024. URL https://arxiv.org/abs/2310.12823.

[194] Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-FLAN: Designing Data and Methods of Effective Agent Tuning for Large Language Models. In Findings of the Association for Computational Linguistics: ACL 2024, pages 9354–9366. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.557. URL https://aclanthology.org/2024.findings-acl.557/.

[195] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. Fireact: Toward language agent fine-tuning. arXiv preprint arXiv:2310.05915, 2023. URL https://arxiv.org/abs/2310.05915.

[196] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 13484–13508. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.754. URL https://aclanthology.org/2023.acl-long.754/.

[197] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv preprint arXiv:2501.12326, 2025. URL https://arxiv.org/abs/2501.12326.

[198] Zihan Ding, Longxu Dou, Qi Gao, Xiangwu Guo, Shengchao Hu, Zilong Huang, Zihang Jiang, Lei Ke, Mengcheng Lan, Weixian Lei, Hanxuan Li, Honglin Li, Xiyun Li, Zaitang Li, Leowei Liang, Xin Luo, Haozhe Ma, Jiayi Mao, Zhoujie Pan, Can Qin, Tianyuan Qu, Weiqi Wang, Wenkai Wang, Yonglin Wang, Yuxin Wang, Chenxu Wu, Yingchen Yu, Chenyu Zhang, and Yuhao Zheng. UI-Mate: Advancing Open-Weight Foundation GUI Agents with In-Context Demonstrations. arXiv preprint arXiv:2608.15930, 2026. URL https://arxiv.org/abs/2608.15930.

[199] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017. URL https://arxiv.org/abs/1707.06347.

[200] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. In International Conference on Learning Representations, pages 37909–37926, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/4038c9208dfc22644c60ad39c24e5c53-Paper-Conference.pdf.

[201] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In Conference on Language Modeling, 2025. URL https://arxiv.org/abs/2503.09516.

[202] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning. In International Conference on Learning Representations, pages 79791–79821, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/c66e1fcc9691aae706250638f36f681b-Abstract-Conference.html.

[203] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. DigiRL: Training In-The-Wild Device-Control Agents with Autonomous Reinforcement Learning. In Advances in Neural Information Processing Systems, volume 37, pages 12461–12495. Curran Associates, Inc., 2024. doi: 10.52202/079017-0397. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/1704ddd0bb89f159dfe609b32c889995-Paper-Conference.pdf.

[204] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. In Advances in Neural Information Processing Systems, 2025. URL https://arxiv.org/abs/2502.18449.

[205] Congmin Zheng, Xiaoyun Mo, Xinbei Ma, Qiqiang Lin, Yin Zhao, Jiachen Zhu, Xingyu Lou, Jun Wang, Zhaoxiang Wang, Weiwen Liu, Zhuosheng Zhang, Yong Yu, and Weinan Zhang. Adaptive Milestone Reward for GUI Agents. arXiv preprint arXiv:2602.11524, 2026. URL https://arxiv.org/abs/2602.11524.

[206] Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. HiPER: Hierarchical Plan–Execute RL for Multi-Turn LLM Agents. In Proceedings of the 43rd International

Conference on Machine Learning, volume 306 of Proceedings of Machine Learning Research, pages 98057–98083. PMLR, 2026. URL https://proceedings.mlr.press/v306/peng26h.html.

[207] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-Group Policy Optimization for LLM Agent Training. In Advances in Neural Information Processing Systems, 2025. URL https://arxiv.org/abs/2505.10978.

[208] Mercor and SkyRL. Training Frontier Knowledge Work Agents: A 397B RL Training Guide with SkyRL. Technical report, September 2026. URL https://www.mercor.com/blog/training-frontier-knowledge-work-agents-a-397b-rl-training-guide-with-skyrl/.

[209] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2306.13649.

[210] Jiaqi WANG, Wenhao Zhang, Weijie Shi, Yaliang Li, and James Cheng. Exploring Temporal Curriculum in On-Policy Distillation for Multi-turn Autonomous Agents. In Third Conference on Language Modeling, 2026. URL https://openreview.net/forum?id=KH9eJRz8WR.

[211] Gengsheng Li, Mao Zheng, Mingyang Song, Ruiqi Liu, Tianyu Yang, Jie Sun, Qiyong Zhong, Haiyun Guo, Junfeng Fang, Dan Zhang, and Jinqiao Wang. On-Policy Distillation with Curriculum Turn-level Guidance for Multi-turn Agents. arXiv preprint arXiv:2606.15912, 2026. URL https://arxiv.org/abs/2606.15912.

[212] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, volume 15, pages 627–635, 2011. URL https://proceedings.mlr.press/v15/ross11a.html.

[213] Changhao Li, Rushi Qiang, Jiawei Huang, Chenxiao Gao, Chao Zhang, Niao He, and Bo Dai. Revisiting DAgger in the Era of LLM-Agents. arXiv preprint arXiv:2605.12913, 2026. URL https://arxiv.org/abs/2605.12913.

[214] Qiyong Zhong, Mao Zheng, Mingyang Song, Xin Lin, Jie Sun, Houcheng Jiang, Xiang Wang, and Junfeng Fang. SOD: Step-wise On-policy Distillation for Small Language Model Agents. arXiv preprint arXiv:2605.07725, 2026. URL https://arxiv.org/abs/2605.07725.

[215] Yubin Wu, Zicheng Cai, Liping Ning, Hua Wang, Zhi Chen, Yaohua Tang, and Hao Chen. LiteGUI: Lightweight GUI Agents via Multi-Solution Guided Distillation and Dual-Level Reinforcement Learning. arXiv preprint arXiv:2605.07505, 2026. URL https://arxiv.org/abs/2605.07505.

[216] Weiwei Li, Junzhuo Liu, Tong Chu, Hengfu Yu, and Wen Li. The Next Screenshot Knows: Gated Hindsight Distillation for Mobile GUI Agents. arXiv preprint arXiv:2608.06065, 2026. URL https://arxiv.org/abs/2608.06065.

[217] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning With Reasoning. In Advances in Neural Information Processing Systems, 2022. URL https://arxiv.org/abs/2203.14465.

[218] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-Rewarding Language Models. In International Conference on Machine Learning, 2024. URL https://arxiv.org/abs/2401.10020.

[219] Pranav Putta, Edmund Mills, Naman Garg, et al. Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2408.07199.

[220] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement Learning via Self-Distillation. In International Conference on Machine Learning, 2026. URL https://arxiv.org/abs/2601.20802.

[221] Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-SD: Skill-Conditioned Self-Distillation for Multi-turn LLM Agents. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/151567. Accepted for publication.

[222] Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, et al. Self-distilled agentic reinforcement learning. arXiv preprint arXiv:2605.15155, 2026.

[223] Xinbei Ma, Congmin Zheng, Jiyang Qiu, Jiale Hong, Yao Yao, Xiangmou Qu, Jiaxin Yin, Xingyu Lou, Jun Wang, Weiwen Liu, Weinan Zhang, Zhuosheng Zhang, Hai Zhao, and Bryan Hooi. Retrospective Progress-Aware Self-Refinement for LLM Agent Training. arXiv preprint arXiv:2606.14302, 2026. URL https://arxiv.org/abs/2606.14302.

[224] Haoxiang Zhang, Qinglin Chen, Hiroaki Hayashi, Zhuofeng Li, Siming Zhang, Jiaxin Zhang, Jixuan Chen, Fang Wu, Pan Lu, Silvio Savarese, Julian McAuley, and Chien-Sheng Wu. Self-Retrospection Distillation: Turning Post-hoc Experiences into Prior Foresight. arXiv preprint arXiv:2610.08077, 2026. URL https://arxiv.org/abs/2610.08077.

[225] Jincheng Ren, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Min Gao, Yan Bai, Jian Yang, and Chenghua Lin. A Self-Evolving Framework for Efficient Terminal Agents via Observational Context Compression. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/153290. Accepted for publication.

[226] Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to Rewrite Tool Descriptions for Reliable LLM-Agent Tool Use. arXiv preprint arXiv:2602.20426, 2026. URL https://arxiv.org/abs/2602.20426.

[227] Sandip Ghoshal, Anshul Mittal, Jyotika Singh, Miguel Ballesteros, Weiyi Sun, Fang Tu, Shailender Singh, Yassine Benajiba, Fahad Shah, Sujeeth Bharadwaj, Sujith Ravi, and Dan Roth. JTPRO: A Joint Tool–Prompt Reflective Optimization Framework for Language Agents. In Findings of the Association for Computational Linguistics: ACL 2026, pages 40573–40595, 2026. doi: 10.18653/v1/2026.findings-acl.2017. URL https://doi.org/10.18653/v1/2026.findings-acl.2017.

[228] Cen Mia Zhao, Haibo Ruan, Wenjie Chen, Pei-fen Tu, Usman Abbasi, and Joel Hesch. Beyond Prompts: Measuring and Optimizing LLM Tool-Agent Harnesses. In Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, 2026. URL https://arxiv.org/abs/2609.05736. Accepted for publication.

[229] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2507.19457.

[230] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, XiongHui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation. In International Conference on Learning Representations, pages 34040–34077, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html.

[231] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv preprint arXiv:2603.28052, 2026. URL https://arxiv.org/abs/2603.28052.

[232] Yaxin Luo, Haobin Jiang, Jialv Zou, Xu Huang, Wenhao Yan, Haodong Li, Zhengrong Yue, Jing Li, Xiaofu Chen, Xiaohan Zhao, Jiacheng Liu, Jiacheng Cui, Zhiqiang Shen, and Xiaotong Li. AutoDesign: Meta-Harness Optimization for Long-Horizon Agentic Design. arXiv preprint arXiv:2608.13560, 2026. URL https://arxiv.org/abs/2608.13560.

[233] Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren, Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan. JIT-Agent: Scaling Harness Intelligence via Just-in-Time Harness Evolution. arXiv preprint arXiv:2608.25593, 2026. URL https://arxiv.org/abs/2608.25593.

[234] Charles Packer, Sarah Wooders, Kevin Lin, et al. MemGPT: Towards LLMs as Operating Systems. arXiv preprint, 2023. URL https://arxiv.org/abs/2310.08560.

[235] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, et al. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2509.25140.

[236] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2510.04618.

[237] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/151264. Accepted for publication.

[238] Junhao Shen, Zhanqiu Zhang, Yiwen Guo, and Hong Cheng. Progressive Agent Skill Generation via Reinforcement Learning. arXiv preprint arXiv:2608.01678, 2026. URL https://arxiv.org/abs/2608.01678.

[239] Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning Self-Evolving Skills for Coding Agents. arXiv preprint arXiv:2605.25430, 2026. URL https://arxiv.org/abs/2605.25430.

[240] Yangbo Wei, Zhen Huang, Shaoqiang Lu, Junhong Qian, Qifan Wang, Chen Wu, and Lei He. SkillSmith: Co-Evolving Skills and Tools for Self-Improving Agent Systems. arXiv preprint arXiv:2606.01314, 2026. URL https://arxiv.org/abs/2606.01314.

[241] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM Agent Search in Modular Design Space. In International Conference on Learning Representations, pages 3841–3865, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/0ae94013da7cd459402fd77874e09ee3-Abstract-Conference.html.

[242] Zhiwei Shang, Yu Huo, Mingrong Gong, An Yan, Zikun Qu, Junhao Dong, Bryan Kian Hsiang Low, Chenglin Wu, and Zhongxiang Dai. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution. arXiv preprint arXiv:2610.05300, 2026. URL https://arxiv.org/abs/2610.05300.

[243] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large Language Models Are Human-Level Prompt Engineers. In International Conference on Learning Representations, 2023. URL https://arxiv.org/abs/2211.01910.

[244] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large Language Models as Optimizers. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2309.03409.

[245] Jiale Cheng, Xiao Liu, Kehan Zheng, Pei Ke, Hongning Wang, Yuxiao Dong, Jie Tang, and Minlie Huang. Black-Box Prompt Optimization: Aligning Large Language Models without Model Training. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3201–3219. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.176. URL https://aclanthology.org/2024.acl-long.176/.

[246] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, et al. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2310.03714.

[247] Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In Conference on Empirical Methods in Natural Language Processing, 2024. URL https://arxiv.org/abs/2406.11695.

[248] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. In International Conference on Learning Representations, pages 21344–21377, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/36b7acf6f6010652b3f2a433774a66fe-Abstract-Conference.html.

[249] Jiayi Zhang, Yongfeng Gu, Jianhao Ruan, Maojia Song, Yiran Peng, Zhiguang Han, Jinyu Xiang, Zhitao Wang, Caiyin Yang, Bang Liu, Chenglin Wu, and Yuyu Luo. Harnessing Agentic Evolution. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/156126. Accepted for publication.

[250] Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 7957–7968. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.494. URL https://aclanthology.org/2023.emnlp-main.494/.

[251] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing Generative AI by Backpropagating Language Model Feedback. Nature, 639(8055):609–616, 2025. doi: 10.1038/s41586-025-08661-4.

[252] Mingkai Deng, Jianyu Wang, Cheng-Ping Hsieh, Yihan Wang, Han Guo, Tianmin Shu, Meng Song, Eric Xing, and Zhiting Hu. RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pages 3369–3391. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.222. URL https://aclanthology.org/2022.emnlp-main.222/.

[253] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2309.08532.

[254] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 13481–13544. PMLR, 2024. URL https://proceedings.mlr.press/v235/fernando24a.html.

[255] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. In Advances in Neural Information Processing Systems, 2024. URL https://arxiv.org/abs/2406.16218.

[256] Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference. In Findings of the Association for Computational Linguistics: EMNLP 2026, 2026. URL https://arxiv.org/abs/2606.05922. Accepted for publication.

[257] Sungho Park, Wonjoong Kim, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, and Victor Rühle. ActiveSaddler: Automated Curriculum Learning for Agent Harness Optimization. arXiv preprint arXiv:2610.00906, 2026. URL https://arxiv.org/abs/2610.00906.

[258] Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams. arXiv preprint arXiv:2606.01770, 2026. URL https://arxiv.org/abs/2606.01770.

[259] Zhengxi Lu, Fei Tang, Guangyi Liu, Jin Ma, Kaitao Song, Xu Tan, Wenqi Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. UI-Copilot: Advancing Long-Horizon GUI Automation via Tool-Integrated Policy Optimization. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 19741–19762. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.904. URL https://aclanthology.org/2026.acl-long.904/.

[260] Zhiyuan He, Siwei Zhang, Zhiwen Zhou, Yuqing Yang, Yu Kang, Yuge Zhang, Luna K. Qiu, Tin Yan Tsui, Jiahang Xu, and Chong Luo. Agent Lightning v1.0: Towards Harnessed Agentic RL. arXiv preprint arXiv:2608.17528, 2026. URL https://arxiv.org/abs/2608.17528.

[261] Dilara Soylu, Christopher Potts, and Omar Khattab. Fine-Tuning and Prompt Optimization: Two Great Steps that Work Better Together. In Conference on Empirical Methods in Natural Language Processing, 2024. doi: 10.18653/v1/2024.emnlp-main.597. URL https://arxiv.org/abs/2407.10930.

[262] Rishabh Tiwari, Kusha Sareen, Lakshya A Agrawal, Joseph E. Gonzalez, Matei Zaharia, Kurt Keutzer, Inderjit S Dhillon, Rishabh Agarwal, and Fnu Devvrit. Learning, Fast and Slow: Towards LLMs That Adapt Continually. In COLM 2026 Workshop on Context Beyond the Window: Persistent Knowledge in Language Models, 2026. URL https://openreview.net/forum?id=3lOz4x7YcZ.

[263] Zhengyu Chen, Teng Xiao, Huaisheng Zhu, Yige Yuan, Luan Zhang, and Jingang Wang. Co-Harness: Co-Evolving Harnesses and Model Weights for LLM Agents. arXiv preprint arXiv:2607.22688, 2026. URL https://arxiv.org/abs/2607.22688.

[264] Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. UI-Mem: Self-Evolving Experience Memory for Online Reinforcement Learning in Mobile GUI Agents. arXiv preprint arXiv:2602.05832, 2026. URL https://arxiv.org/abs/2602.05832.

[265] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/156011. Accepted for publication.

[266] Junhao Shen, Teng Zhang, Xiaoyan Zhao, and Hong Cheng. Dynamic skill lifecycle management for agentic reinforcement learning. arXiv preprint arXiv:2605.10923, 2026. URL https://arxiv.org/abs/2605.10923.

[267] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self Improving AI with Harness & Weight Updates. arXiv preprint arXiv:2605.27276, 2026. URL https://arxiv.org/abs/2605.27276.

[268] Wei Zhang, Hongji Li, Song Sun, Peng Yu, Xue Yang, Lei Zhao, and Peng Jiang. DMRL: Document-Mediated Reinforcement Learning for Skill Optimization in Advertising Recommendation. arXiv preprint arXiv:2609.02170, 2026. URL https://arxiv.org/abs/2609.02170.

[269] Chuan Xiao, Zhengbo Jiao, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, Linfeng Zhang, and Lin Qu. Socratic-SWE: Self-Evolving Coding Agents via Trace-Derived Agent Skills. arXiv preprint arXiv:2606.07412, 2026. URL https://arxiv.org/abs/2606.07412.

[270] Zekai Wang, Yingqiang Ge, Zekun Wang, Hai Wang, Yuhui Xu, Joshua Frandsen, Shancong Fu, Ashia C. Wilson, and Chandan K. Reddy. VERSE: Verified self-evolving optimizer for agent harnesses. arXiv preprint arXiv:2610.02616, 2026. URL https://arxiv.org/abs/2610.02616.

[271] Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, Shao-Hua Sun, and Hung-yi Lee. Joint Optimization of Tool Creation and Use for Large Language Model Agents. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/155013. Accepted for publication.

[272] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelovic, and Jakob Nikolas Kather. LLM agents making agent tools. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 26092–26130, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1266. URL https://aclanthology.org/2025.acl-long.1266/.

[273] Shirley Wu, Parth Sarthi, Shiyu Zhao, Aaron Lee, Herumb Shandilya, Adrian Mladenic Grobelnik, et al. Optimas: Optimizing Compound AI Systems with Globally Aligned Local Rewards. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2507.03041.

[274] Beining Wu, Zihao Ding, and Jun Huang. Not All Experience Belongs in the Weights: Component Routing for Self-Improving GUI Agents. arXiv preprint arXiv:2610.01787, 2026. URL https://arxiv.org/abs/2610.01787.

[275] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents. In International Conference on Learning Representations, pages 52989–53046, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/e9df36b21ff4ee211a8b71ee8b7e9f57-Paper-Conference.pdf.

[276] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In Advances in Neural Information Processing Systems, volume 37, pages 74325–74362, 2024. doi: 10.52202/079017-2365. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/877b40688e330a0e2a3fc24084208dfa-Abstract-Datasets_and_Benchmarks_Track.html.

[277] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a Generalist Agent for the Web. In Advances in Neural Information Processing Systems, volume 36, pages 28091–28114, 2023. doi: 10.52202/075280-1220. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/5950bf290a1570ea401bf98882128160-Abstract-Datasets_and_Benchmarks.html.

[278] Xing Han Lu, Zdeněk Kasner, and Siva Reddy. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 33007–33056. PMLR, 2024. URL https://proceedings.mlr.press/v235/lu24e.html.

[279] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simon Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, Natalie Kim, Samuel Miserendino, Gildas Chabot, David Li, Patrick Chao, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks. In International Conference on Learning Representations, pages 24005–24040, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/290c2430f91912204f30bbcc990fff1d-Paper-Conference.pdf.

[280] Liya Zhu, Xin Ma, Tao Liu, et al. StartupBench: Benchmarking General-Purpose Agents on Market-Validated End-to-End Workflows. arXiv preprint, 2026. URL https://arxiv.org/abs/2608.17800.

[281] Elaine Lau, Markus Dücker, Ronak Chaudhary, Hui Wen Goh, Rosemary Wei, Vaibhav Kumar, Saed Qunbar, Guram Gogia, Yi Liu, Scott Millslagle, Nasim Borazjanizadeh, Ulyana Tkachenko, Samuel Eshun Danquah, Collin Schweiker, Vijay Karumathil, Asrith Devalaraju, Andrew P. Martin, Varsha Sandadi, Haemi Nam, Punit Arani, Ray Epps, Abdullah Arif, Sahil Bhaiwala, Curtis Northcutt, Skyler Wang, Anish Athalye, Jonas Mueller, and Francisco Guzmán. BankerToolBench: Evaluating AI Agents in End-to-End Investment Banking Workflows. In Advances in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139094. Accepted for publication.

[282] Yuhao Zhang, O. Ozan Koyluoglu, Thejas Venkatesh, Richard Diehl Martinez, Vishank Bhatia, Arash Alidoust, and Ashwin Paranjape. FrontierFinance: A Challenging Benchmark for Measuring Frontier Intelligence of Finance Agents. arXiv preprint, 2026. URL https://arxiv.org/abs/2608.11683.

[283] Haozhe Zhang, Kaichen Liu, Miaomiao Chen, Lei Li, Shaojie Yang, Cheng Peng, and Hanjie Chen. BenchCAD: A Comprehensive, Industry-Standard Benchmark for Programmatic CAD. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.10865.

[284] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 881–905. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.50. URL https://aclanthology.org/2024.acl-long.50/.

[285] Yiyou Sun, Xinyang Han, Weichen Zhang, et al. Agents' Last Exam. arXiv preprint arXiv:2606.05405, 2026. URL https://arxiv.org/abs/2606.05405. Accepted at NeurIPS 2026.

[286] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? In International Conference on Learning Representations, pages 32597–32649, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/50e9ad960ae78b741a6b4fea533f2eaf-Abstract-Conference.html.

[287] Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. DiscoveryBench: Towards Data-Driven Discovery with Large Language Models. In International Conference on Learning Representations, pages 4556–4579, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/0d70af566e69f1dfb687791ecf955e28-Abstract-Conference.html.

[288] Mantas Mazeika, Alice Gatti, Cristina Menghini, Udari Madhushani Sehwag, Shivam Singhal, Yury Orlovskiy, Steven Basart, Manasi Sharma, Denis Peskoff, Elaine Lau, Jaehyuk Lim, Lachlan Carroll, Alice Blair, Vinaya Sivakumar, Sumana Basu, Brad Kenstler, Yuntao Ma, Julian Michael, Xiaoke Li, Oliver Ingebretsen, Aditya Mehta, Jean Mottola, John Teichmann, Kevin Yu, Zaina Shaik, Adam Khoja, Richard Ren, Jason Hausenloy, Long Phan, Ye Htet, Ankit Aich, Tahseen Rabbani, Vivswan Shah, Andriy Novykov, Felix Binder, Kirill Chugunov, Luis Ramirez, Matias Geralnik, Hernán Mesura, Dean Lee, Ed-Yeremai Hernandez Cardona, Annette Diamond, Summer Yue, Alexandr Wang, Bing Liu, Ernesto Hernandez, and Dan Hendrycks. Remote labor index: Measuring ai automation of remote work. arXiv preprint arXiv:2510.26787, 2025. URL https://arxiv.org/abs/2510.26787.

[289] Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. Finance Agent Benchmark: Benchmarking LLMs on Real-world Financial Research Tasks. arXiv preprint arXiv:2508.00828, 2025. URL https://arxiv.org/abs/2508.00828.

[290] Vals AI. Finance Agent v2: Leaderboard and Methodology, 2026. URL https://www.vals.ai/benchmarks/fabv2. Accessed 17 October 2026.

[291] Harvey AI. Harvey LAB: The Legal Agent Benchmark, 2026. URL https://github.com/harveyai/harvey-labs/tree/v1.0. Accessed 17 October 2026.

[292] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Alan Huang, Songyang Zhang, Kai Chen, Zhixin Yin, Zongwen Shen, Jidong Ge, and Vincent Ng. LawBench: Benchmarking Legal Knowledge of Large Language Models. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 7933–7962. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.452. URL https://aclanthology.org/2024.emnlp-main.452/.

[293] Vals AI. Legal Research Bench: Leaderboard and Methodology, n.d.. URL https://www.vals.ai/benchmarks/legal_research. Accessed 17 October 2026.

[294] Qianchu Liu, Sheng Zhang, Guanghui Qin, et al. HealthAgentBench: A Unified Benchmark Suite of Realistic Agentic Healthcare Environments for Challenging Frontier AI Agents. arXiv preprint arXiv:2606.31179, 2026. URL https://arxiv.org/abs/2606.31179.

[295] Vals AI. MedCode: Leaderboard and Methodology, n.d.. URL https://www.vals.ai/benchmarks/medcode. Accessed 17 October 2026.

[296] Vals AI. MedScribe: Leaderboard and Methodology, n.d.. URL https://www.vals.ai/benchmarks/medscribe. Accessed 17 October 2026.

[297] Artificial Analysis. Medical Long Context Reasoning (MLCR-AA), 2026. URL https://artificialanalysis.ai/evaluations/mlcr-aa. Accessed 17 October 2026.

[298] Keqi Han, Ryan Young, Annabel Strauss, et al. PSEBench: A Controllable and Verifiable Benchmark for Evaluating LLMs in Patient Safety Event Triage. arXiv preprint arXiv:2606.05463, 2026. URL https://arxiv.org/abs/2606.05463.

[299] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. MMLONGBENCH-DOC: Benchmarking Long-context Document Understanding with Visualizations. In Advances in Neural Information Processing Systems, volume 37, pages 95963–96010, 2024. doi: 10.52202/079017-3041. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ae0e43289bffea0c1fa34633fc608e92-Abstract-Datasets_and_Benchmarks_Track.html.

[300] Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. arXiv preprint arXiv:2508.09124, 2025. URL https://arxiv.org/abs/2508.09124.

[301] Baidu Frontier Research. OmegaUse-OfficeVal: Benchmarking LLM Agents on Long-Horizon Office-Suite Tasks with Economic Grounding, 2026. URL https://huggingface.co/datasets/baidu-frontier-research/OmegaUse-OfficeVal. Accessed 17 October 2026.

[302] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios and Interactions. Transactions on Machine Learning Research, 2026. URL https://openreview.net/forum?id=EPlpe3Fx1x.

[303] Wenjie Fu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Lukas Wutschitz, Robert Sim, Saravan Rajmohan, and Dongmei Zhang. CI-Work: Benchmarking Contextual Integrity in Enterprise LLM Agents. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), pages 1483–1508. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-industry.103. URL https://aclanthology.org/2026.acl-industry.103/.

[304] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. Can LLMs Help You at Work? A Sandbox for Evaluating LLM Agents in Enterprise Environments. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 9167–9201. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.466. URL https://aclanthology.org/2025.emnlp-main.466/.

[305] Jun Takahashi, Atsunori Moteki, Akiyoshi Uchida, Shoichi Masui, Fan Yang, Kanji Uchino, Yueqi Song, Yonatan Bisk, Graham Neubig, Ikuo Kusajima, Yasuto Watanabe, Hiroyuki Ishida, Koki Nakagawa, and Shan Jiang. FieldWorkArena: Agentic AI Benchmark for Real Field Work Tasks. In Proceedings of the 28th International Conference on Pattern Recognition, 2026. URL https://arxiv.org/abs/2505.19662. Accepted for publication.

[306] Kaiyuan Chen, Qimin Wu, Taiyu Hou, et al. AgentIF-OneDay: A Task-level Instruction-Following Benchmark for General AI Agents in Daily Scenarios. arXiv preprint arXiv:2601.20613, 2026. URL https://arxiv.org/abs/2601.20613.

[307] Chaithanya Bandi, Razvan-Gabriel Dumitru, Ben Hertzberg, Divyansh Agarwal, Geobio Boo, Tejas Polakam, Jeff Da, HiJae Kim, Vipul Gupta, Manasi Sharma, Andrew Park, Martin Villanueva Dimakis, Ernesto Gabriel Hernandez Montoya, Rafael Cruz, MohammadHossein Rezaei, Chetan Rane, Daniel Yue Zhang, Brad Kenstler, and Bing Liu. MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers. In Advances

in Neural Information Processing Systems, 2026. URL https://neurips.cc/virtual/2026/poster/139650. Accepted for publication.

[308] Peijie Yu, Yifan Yang, Jinjian Li, Zelong Zhang, Haorui Wang, Xiao Feng, and Feng Zhang. Multi-Mission Tool Bench: Assessing the Robustness of LLM based Agents through Related and Dynamic Missions. arXiv preprint arXiv:2504.02623, 2025. URL https://arxiv.org/abs/2504.02623.

[309] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench : Evaluating the Collaboration and Competition of LLM agents. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 8580–8622. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.421. URL https://aclanthology.org/2025.acl-long.421/.

[310] Lawrence Keunho Jang, Jing Yu Koh, Daniel Fried, and Ruslan Salakhutdinov. Odysseys: Benchmarking Web Agents on Realistic Long Horizon Tasks. arXiv preprint arXiv:2604.24964, 2026. URL https://arxiv.org/abs/2604.24964.

[311] Shuangrui Ding, Xuanlang Dai, Long Xing, et al. WildClawBench: A Benchmark for Real-World, Long-Horizon Agent Evaluation. arXiv preprint arXiv:2605.10912, 2026. URL https://arxiv.org/abs/2605.10912.

[312] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Keunho Jang, and Zheng Hui. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 4874–4910. PMLR, 2025. URL https://proceedings.mlr.press/v267/bonatti25a.html.

[313] Yang Liu, Jiaqi Li, Jun Bai, Qianyu Yang, Zixia Jia, Tiliang Duan, Chun Zhang, Jiayun Dong, Lingyue Yin, Jianpeng Jiao, Yanglihong Xiao, Zaiyuan Wang, Tao Peng, Xiaobo Hu, Kaiyuan Chen, Ge Zhang, Gang Yao, Hao Chen, and Zilong Zheng. $OneMillion-Bench: How Far are Language Agents from Human Experts? In Advances in Neural Information Processing Systems (Evaluations and Datasets Track), 2026. URL https://neurips.cc/virtual/2026/poster/139822. Accepted for publication.

[314] Pattaraphon Kenny Wongchamcharoen, Kris Gulati, Min Min Fong, and Abhishek Nagaraj. CentaurBench: Benchmarking LLM Capabilities on Augmenting vs. Automating Real-World Work Tasks. arXiv preprint arXiv:2608.18554, 2026. URL https://arxiv.org/abs/2608.18554.

[315] Shihan Dou, Ming Zhang, Zhangyue Yin, et al. CL-bench: A Benchmark for Context Learning. arXiv preprint arXiv:2602.03587, 2026. URL https://arxiv.org/abs/2602.03587.

[316] JobBench. JobBench: Professional Task Dataset, n.d. URL https://huggingface.co/datasets/JobBench/job-bench. Accessed 17 October 2026.

[317] Chengyu Shen, Yujie Fu, Gangtao Xin, et al. OmniaBench: Benchmarking General AI Agents Across Diverse Scenarios. arXiv preprint, 2026. URL https://arxiv.org/abs/2607.14989.

[318] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://aclanthology.org/2023.emnlp-main.153/.

[319] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, et al. Prometheus: Inducing Fine-grained Evaluation Capability in Language Models. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2310.08491.

[320] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The BrowserGym Ecosystem for Web Agent Research. Transactions on Machine Learning Research, 2025. URL https://openreview.net/forum?id=5298fKGmv3. Expert Certification.

[321] Harbor contributors. Harbor: A Framework for Evaluating and Improving Agents, 2026. URL https://harborframework.com/. Accessed 21 October 2026.

[322] UK AI Security Institute. Inspect: A Framework for Large Language Model Evaluations, 2026. URL https://inspect.aisi.org.uk/. Accessed 21 October 2026.

[323] Earendil Works. Pi: A Minimal, Extensible Coding Agent Harness, 2026. URL https://github.com/earendil-works/pi. Accessed 21 October 2026.

[324] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2510.11977.

[325] Yilun Yao et al. Harness-Bench: Evaluating Agent Harnesses across Models and Realistic Workflows. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.27922.

[326] Melissa Z. Pan, Shuo Yang, Negar Arabzadeh, Wei-Lin Chiang, Ion Stoica, and Matei Zaharia. HarnessTax: How Much Does Harness Matter for Coding Agents?, 2026. URL https://harnesstax.github.io/.

[327] Chenyang Yang, Xinran Zhao, Tongshuang Wu, and Christian Kästner. Better Harnesses, Smaller Models: Building 90% Cheaper Agents via Automated Harness Adaptation. arXiv preprint, 2026. URL https://arxiv.org/abs/2607.08938.

[328] Zora Zhiruo Wang et al. How Well Does Agent Development Reflect Real-World Work? arXiv preprint, 2026. URL https://arxiv.org/abs/2603.01203.

[329] Weiwen Liu, Jiarui Qin, Xu Huang, Xingshan Zeng, Yunjia Xi, Jianghao Lin, Chuhan Wu, Yasheng Wang, Lifeng Shang, Ruiming Tang, Defu Lian, Yong Yu, and Weinan Zhang. Position: The Real Barrier to LLM Agent Usability is Agentic ROI. arXiv preprint arXiv:2505.17767v2, 2026. URL https://arxiv.org/abs/2505.17767v2.

[330] Veronica Chatrath et al. READY or Not: Reliable Enterprise Agent Deployment. arXiv preprint, 2026. URL https://arxiv.org/abs/2609.02095.

[331] Raja Parasuraman and Victor Riley. Humans and Automation: Use, Misuse, Disuse, Abuse. Human Factors, 39 (2):230–253, 1997. doi: 10.1518/001872097778543886. URL https://doi.org/10.1518/001872097778543886.

[332] Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting LLM agents from reactive responses to active assistance. In International Conference on Learning Representations, 2025. URL https://arxiv.org/abs/2410.12361.

[333] Jio Oh, Seunghyun Do, Young-Jun Lee, Steven Euijong Whang, and Dongyeop Kang. Foundations of proactive agents: Principles, technical layers, and Proactivity-Gym. arXiv preprint arXiv:2609.37267, 2026. URL https://arxiv.org/abs/2609.37267.

[334] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. PAST-Bench: Benchmarking the Foundations of Recursive Self-Improvement in Personal Agents. arXiv preprint arXiv:2608.04003, 2026. URL https://arxiv.org/abs/2608.04003.

[335] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks. arXiv preprint arXiv:2411.04468, 2024. URL https://arxiv.org/abs/2411.04468.

[336] Anthropic. Introducing Claude Duet, October 2026. URL https://claude.com/resources/articles/claude-duet. Published October 21, 2026; accessed October 22, 2026.

[337] Abdul Mannan Mohammed, Azhar Ali Mohammad, Jason A. Ortiz, Carsten Neumann, Grace Bochenek, Dirk Reiners, and Carolina Cruz-Neira. A Human Digital Twin Architecture for Knowledge-based Interactions and Context-Aware Conversations. In Proceedings of the Interservice/Industry Training, Simulation, and Education Conference (I/ITSEC), 2024. URL https://arxiv.org/abs/2504.03147. Paper No. 24366.

[338] Joon Sung Park, Carolyn Q. Zou, Jonne Kamphorst, Niles Egan, Aaron Shaw, Benjamin Mako Hill, Carrie Cai, Meredith Ringel Morris, Percy Liang, Robb Willer, and Michael S. Bernstein. LLM Agents Grounded in Self-Reports Enable General-Purpose Simulation of Individuals. arXiv preprint arXiv:2411.10109v3, 2026. URL https://arxiv.org/abs/2411.10109v3.

[339] Bangde Du, Minghao Guo, Songming He, Ziyi Ye, Xi Zhu, Weihang Su, Shuqi Zhu, Yujia Zhou, Yongfeng Zhang, Qingyao Ai, and Yiqun Liu. TwinVoice: A Multi-dimensional Benchmark Towards Digital Twins via LLM Persona Simulation. In Findings of the Association for Computational Linguistics: ACL 2026, pages 19604–19628, 2026. doi: 10.18653/v1/2026.findings-acl.981. URL https://aclanthology.org/2026.findings-acl.981/.

[340] Pengze Li, Yutong Hu, Jianfu Li, Garit Gemeinhardt, Fang Li, Muhammad Amith, Licong Cui, Antonio J. Forte, and Cui Tao. Developing provider digital twins for personalized provider-patient communication via a RAG-based conversational framework. Computational and Structural Biotechnology Journal, 32:24–33, 2026. doi: 10.1016/j.csbj.2025.12.007. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC12809031/.

[341] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. arXiv preprint arXiv:2501.09674, 2025. URL https://arxiv.org/abs/2501.09674.

[342] Yian Wang, Agam Goyal, Eshwar Chandrasekharan, and Hari Sundaram. Facts without rules: Boundary metadata collapse in multi-agent LLM handoffs. arXiv preprint arXiv:2608.29028, 2026. URL https://arxiv.org/abs/2608.29028.

[343] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In Advances in Neural Information Processing Systems, 2025. URL https://arxiv.org/abs/2503.13657.

[344] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a Science of Scaling Agent Systems. arXiv preprint arXiv:2512.08296, 2025. URL https://arxiv.org/abs/2512.08296.

[345] Gary Klein, David D. Woods, Jeffrey M. Bradshaw, Robert R. Hoffman, and Paul J. Feltovich. Ten challenges for making automation a "team player in joint human-agent activity. IEEE Intelligent Systems, 19(6):91–95, 2004. doi: 10.1109/MIS.2004.74.

[346] Yijia Shao, Humishka Zope, Yucheng Jiang, Jiaxin Pei, David Nguyen, Erik Brynjolfsson, and Diyi Yang. Future of Work with AI Agents: Auditing Automation and Augmentation Potential across the U.S. Workforce. arXiv preprint arXiv:2506.06576, 2025. URL https://arxiv.org/abs/2506.06576v3.

[347] National Center for O*NET Development. The O*NET content model. URL https://www.onetcenter.org/content.html. Accessed 13 October 2026.

[348] Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. GPTs are GPTs: Labor market impact potential of LLMs. Science, 384(6702):1306–1308, 2024. doi: 10.1126/science.adj0998. URL https://doi.org/10.1126/science.adj0998.

[349] Xuhui Zhou, Weiwei Sun, Qianou Ma, Yiqing Xie, Jiarui Liu, Weihua Du, Sean Welleck, Yiming Yang, Graham Neubig, Sherry Tongshuang Wu, and Maarten Sap. Mind the Sim2Real gap in user simulation for agentic tasks. In Conference on Language Modeling, 2026. URL https://arxiv.org/abs/2603.11245.

[350] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. ComputerRL: Scaling End-to-End Online Reinforcement Learning for Computer Use Agents. In International Conference on Learning Representations, pages 23553–23591, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/28707388472adc117714e83d44bfaebb-Paper-Conference.pdf.

[351] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. arXiv preprint arXiv:2508.03680, 2025. URL https://arxiv.org/abs/2508.03680.

[352] Jialiang Huang, Hongxuan Tang, Jingchang Chen, et al. DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale. arXiv preprint arXiv:2609.22978, 2026. URL https://arxiv.org/abs/2609.22978.

[353] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training Software Engineering Agents and Verifiers with SWE-Gym. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 47717–47737. PMLR, 2025. URL https://proceedings.mlr.press/v267/pan25g.html.

[354] Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural Environment Generation and Hybrid Verifiers for Scaling Open-Weights SWE Agents. In Second Conference on Language Modeling, 2025. URL https://openreview.net/forum?id=7evvwwdo3z.

[355] Weiyi Xu, Xiaowen Yang, Wen Da, Hang Xu, Canwei Li, Hongjie You, Pusen Dong, Yucheng Zeng, Zhaokai Luo, and Mu Chuan. Skill2Env: Capability-Oriented Environment Synthesis from Skills for General Agents. arXiv preprint arXiv:2609.33772, 2026. URL https://arxiv.org/abs/2609.33772.

[356] Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, Zuxuan Wu, Qi Zhang, Xuanjing Huang, and Yu-Gang Jiang. AgentGym-RL: Training LLM Agents for Long-Horizon Decision Making through Multi-Turn Reinforcement Learning. arXiv preprint arXiv:2509.08755, 2025. URL https://arxiv.org/abs/2509.08755.

[357] Md Tawkat Islam Khondaker, Raymond Li, Muhammad Abdul-Mageed, Laks V. S. Lakshmanan, and Issam H. Laradji. DRFLOW: A Deep Research Benchmark for Personalized Workflow Prediction, 2026. URL https://arxiv.org/abs/2606.18191. Preprint; arXiv; institutional page reports EMNLP 2026; verified 2026-10-16.

[358] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025. URL https://arxiv.org/abs/2501.13956.

[359] Yongye Su, Wujiang Xu, Chaoji Zuo, and Elisa Bertino. ChronoMem: Version control and semantic rollback for large language model agent memory. arXiv preprint arXiv:2607.27773, 2026. URL https://arxiv.org/abs/2607.27773.

[360] Prateek Chhikara, Dev Khant, Saket Aryan, et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv preprint, 2025. URL https://arxiv.org/abs/2504.19413.

[361] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, pages 1–22. ACM, 2023. doi: 10.1145/3586183.3606763. URL https://doi.org/10.1145/3586183.3606763.

[362] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In Advances in Neural Information Processing Systems, 2025. URL https://arxiv.org/abs/2502.12110.

[363] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, et al. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. In Annual Meeting of the Association for Computational Linguistics, 2026. URL https://arxiv.org/abs/2508.19828.

[364] Guanming Liu, Yuqi Ren, Hansu Gu, Peng Zhang, Weihang Wang, Jiahao Liu, Ning Gu, and Tun Lu. StreamMem-Bench: Streaming Evaluation of Agent Memory for Future-Oriented Assistance. In Findings of the Association for Computational Linguistics: EMNLP 2026, 2026. URL https://arxiv.org/abs/2606.14571. Accepted for publication.

[365] In Gim, Seung seob Lee, and Lin Zhong. Asynchronous LLM function calling. arXiv preprint arXiv:2412.07017, 2024. URL https://arxiv.org/abs/2412.07017.

[366] Fangru Lin, Emanuele La Malfa, Valentin Hofmann, Elle Michelle Yang, Anthony Cohn, and Janet B. Pierrehumbert. Graph-enhanced large language models in asynchronous plan reasoning. In International Conference on Machine Learning, 2024. URL https://arxiv.org/abs/2402.02805.

[367] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM Agents Are Experiential Learners. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 19632–19642. Association for the Advancement of Artificial Intelligence (AAAI), 2024. doi: 10.1609/aaai.v38i17.29936. URL https://doi.org/10.1609/aaai.v38i17.29936.

[368] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. Transactions on Machine Learning Research, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

[369] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. In Proceedings of the 42nd International Conference on Machine Learning, volume 267 of Proceedings of Machine Learning Research, pages 63897–63911. PMLR, 2025. URL https://proceedings.mlr.press/v267/wang25bx.html.

[370] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stanczak, Peter Shaw, Christopher Pal, and Siva Reddy. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In Second Conference on Language Modeling, 2025. URL https://openreview.net/forum?id=fQcUZMPIvu.

[371] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems, volume 36, pages 46595–46623, 2023. doi: 10.52202/075280-2020. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

[372] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In Advances in Neural Information Processing Systems, 2024. URL https://arxiv.org/abs/2404.13076.

[373] Shashwat Goel, Joschka Struber, Ilze Amanda Auzina, Karuna K. Chandra, Ponnurangam Kumaraguru, Douwe Kiela, Ameya Prabhu, Matthias Bethge, and Jonas Geiping. Great models think alike and this undermines AI oversight. In International Conference on Machine Learning, 2025. URL https://arxiv.org/abs/2502.04313.

[374] Isabel Corpus, Eric Gilbert, Allison Koenecke, and Mor Naaman. Introducing AI to an online petition platform changed outputs but not outcomes. Nature Human Behaviour, 2026. doi: 10.1038/s41562-026-02580-8. URL https://www.nature.com/articles/s41562-026-02580-8.

[375] Wenle Liao, Zhao Wang, Jingchao Zhang, Jiajie Jin, Yimeng Xu, and Zhicheng Dou. StateGuard: Analytical-state management with validity-aware intervention for long-horizon data agents. arXiv preprint arXiv:2609.34134, 2026. URL https://arxiv.org/abs/2609.34134.

[376] Wil M. P. van der Aalst, Mathias Weske, and Dolf Grünbauer. Case handling: A new paradigm for business process support. Data & Knowledge Engineering, 53(2):129–162, 2005. doi: 10.1016/j.datak.2004.07.003.

[377] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve LLM agent reliability under non-atomic failures. arXiv preprint arXiv:2608.02645, 2026. URL https://arxiv.org/abs/2608.02645.

[378] Pat Helland. Life beyond distributed transactions. ACM Queue, 14(5):69–98, 2016. doi: 10.1145/3012426.3025012.

[379] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. Foundations and Trends in Databases, 1(4):379–474, 2009. doi: 10.1561/1900000006.

[380] Hector Garcia-Molina and Kenneth Salem. Sagas. In Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 249–259, 1987. doi: 10.1145/38713.38742.

[381] Edward Y. Chang and Longling Geng. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. Proceedings of the VLDB Endowment, 18(12):4874–4886, 2025. doi: 10.14778/3750601.3750611. URL https://doi.org/10.14778/3750601.3750611.

[382] Bardia Mohammadi, Nearchos Potamitis, Lars Henning Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. In ICLR 2026 Workshop on Agentic AI in the Wild: From Hallucinations to Reliable Autonomy, 2026. URL https://openreview.net/forum?id=gele0qdZXT.

[383] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In International Conference on Machine Learning, 2023. URL https://arxiv.org/abs/2210.10760.

[384] Yiming Pan, Chengwei Hu, Xuancheng Huang, et al. AeSlides: Incentivizing Aesthetic Layout in LLM-Based Slide Generation via Verifiable Rewards. arXiv preprint, 2026. URL https://arxiv.org/abs/2604.22840.

[385] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Who Verifies the Verifier? Co-Evolving Inspectable Graders with Self-Improving Agents. In NeurIPS 2026 Workshop on Who Verifies the Agents? Toward Reliable Agent Development, 2026. URL https://arxiv.org/abs/2610.11464. Accepted for publication.

[386] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In Advances in Neural Information Processing Systems, 2017. URL https://arxiv.org/abs/1706.08840.

[387] Jialong Li and Jialing Zhu. Auditing self-evolution in financial agents: Capability gains, security drift, and execution-interface mismatch. arXiv preprint arXiv:2608.17684, 2026. URL https://arxiv.org/abs/2608.17684.

[388] Fabrizio Dell'Acqua, Charles Ayoubi, Hila Lifshitz, Raffaella Sadun, Ethan Mollick, Lilach Mollick, Yi Han, Jeff Goldman, Hari Nair, Stewart Taub, and Karim Lakhani. The cybernetic teammate: A field experiment on generative AI reshaping teamwork and expertise. Working Paper 33641, National Bureau of Economic Research, 2025. URL https://www.nber.org/papers/w33641.

[389] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 ai on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. URL https://arxiv.org/abs/2507.09089.

[390] Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. Science, 381(6654):187–192, 2023. doi: 10.1126/science.adh2586.

[391] Erik Brynjolfsson, Daniel Rock, and Chad Syverson. The productivity J-curve: How intangibles complement general purpose technologies. American Economic Journal: Macroeconomics, 13(1):333–372, 2021. doi: 10.1257/mac.20180386.

[392] Erik Brynjolfsson. The Productivity Paradox of Information Technology. Communications of the ACM, 36(12):66–77, 1993. doi: 10.1145/163298.163309. URL https://doi.org/10.1145/163298.163309.

[393] Lisanne Bainbridge. Ironies of automation. Automatica, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8.